

C.9 DEZASAMBLARE ȘI PATCHING CU X96DBG

PAUL A. GAGNIUC



Academia Tehnică Militară „Ferdinand I”

PRINCIPALELE PĂRȚI ALE PREZENTĂRII

C.9 Dezasamblare și Patching cu X64dbg:

- 9.1 DEZASAMBLARE ȘI PETICIRE (CALL VS JMP)
- 9.2 EXECUȚIA DE COD ÎN SECȚIUNEA .DATA
- 9.3 MODIFICAREA STRINGURILOR ÎN OBIECTE
- 9.4 MODIFICAREA STRINGURILOR ÎN MESAJE
- 9.5 SCHIMBAREA MASINII VIRTUALE



9.1

DEZASAMBLARE ȘI PETICIRE (CALL VS JMP)



1. Fișierul mic.exe este încărcat și oprit la adresa Entry Point (00402000), unde se observă începutul execuției: instrucțiuni standard de inițializare. Aceasta este zona de start a codului propriu al programului.

mic.exe - PID: 16472 - Module: mic.exe - Thread: Main Thread 10072 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads

00402000 6A 00 push 0
00402002 68 00104000 push mic.401000
00402004 68 12104000 push mic.401012
00402006 6A 00 push 0
00402008 FF15 80304000 call dword ptr ds:[<ExitProcess>]
0040200A 6A 00 push 0
0040200C FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040200E 0000 add byte ptr ds:[eax],al
00402010 0000 add byte ptr ds:[eax],al
00402012 0000 add byte ptr ds:[eax],al
00402014 0000 add byte ptr ds:[eax],al
00402016 0000 add byte ptr ds:[eax],al
00402018 0000 add byte ptr ds:[eax],al
0040201A 0000 add byte ptr ds:[eax],al
0040201C 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
00402020 0000 add byte ptr ds:[eax],al
00402022 0000 add byte ptr ds:[eax],al
00402024 0000 add byte ptr ds:[eax],al
00402026 0000 add byte ptr ds:[eax],al
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al
00402066 0000 add byte ptr ds:[eax],al
00402068 0000 add byte ptr ds:[eax],al

EAX 0000FFFF
EBX 00343000
ECX 00402000
EDX 00402000
ESI 00402000
EDI 00402000
EIP 00402000

EFLAGS 00000244
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 000036B7 (ERROR_SXS_KEY)
LastStatus C0150008 (STATUS_SXS_KEY)

GS 002B FS 0053
ES 002B DS 002B
CS 002B SS 002B

ST(0) 00000000000000000000000000000000 x87r0 E
ST(1) 00000000000000000000000000000000 x87r1 E
ST(2) 00000000000000000000000000000000 x87r2 E
ST(3) 00000000000000000000000000000000 x87r3 E
ST(4) 00000000000000000000000000000000 x87r4 E
ST(5) 00000000000000000000000000000000 x87r5 E
ST(6) 00000000000000000000000000000000 x87r6 E
ST(7) 00000000000000000000000000000000 x87r7 E

X87TagWord FFFF
v077W 0.2 (Empty) v077W 1.2 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 00343000 00343000
2: [esp+8] 75E6FCB0 <kernel32.BaseThreadInitThunk>
3: [esp+C] 0000FFDC 0000FFDC
4: [esp+10] 77D382AE ntdll.77D382AE
5: [esp+14] 00343000 00343000

.text:00402000 mic.exe:\$2000 #400 <OptionalHeader.AddressOfEntryPoint>

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x] [v]

Address Hex ASCII

77CD1000 16 00 18 00 80 7E CD 77 14 00 16 00 E0 7C CD 77Iw...Iw
77CD1010 00 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD1020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD1030 06 00 08 00 70 7E CD 77 06 00 08 00 88 7E CD 77p.Iw...Iw
77CD1040 06 00 08 00 78 7E CD 77 06 00 08 00 88 7E CD 77X.Iw...Iw
77CD1050 1C 00 1E 00 14 7D CD 77 20 00 22 00 18 82 CD 77Iw...Iw
77CD1060 84 00 86 00 80 84 CD 77 C0 6C D0 77 80 47 DD 77 8F Yw.GYwXBw.GYw
77CD1070 80 84 CD 77 C0 45 D0 77 50 20 D0 77 50 6A D0 77 8F Yw.GYwXBw.GYw
77CD1080 40 46 DD 77 00 47 DD 77 80 47 DD 77 80 47 DD 77 P&WPJW.FYw.GYw
77CD1090 50 26 D0 77 50 6A D0 77 00 46 DD 77 00 47 DD 77 B0W.GYw.....
77CD10A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD10B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD10C0 EE E3 D3 F0 06 00 00 00 9C 7C CD 77 01 00 00 00 1a0b.....Iw
77CD10D0 9A 88 13 35 96 5D BD 4F 8E 2D A2 44 02 25 F9 3AS.]yO.-d.Nu
77CD10E0 06 00 01 00 80 7C CD 77 02 00 00 00 E3 28 2F 4AIw...&(/J
77CD10F0 B9 53 41 44 BA 9C D6 90 4A 4A 6E 38 06 00 02 00 *SAD'.O.JjN8...
77CD1100 64 7C CD 77 03 00 00 00 76 6C 67 1F E1 80 39 42 dIw...vlg.a.98
77CD1110 95 8B 83 D0 F6 D0 DA 78 06 00 03 00 48 7C CD 77 -.D&Dux...H]Iw

Command: Default

Paused [INT3 breakpoint "entry breakpoint" at <mic.OptionalHeader.AddressOfEntryPoint> (00402000)] Time Wasted Debugging: 0:11:13:56

2. Se selectează manual blocul de cod dorit.
3. Click dreapta pe blocul selectat, de unde se alege opțiunea „Binary”.
4. Se apasă „Copy” pentru a salva codul în format binar.

mic.exe - PID: 16472 - Module: mic.exe - Thread: Main Thread 10072 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads

00402000 6A 00 push 0
00402002 68 00104000 push mic.401000
00402004 68 12104000 push mic.401012
00402006 6A 00 push 0
00402008 FF15 80304000 call dword ptr ds:[<ExitProcess>]
0040200A 6A 00 push 0
0040200C FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040200E 0000 add byte ptr ds:[eax],al
00402010 0000 add byte ptr ds:[eax],al
00402012 0000 add byte ptr ds:[eax],al
00402014 0000 add byte ptr ds:[eax],al
00402016 0000 add byte ptr ds:[eax],al
00402018 0000 add byte ptr ds:[eax],al
0040201A 0000 add byte ptr ds:[eax],al
0040201C 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
00402020 0000 add byte ptr ds:[eax],al
00402022 0000 add byte ptr ds:[eax],al
00402024 0000 add byte ptr ds:[eax],al
00402026 0000 add byte ptr ds:[eax],al
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al
00402066 0000 add byte ptr ds:[eax],al
00402068 0000 add byte ptr ds:[eax],al

EAX 0000FFFF
EBX 00343000
ECX 00402000
EDX 00402000
ESI 00402000
EDI 00402000
EIP 00402000

EFLAGS 00000244
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 000036B7 (ERROR_SXS_KEY)
LastStatus C0150008 (STATUS_SXS_KEY)

GS 002B FS 0053
ES 002B DS 002B
CS 002B SS 002B

ST(0) 00000000000000000000000000000000 x87r0 E
ST(1) 00000000000000000000000000000000 x87r1 E
ST(2) 00000000000000000000000000000000 x87r2 E
ST(3) 00000000000000000000000000000000 x87r3 E
ST(4) 00000000000000000000000000000000 x87r4 E
ST(5) 00000000000000000000000000000000 x87r5 E
ST(6) 00000000000000000000000000000000 x87r6 E
ST(7) 00000000000000000000000000000000 x87r7 E

X87TagWord FFFF
v077W 0.2 (Empty) v077W 1.2 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 00343000 00343000
2: [esp+8] 75E6FCB0 <kernel32.BaseThreadInitThunk>
3: [esp+C] 0000FFDC 0000FFDC
4: [esp+10] 77D382AE ntdll.77D382AE
5: [esp+14] 00343000 00343000

.text:00402000 mic.exe:\$2000 #400 <OptionalHeader.AddressOfEntryPoint>

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x] [v]

Address Hex ASCII

77CD1000 16 00 18 00 80 7E CD 77 14 00 16 00 E0 7C CD 77Iw...Iw
77CD1010 00 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD1020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD1030 06 00 08 00 70 7E CD 77 06 00 08 00 88 7E CD 77p.Iw...Iw
77CD1040 06 00 08 00 78 7E CD 77 06 00 08 00 88 7E CD 77X.Iw...Iw
77CD1050 1C 00 1E 00 14 7D CD 77 20 00 22 00 18 82 CD 77Iw...Iw
77CD1060 84 00 86 00 80 84 CD 77 C0 6C D0 77 80 47 DD 77 8F Yw.GYwXBw.GYw
77CD1070 80 84 CD 77 C0 45 D0 77 50 20 D0 77 50 6A D0 77 8F Yw.GYwXBw.GYw
77CD1080 40 46 DD 77 00 47 DD 77 80 47 DD 77 80 47 DD 77 P&WPJW.FYw.GYw
77CD1090 50 26 D0 77 50 6A D0 77 00 46 DD 77 00 47 DD 77 B0W.GYw.....
77CD10A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD10B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
77CD10C0 EE E3 D3 F0 06 00 00 00 9C 7C CD 77 01 00 00 00 1a0b.....Iw
77CD10D0 9A 88 13 35 96 5D BD 4F 8E 2D A2 44 02 25 F9 3AS.]yO.-d.Nu
77CD10E0 06 00 01 00 80 7C CD 77 02 00 00 00 E3 28 2F 4AIw...&(/J
77CD10F0 B9 53 41 44 BA 9C D6 90 4A 4A 6E 38 06 00 02 00 *SAD'.O.JjN8...
77CD1100 64 7C CD 77 03 00 00 00 76 6C 67 1F E1 80 39 42 dIw...vlg.a.98
77CD1110 95 8B 83 D0 F6 D0 DA 78 06 00 03 00 48 7C CD 77 -.D&Dux...H]Iw

Command: Default

Paused [mic.exe: 00402000 -> 00402013 (0x00000014 bytes)] Time Wasted Debugging: 0:11:14:58

Este prezentată prima linie din Entry Point în mic.exe!

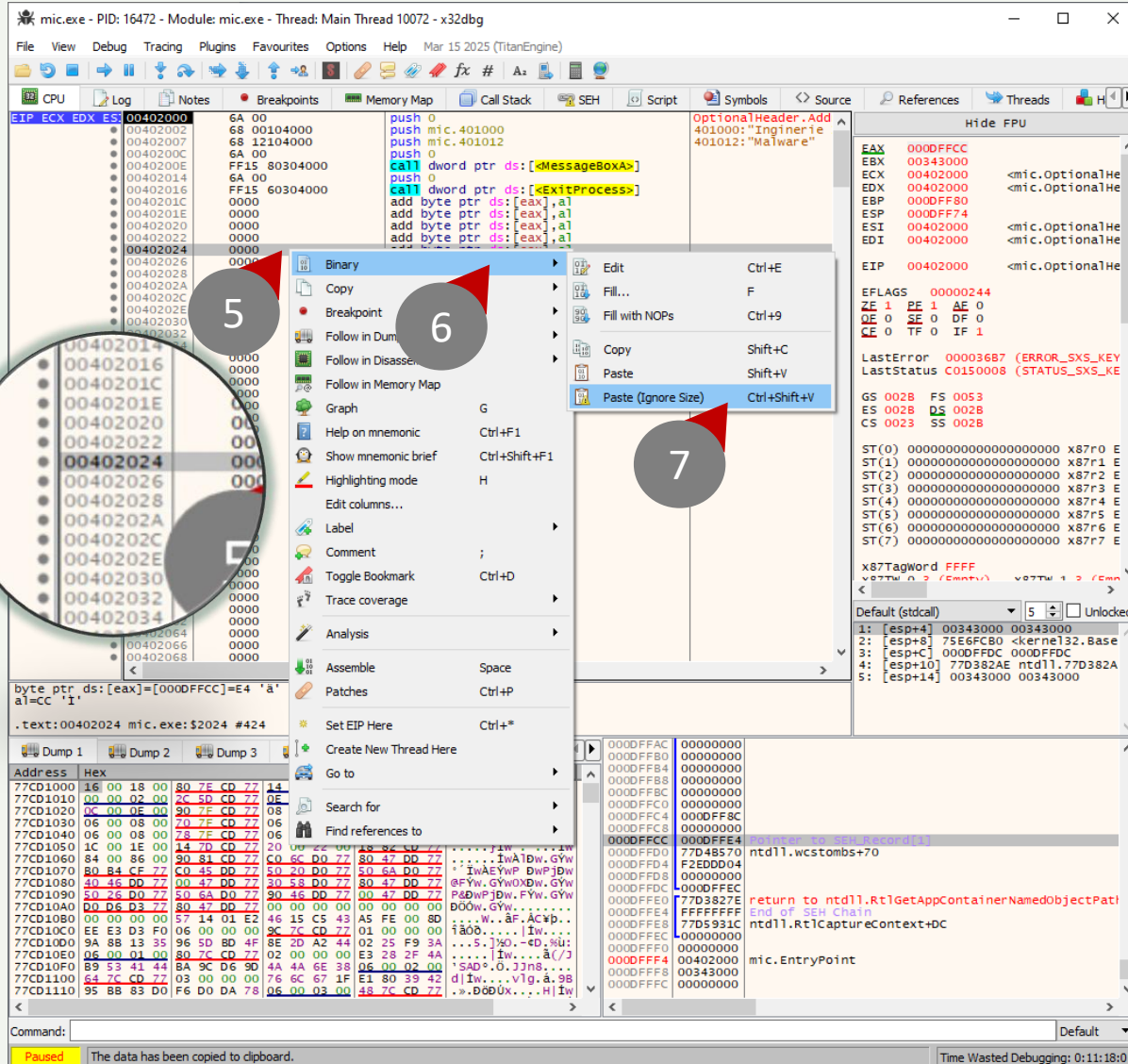
Selectăm o secțiune de cod universal și îl copiem!

5. Ne poziționăm într-o zonă liberă din secțiunea .text, după codul existent.

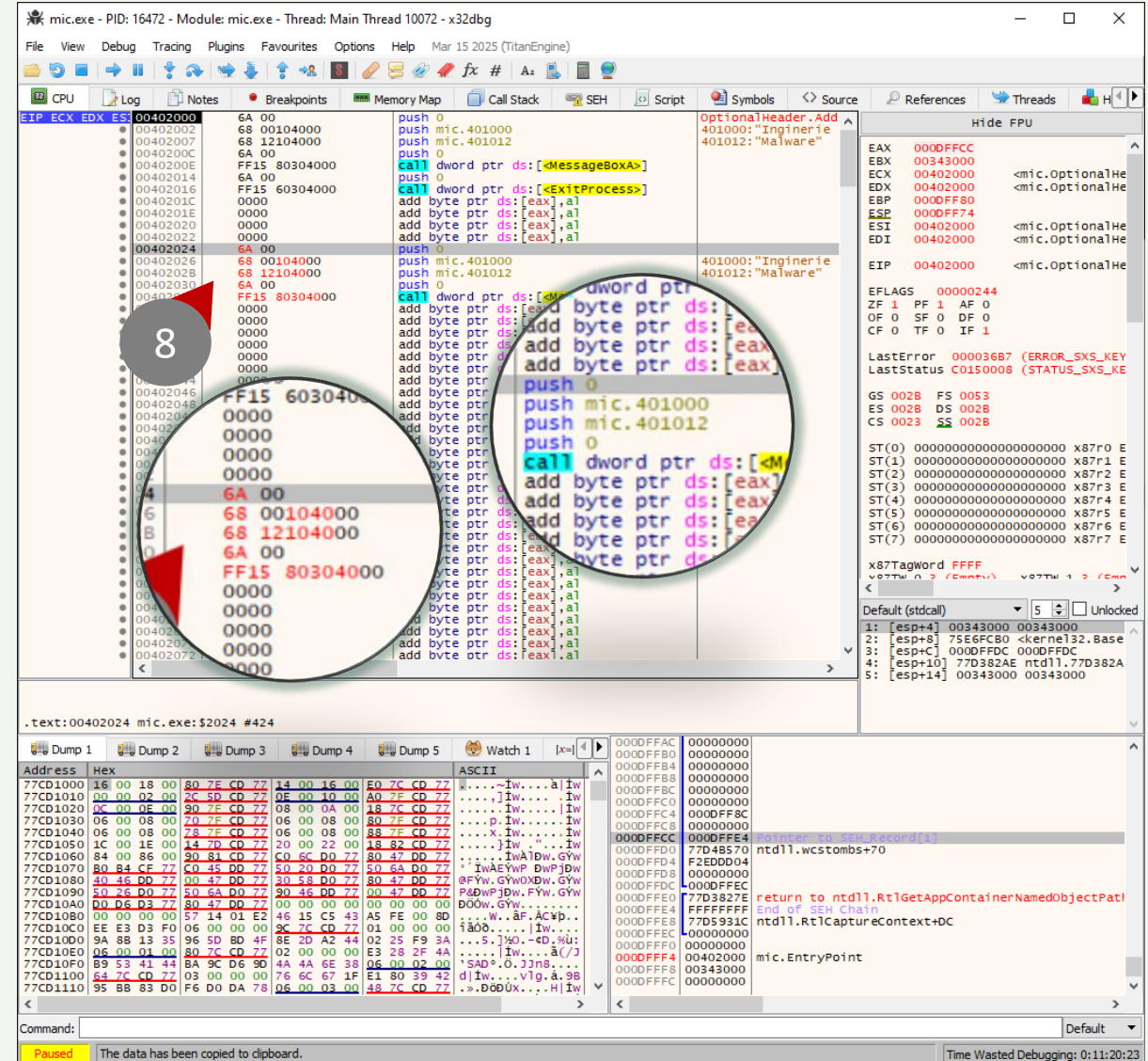
6. Click dreapta pentru a deschide meniul contextual.

7. Alegem opțiunea Paste (Ignore size) din submeniul „Binary” – pentru a lipi codul fără a ține cont de dimensiunea inițială a instrucțiunilor.

8. Codul selectat anterior este duplicat corect în regiunea aleasă, exact cum a fost copiat.



Apoi, codul copiat îl introducem într-o zonă liberă...



După apăsarea submeniului Paste...

mic.exe - PID: 16472 - Module: mic.exe - Thread: Main Thread 10072 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

EIP ECX EDX ES: 00402000 6A 00 00402002 68 00104000 push 0 00402007 68 12104000 push mic.401000 0040200C 6A 00 0040200E FF15 80304000 push mic.401012 00402014 6A 00 00402016 FF15 60304000 call dword ptr ds:[<MessageBox>] 0040201C 0000 0000 call dword ptr ds:[<ExitProcess>] 00402020 0000 0000 add byte ptr ds:[eax],al 00402022 0000 0000 add byte ptr ds:[eax],al 00402024 6A 00 0000 add byte ptr ds:[eax],al 00402026 0000 0000 push 0 401000:"Inginerie 401012:"Malware"

1 2 3

Binary Copy Selection Ctrl+C Selection to File Selection (Bytes only) Selection (No Bytes) Selection to File (No Bytes) Address Alt+Ins Disassembly

ST(0) 0000000000000000 x87r0 E
ST(1) 0000000000000000 x87r1 E
ST(2) 0000000000000000 x87r2 E
ST(3) 0000000000000000 x87r3 E
ST(4) 0000000000000000 x87r4 E
ST(5) 0000000000000000 x87r5 E
ST(6) 0000000000000000 x87r6 E
ST(7) 0000000000000000 x87r7 E

x87TagWord FFFF
v07TW 0 2 (Empty) v07TW 1 2 (Em
<

Default (stdcall) 5 Unlocked

1: [esp+4] 00343000 00343000
2: [esp+8] 75E6FCB0 <kernel32.Base
3: [esp+C] 000DFFDC 000DFFDC
4: [esp+10] 77D382AE ntdll.77D382A
5: [esp+14] 00343000 00343000

.text:00402024 mic.exe:\$20;

Dump 1 Dump 2

Address	Hex
77CD1000	16 00 18 00 80 7E
77CD1010	00 00 02 00 2C 50
77CD1020	0C 00 0E 00 90 7E
77CD1030	06 00 08 00 70 7E
77CD1040	06 00 08 00 78 7E
77CD1050	1C 00 1E 00 14 7D
77CD1060	84 00 86 00 80 81 CD 77 C0 6C D0 77 80 47 DD 77
77CD1070	80 B4 CF 77 C0 45 DD 77 50 20 D0 77 50 6A D0 77
77CD1080	40 46 DD 77 00 47 DD 77 30 58 D0 77 80 47 DD 77
77CD1090	50 26 D0 77 50 6A D0 77 90 46 DD 77 00 47 DD 77
77CD10A0	00 D6 D3 77 80 47 DD 77 00 00 00 00 00 00 00
77CD10B0	00 00 00 00 57 14 01 E2 46 15 C5 43 A5 FE 00 80
77CD10C0	EE E3 D3 F0 06 00 00 00 9C 7C CD 77 01 00 00 00
77CD10D0	9A 8B 13 35 96 5D BD 4F 8E 2D A2 44 02 25 F9 3A
77CD10E0	06 00 01 00 80 7C CD 77 E3 28 2F 4A 00 00 00 00
77CD10F0	B9 53 41 44 8A 9C D6 9D 4A 4A 6E 38 06 00 02 00
77CD1100	64 7C CD 77 03 00 00 00 76 6C 67 1F E1 80 39 42
77CD1110	95 BB 83 D0 F6 D0 DA 78 06 00 03 00 48 7C CD 77

Command: mic.exe: 00402024 -> 00402025 (0x00000002 bytes)

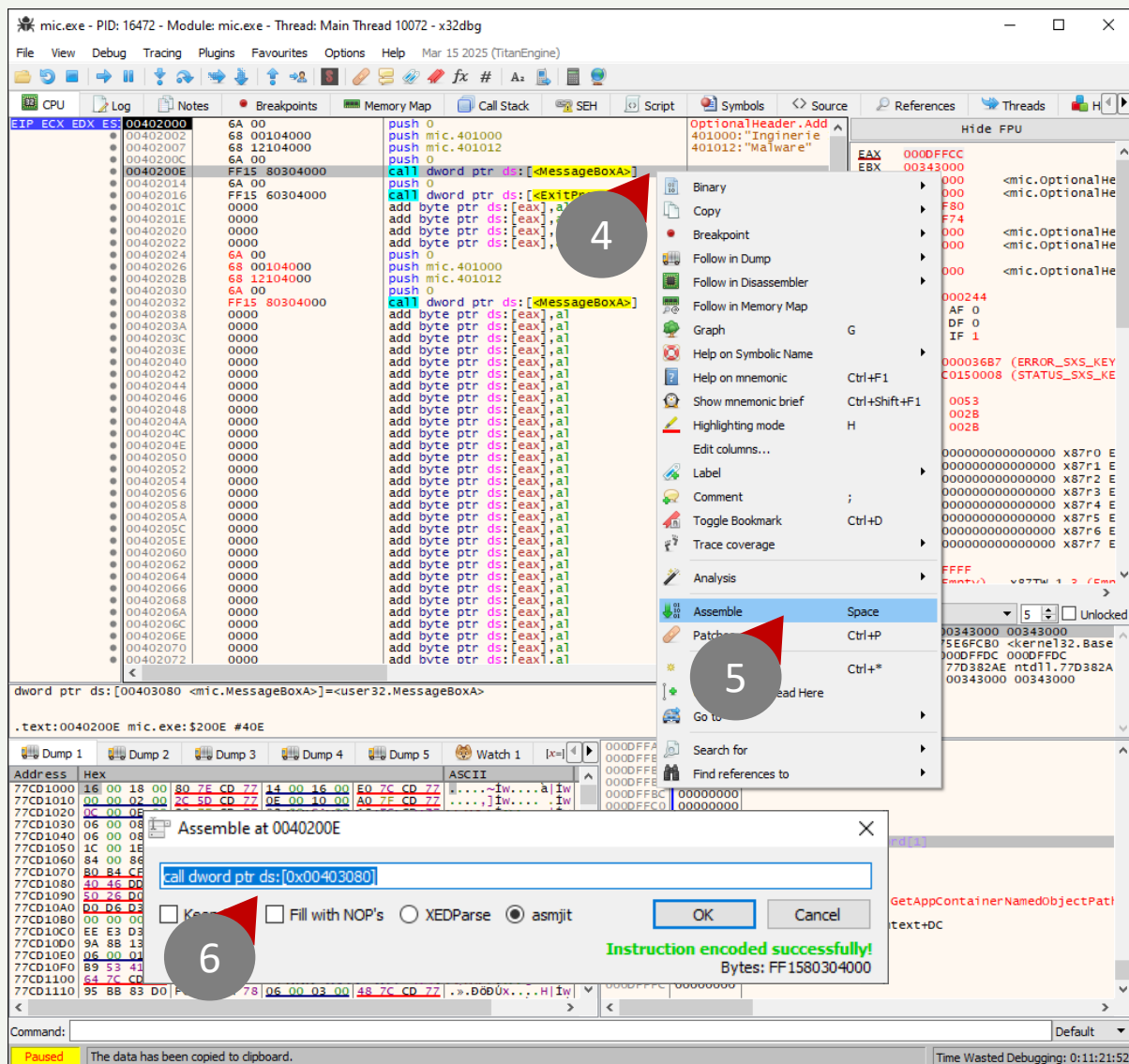
Time Wasted Debugging: 0:11:21:15

1. Facem click dreapta pe prima instructiune din codul duplicat (nou introdus).
2. Din meniul aplegem optiunea Copy.
3. Apoi selectam Address, pentru a copia adresa exacta la care incepe noua secventa de cod.

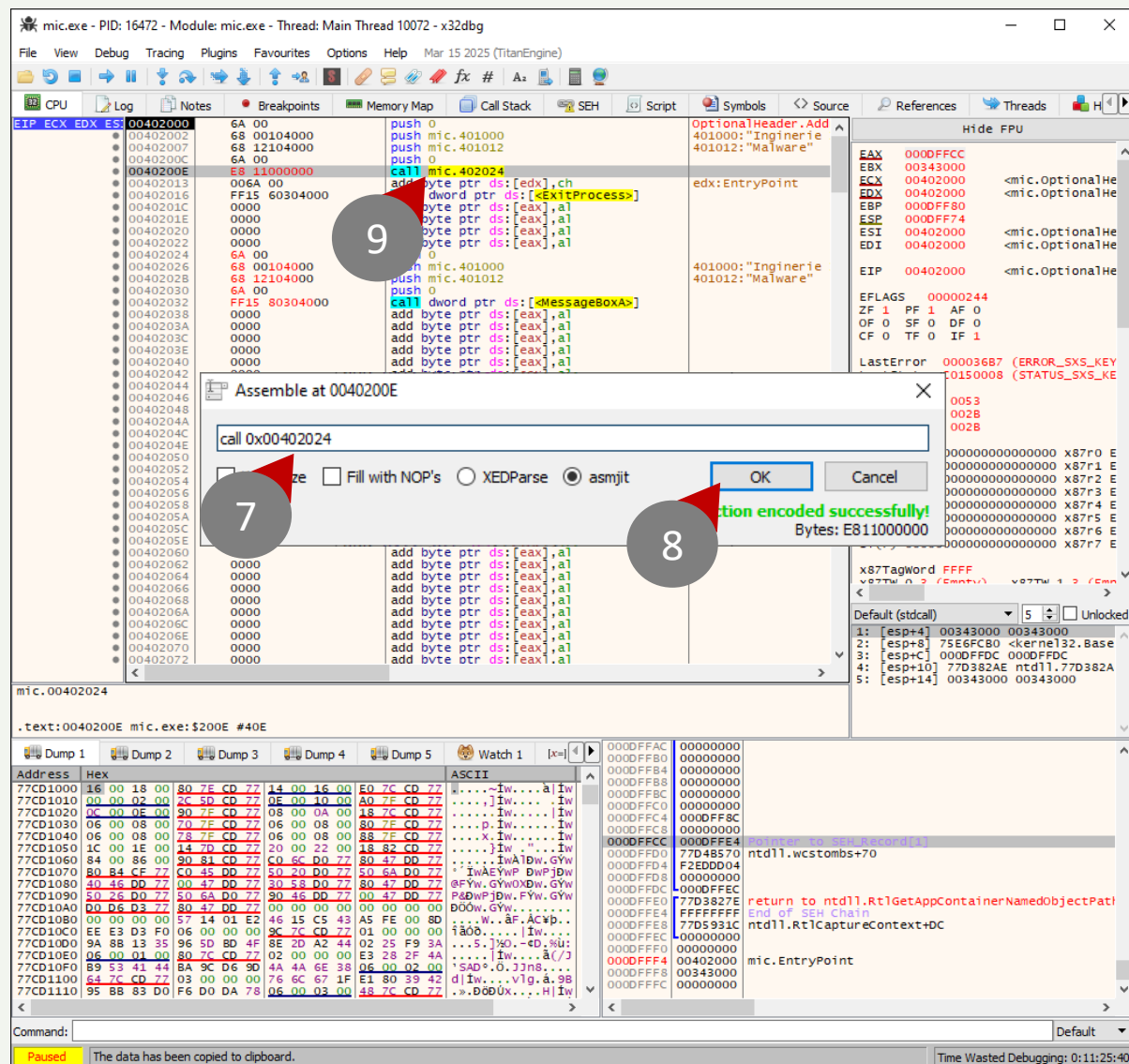
Copiem adresa de inceput a codului duplicat, dupa care vom folosi aceasta adresa pentru a redirectiona fluxul de executie. Astfel, programul va sari catre noul cod injectat.



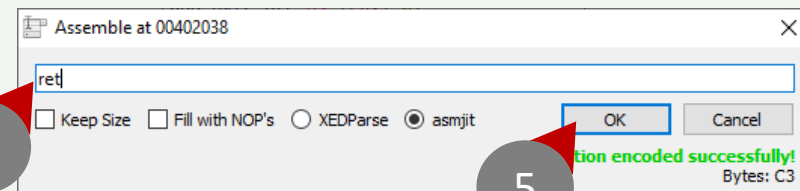
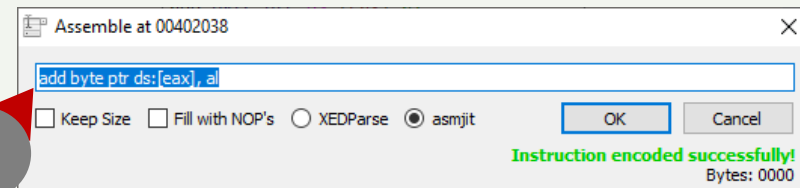
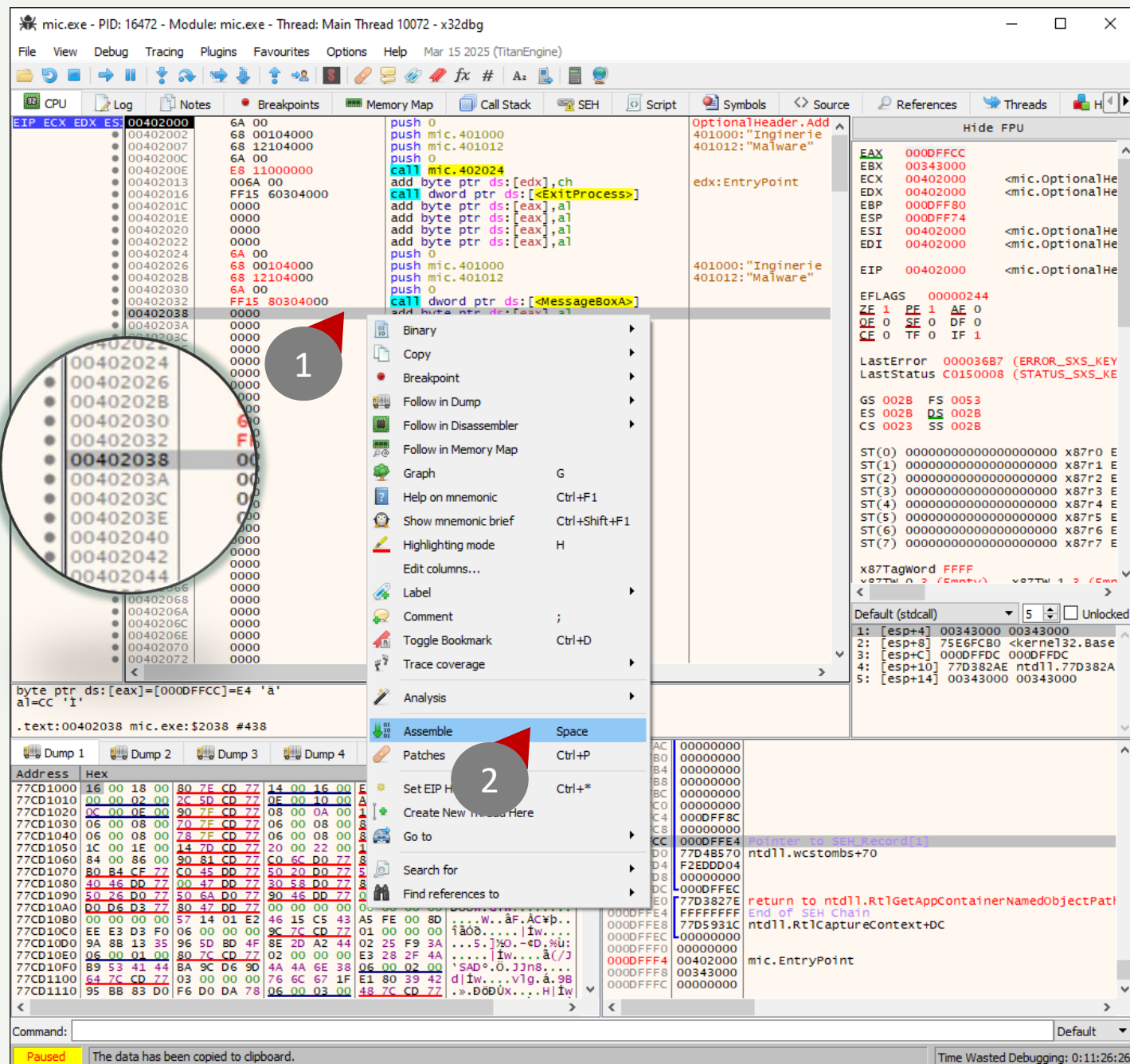
4. Se face click dreapta pe instrucțiunea CALL din codul original.
5. Se alege opțiunea Assemble pentru a edita instrucțiunea.
6. În fereastra de editare, se înlocuiește instrucțiunea (ex: call dword ptr ds:[0x00403080]) cu un call către adresa unde am copiat codul.



7. Se scrie adresa codului duplicat (ex: 0x00402024) în câmpul de editare.
8. Se apasă OK – instrucțiunea este validată și înlocuită cu succes.
9. Fluxul de execuție va fi redirecționat către codul introdus manual.

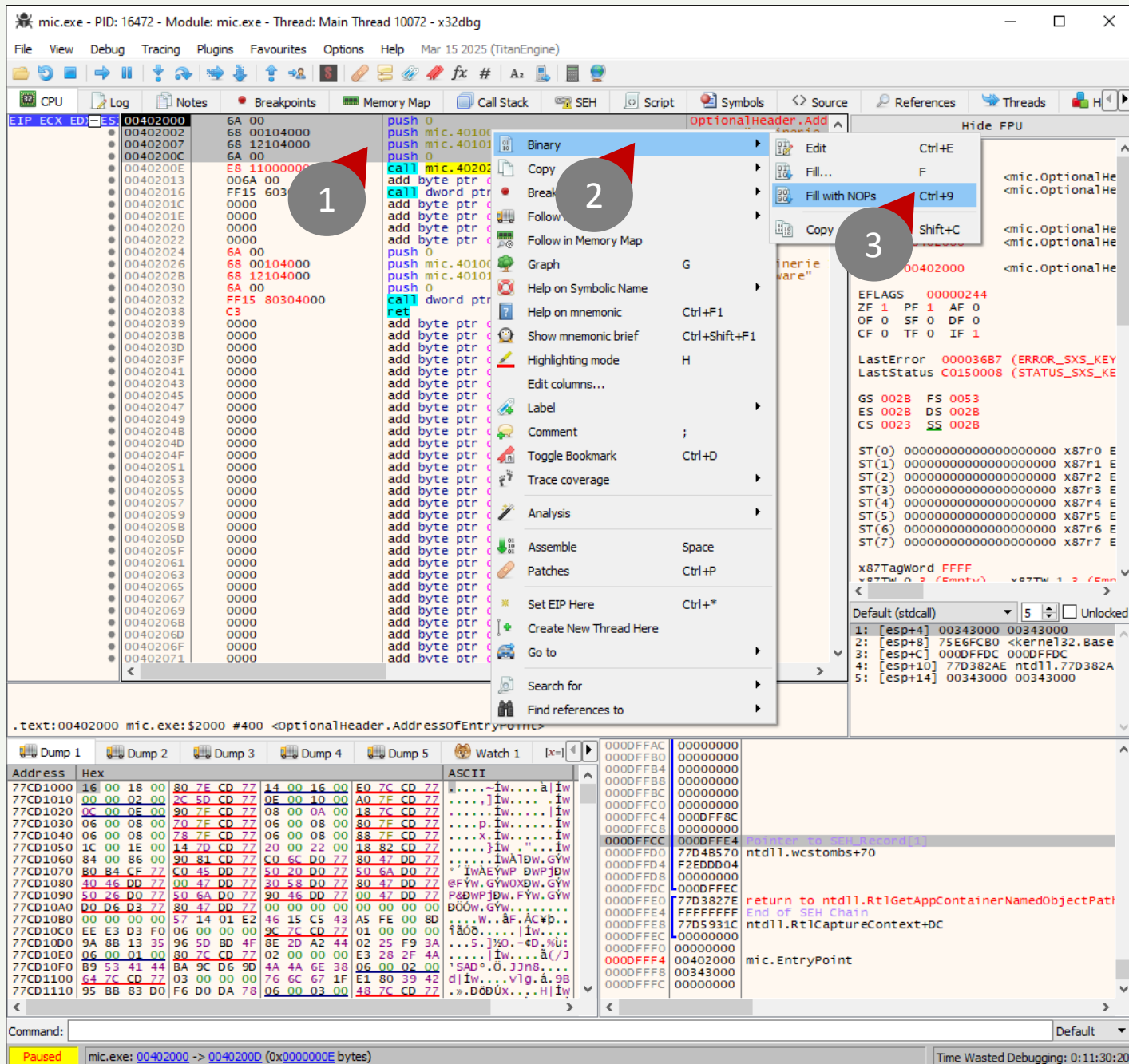


Valoarea 0x00403080 este adresa de sub eticheta "<MessageBoxA>" ...



1. Facem click dreapta pe adresa imediat după codul duplicat.
2. Alegem opțiunea Assemble pentru a o modifica.
3. Apare în clar valoarea de la acea adresă (ex: **add byte ptr ds:[eax], al**).
4. Instrucțiunea „**add byte ptr ds:[eax]**” este înlocuită cu **ret**, pentru a reveni la fluxul inițial de execuție.
5. Apăsăm OK pentru a salva modificarea.

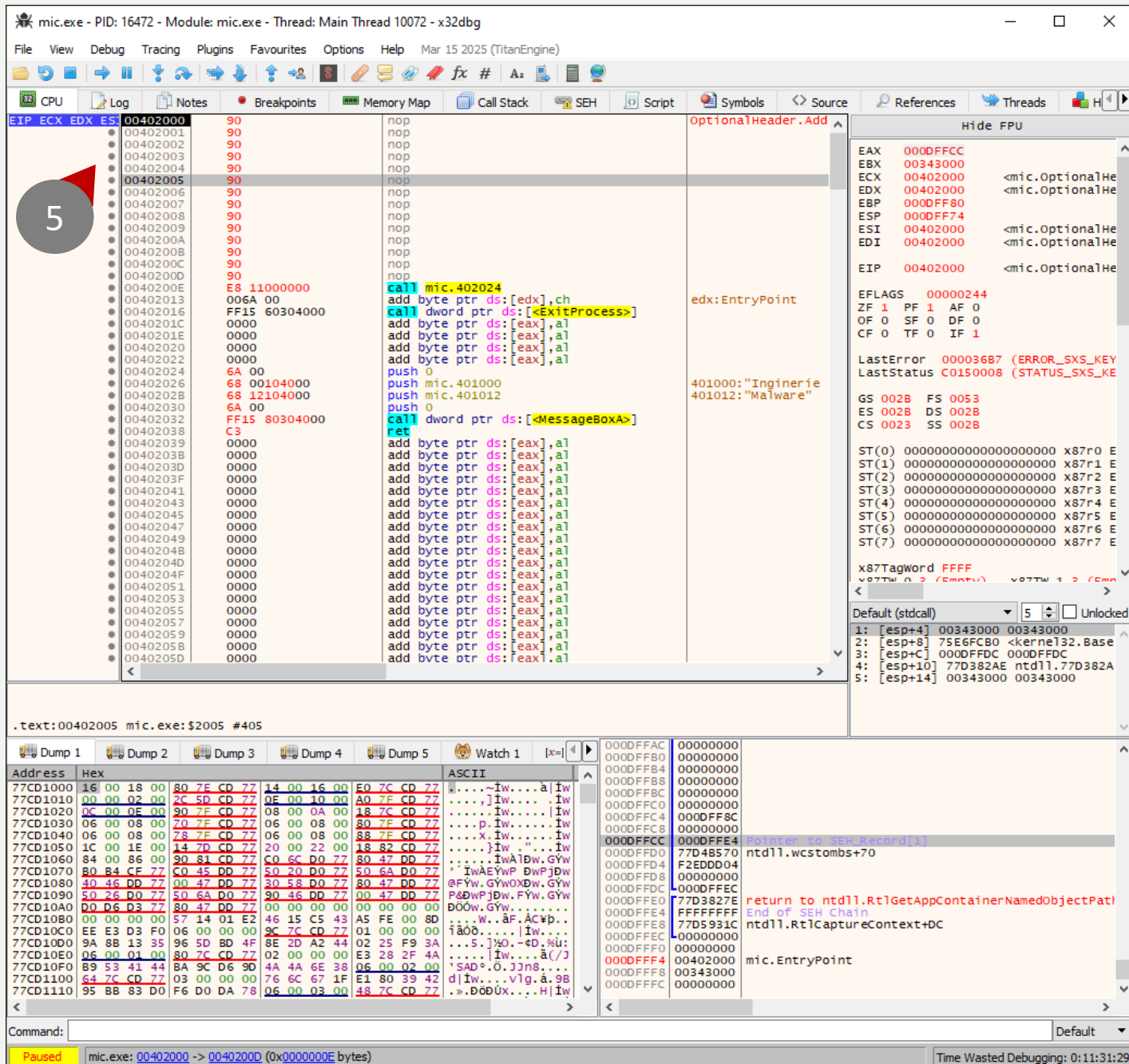
În acest pas final, ne asigurăm că execuția codului injectat se încheie corect. Adăugarea instrucțiunii **ret** permite revenirea la punctul de unde a fost făcut apelul (**call**), menținând stabilitatea programului.



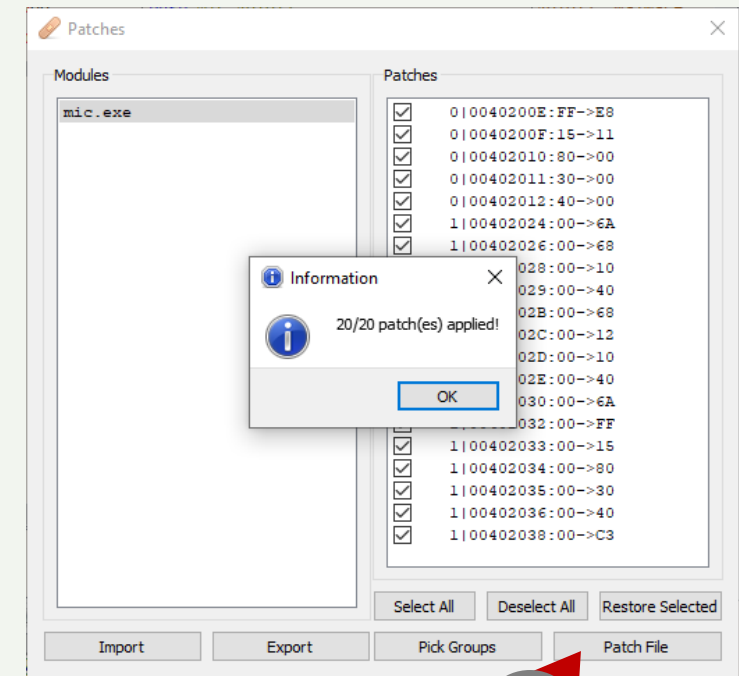
Selectarea și copierea blocului de cod

1. Se selectează zona de cod care trebuie neutralizată.
2. Click dreapta pe selecție → Binary.
3. Se alege Fill with NOPs pentru a umple blocul de cod cu instrucțiuni NOP (No Operation).

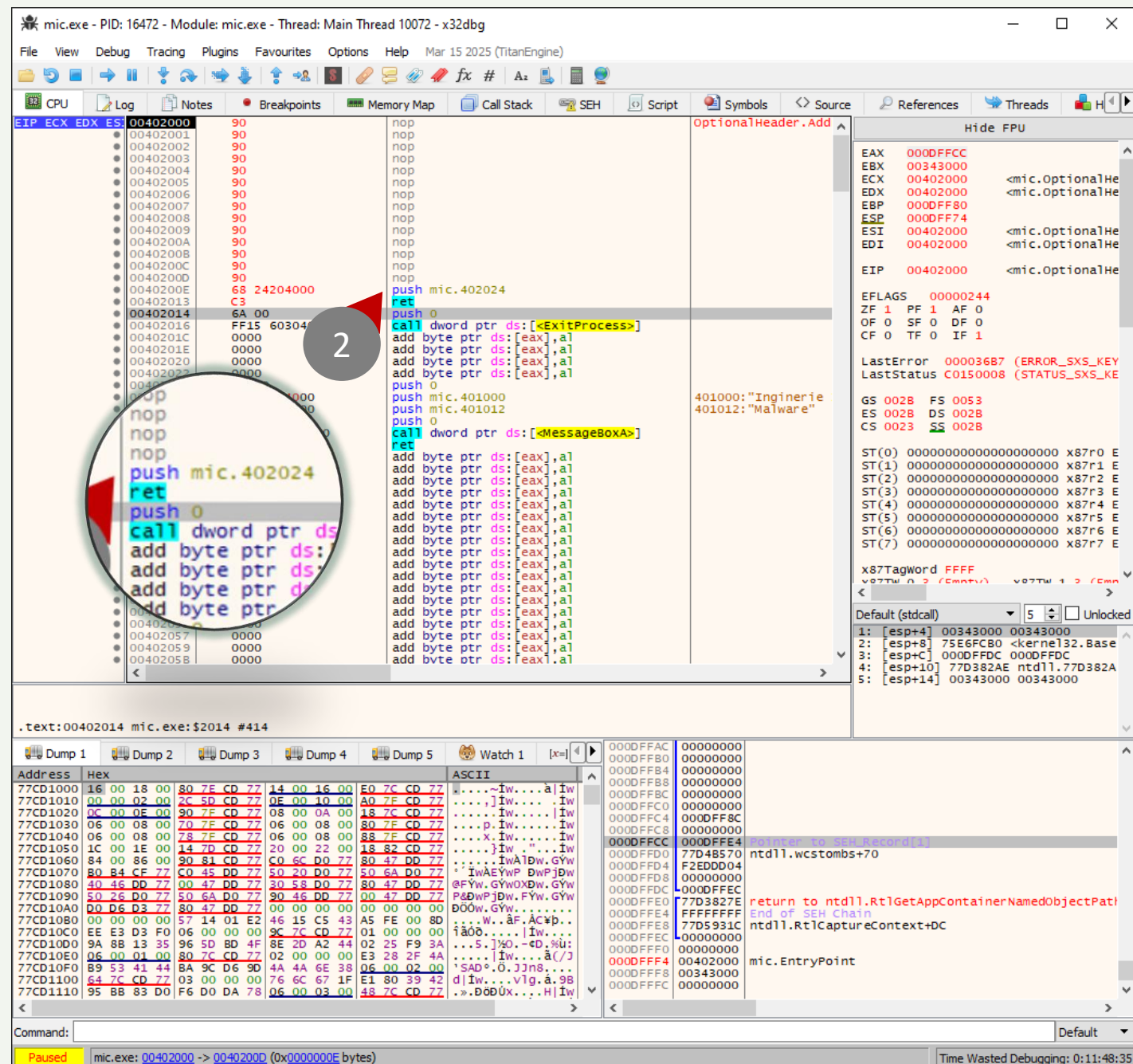
4. Apare fereastra „Size”, în care se confirmă dimensiunea blocului. Este afișată dimensiunea exactă (în bytes) a blocului selectat. Se confirmă cu OK, iar codul este înlocuit în memorie cu instrucțiuni NOP (0x90), care nu au niciun efect la execuție.



5. Toate modificările realizate în memoria procesului sunt vizibile, anume, codul injectat este activ, iar codul remanent a fost înlocuit cu instrucțiuni NOP. Aceasta este forma finală a codului care urmează a fi salvat în fișier.
6. Se aplică patch-urile în fișierul original folosind butonul Patch File.



Patch-ul a fost aplicat cu succes. Modificările făcute în memoria procesului sunt acum salvate permanent în fișierul mic.exe.



mic.exe - PID: 16472 - Module: mic.exe - Thread: Main Thread 10072 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols References Threads H

0040200C 90 nop
0040200D 90 nop
0040200E 68 24204000 push mic.402024
00402013 C3 ret
00402014 6A 00 push 0
00402016 FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040201C 0000 add ds:[eax],al
0040201E 0000 add ds:[eax],al
00402020 0000 add ds:[eax],al
00402022 0000 add ds:[eax],al
00402024 6A 00 push 0
00402026 68 00104000 push mic.401000
00402028 68 12104000 push mic.401012
00402030 6A 00 push 0
00402032 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 6A 00 push 0
00402041 68 00104000 push mic.401000
00402046 68 12104000 push mic.401012
00402048 6A 00 push 0
0040204D FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402053 90 nop
00402054 90 nop
00402055 90 nop
00402056 90 nop
00402057 90 nop
00402058 90 nop
00402059 6A 00 push 0
0040205B 68 00104000 push mic.401000
00402060 68 12104000 push mic.401012
00402065 6A 00 push 0
00402067 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
0040206D 90 nop
0040206E 90 nop
0040206F C3 ret
00402070 0000 add byte ptr ds:[eax],al
00402072 0000 add byte ptr ds:[eax],al
00402074 0000 add byte ptr ds:[eax],al
00402076 0000 add byte ptr ds:[eax],al
00402078 0000 add byte ptr ds:[eax],al

401000: "Inginerie
401012: "Malware"

401000: "Inginerie
401012: "Malware"

401000: "Inginerie
401012: "Malware"

Hide FPU

EAX 000DFFC
EBX 00343000
ECX 00402000 <mic.OptionalHe
EDX 00402000 <mic.OptionalHe
EBP 000FF80
ESP 000FF74
ESI 00402000 <mic.OptionalHe
EDI 00402000 <mic.OptionalHe
EIP 00402000 <mic.OptionalHe

EFLAGS 00000244
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 000036B7 (ERROR_SXS_KEY
LastStatus C0150008 (STATUS_SXS_KE

GS 0028 FS 0053
ES 0028 DS 0028
CS 0023 SS 0028

ST(0) 000000000000000000 x87r0 E
ST(1) 00000000000000000000 x87r1 E
ST(2) 00000000000000000000 x87r2 E
ST(3) 00000000000000000000 x87r3 E
ST(4) 00000000000000000000 x87r4 E
ST(5) 00000000000000000000 x87r5 E
ST(6) 00000000000000000000 x87r6 E
ST(7) 00000000000000000000 x87r7 E

x87TagWord FFFF
x87TW 0 2 (Empty) x87TW 1 2 (Em

Default (stdcall) 5 Unlocked

1: [esp+4] 00343000 00343000
2: [esp+8] 75E6FCB0 <kernel32.Base
3: [esp+C] 000DFFC 000DFFC
4: [esp+10] 77D382AE ntdll.77D382A
5: [esp+14] 00343000 00343000

mic.00402024
.text:0040200E mic.exe:\$200E #40E

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x=]

Address Hex ASCII

77CD1000 16 00 18 00 80 7E CD 77 14 00 16 00 E0 7C CD 77~Iw...aIw
77CD1010 00 00 02 00 2C 5D CD 77 0E 00 10 00 A0 7E CD 77Iw...Iw
77CD1020 0C 00 0E 00 90 7E CD 77 08 00 0A 00 18 7C CD 77Iw...Iw
77CD1030 06 00 08 00 70 7F CD 77 06 00 08 00 80 7E CD 77p.Iw...Iw
77CD1040 0C 00 08 00 78 7F CD 77 06 00 08 00 88 7E CD 77x.Iw...Iw
77CD1050 1C 00 1E 00 14 7D CD 77 20 00 22 00 18 82 CD 77}Iw...Iw
77CD1060 84 00 86 00 80 81 CD 77 C0 6C D0 77 80 47 DD 77IAWdW.GYw
77CD1070 80 B4 CF 77 C0 45 DD 77 50 20 D0 77 50 6A D0 77IwAwP.DwPjDw
77CD1080 40 46 DD 77 00 47 DD 77 50 58 D0 77 80 47 DD 77@FYw.GYwXDW.GYw
77CD1090 50 26 D0 77 50 6A D0 77 80 46 DD 77 00 47 DD 77P&DwPjDw.FYw.GYw
77CD10A0 D0 D6 D3 77 80 47 DD 77 00 00 00 00 00 00 00 00D&Ww.GYw...
77CD10B0 00 00 00 00 57 14 01 E2 46 15 C5 43 A5 FE 00 80W...&F.AC+p...
77CD10C0 EE E3 D3 F0 06 00 00 00 9C 7C CD 77 01 00 00 80Iw...
77CD10D0 9A 8B 13 35 9E 5D BD 4F 8E 2D A2 44 02 25 F9 3AS.%&D.%u...
77CD10E0 06 00 01 00 80 7C CD 77 02 00 00 00 E3 28 2F 4AIw...&(/J
77CD10F0 B9 53 41 44 BA 9C D6 9D 4A 4A 6E 38 06 00 02 00SAD?.O.Jjns...
77CD1100 64 7C CD 77 03 00 00 00 76 6C 67 1F E1 80 39 42 d|Iw...vlg.&9B
77CD1110 95 BB 83 D0 F6 D0 DA 78 06 00 03 00 48 7C CD 77&D&Ux...&H|Iw

Command: [mic.exe: 00402053 -> 00402058 (0x00000006 bytes)]

Paused Time Wasted Debugging: 0:11:58:45

Inginerie Inversa X

Malware

OK

Inginerie Inversa X

Malware

OK

Inginerie Inversa X

Malware

OK

a) Execuție în cascadă prin ret 1 La adresa 00402024, codul injectat este apelat prima dată (prin call din Entry Point).Acesta afișează primul MessageBox. 2 Instrucțiunea ret duce execuția la 00402029, unde se află a doua copie a codului injectat.Se afișează al doilea MessageBox. 3 După al doilea ret, execuția ajunge la 00402039, care conține al treilea bloc identic.Se afișează și al treilea MessageBox. Codul este valid și se încheie controlat.

b) Prin alinierea directă a celor trei blocuri și utilizarea ret, execuția curge automat dintr-un bloc în altul, fără salturi explicite.Aceasta este o tehnică eficientă de execuție în lanț în contexte precum shellcode, injecții, sau patching.

mic.exe - PID: 16472 - Module: mic.exe - Thread: Main Thread 10072 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (T)

CPU Log Notes Breakpoints Memory Map Call

Rezumat pe scurt:
push 402024 + ret → sare la 402024
lanț de call-uri → ret → se întoarce la 402013
la 402013 este un alt ret, care revine unde era pe stivă adresa anterioară: 402014 la 402014 începe secvența call ExitProcess

1
2
3
4
5
6
7
8
9

mic.00402024
.text:0040200E mic.exe:\$200E #40E

Address Hex
00401000 31 20 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73 1 ginerie Invers
00401010 61 00 40 61 6C 77 61 72 65 00 00 00 00 00 00 00 a.Malware.....
00401020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401040 32 20 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73 2 ginerie Invers
00401050 61 00 40 61 6C 77 61 72 65 00 00 00 00 00 00 00 a.Malware.....
00401060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401080 33 20 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73 3 ginerie Invers
00401090 61 00 54 6F 61 74 65 20 63 65 6C 65 20 74 72 65 a.Toate cele tre
004010A0 69 20 60 73 67 62 6F 78 20 75 72 69 20 69 61 75 a.Toate cele tre
004010B0 20 61 63 65 73 74 20 74 65 78 74 20 64 65 a.Toate cele tre
004010C0 61 20 61 63 65 69 61 73 69 20 61 64 72 65 a.Toate cele tre
004010D0 21 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Command: [Paused] mic.exe: 00401101 -> 00401101 (0x00000001 bytes) Time Wasted Debugging: 0:12:17:05

1 ginerie Inversa

Toate cele trei msgbox-uri iau acest text de la aceiasi adresa!

OK

2 ginerie Inversa

Toate cele trei msgbox-uri iau acest text de la aceiasi adresa!

OK

3 ginerie Inversa

Toate cele trei msgbox-uri iau acest text de la aceiasi adresa!

OK

Pentru exercițiu, toate mesajele afișate în cele trei MessageBox-uri sunt luate de la aceeași adresă 4010A2 din segmentul .data. În schimb, titlurile acestor MessageBox-uri sunt luate de la adrese diferite, pentru a demonstra studenților că este posibil să reutilizăm același mesaj, dar să personalizăm titlul fiecărui dialog.

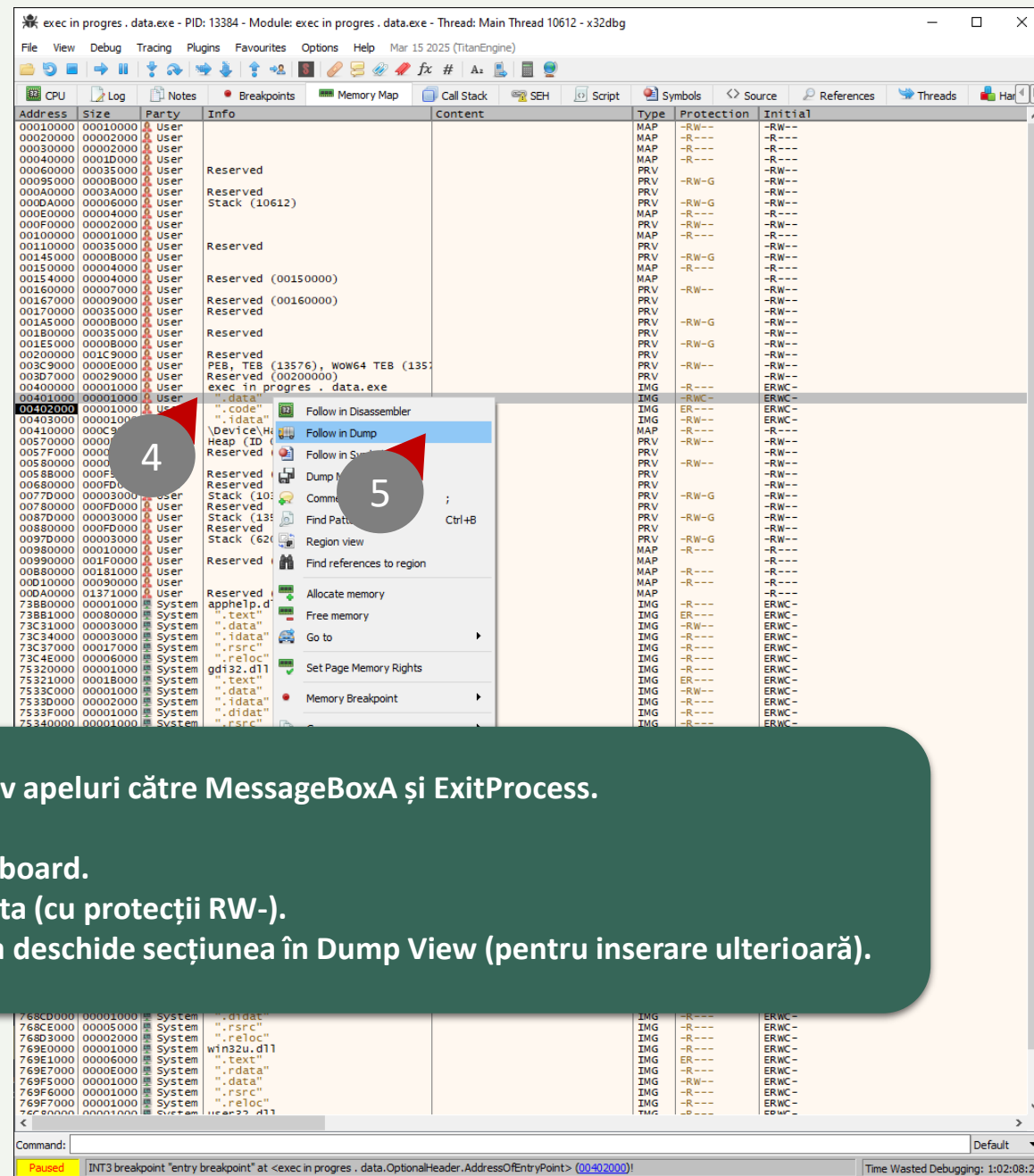
1 push 402024 + ret → simulează un call către adresa 402024 2 Se ajunge la 402024 unde începe lanțul de execuție 3 Se face call către blocul de la 203F 4 La 203F, se face un nou call către 2059 5 La 2059, se execută ultima secvență, care se încheie cu ret 6 ret de la 2058 revine la 203D (după call din 203F) 7 ret de la 203D revine la 2013 (după push-ret inițial) 8 La 2013 avem încă un ret, care duce execuția la 2014 9 Programul continuă la 2014 cu call ExitProcess – terminare normală

9.2 EXECUTIA DE COD ÎN SECTIUNEA .DATA



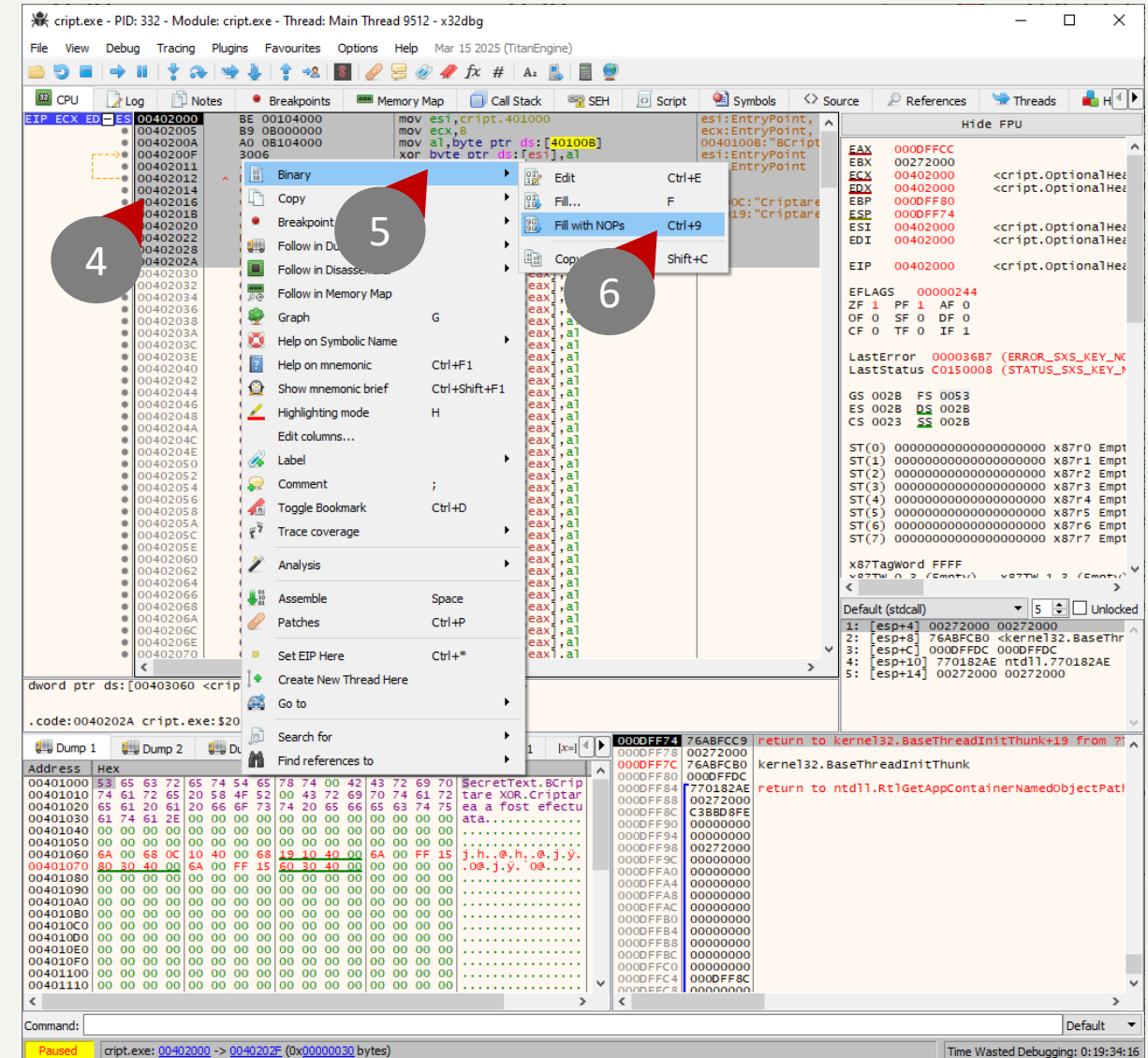
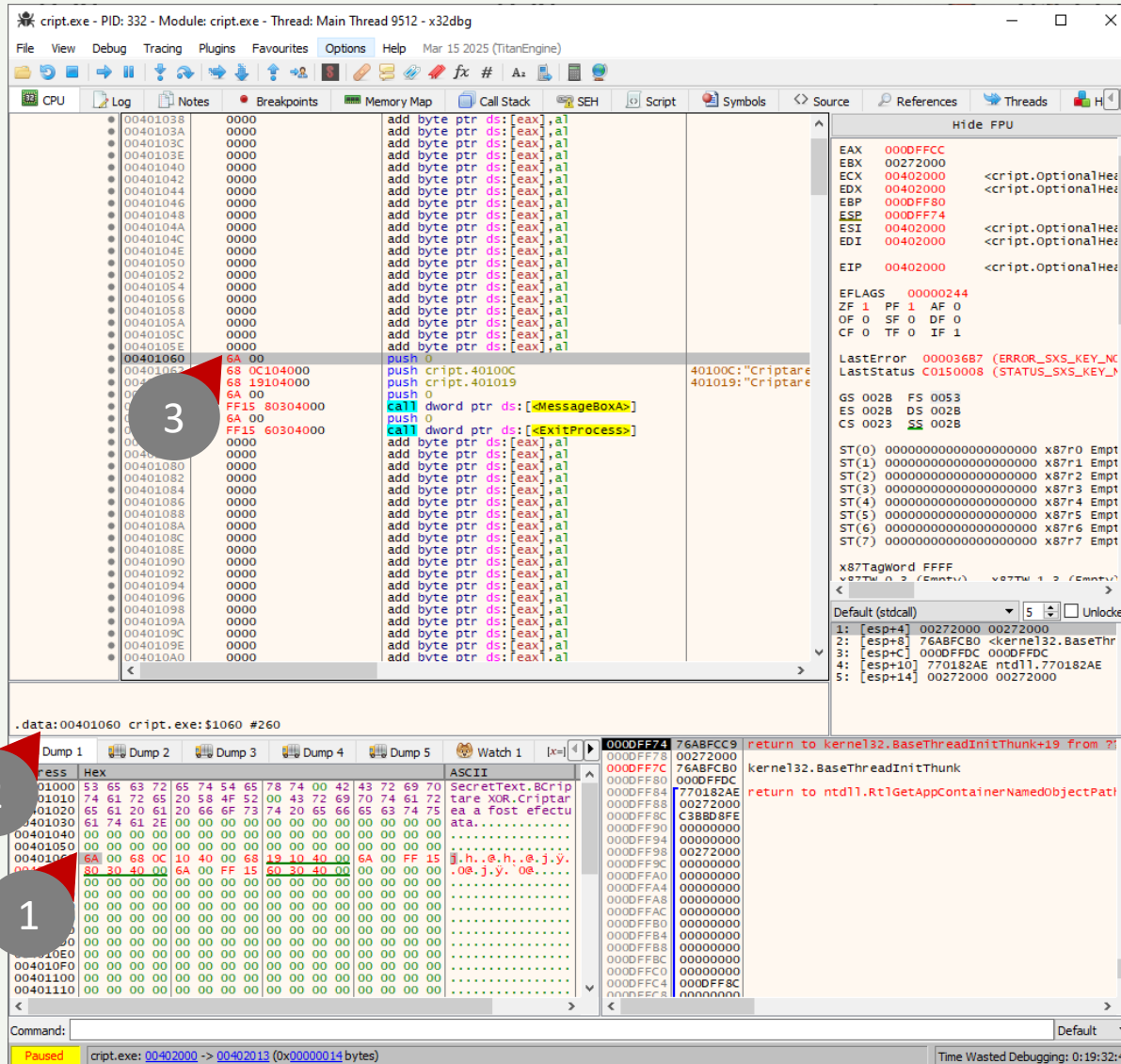
CONTEXT INTRODUCATIV

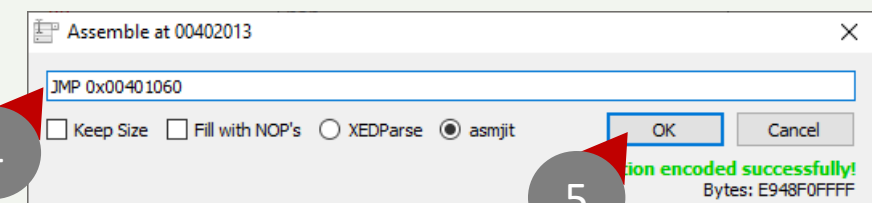
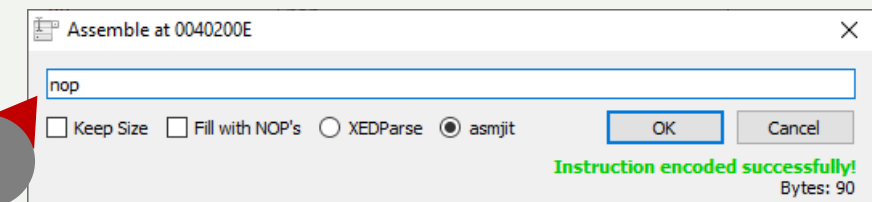
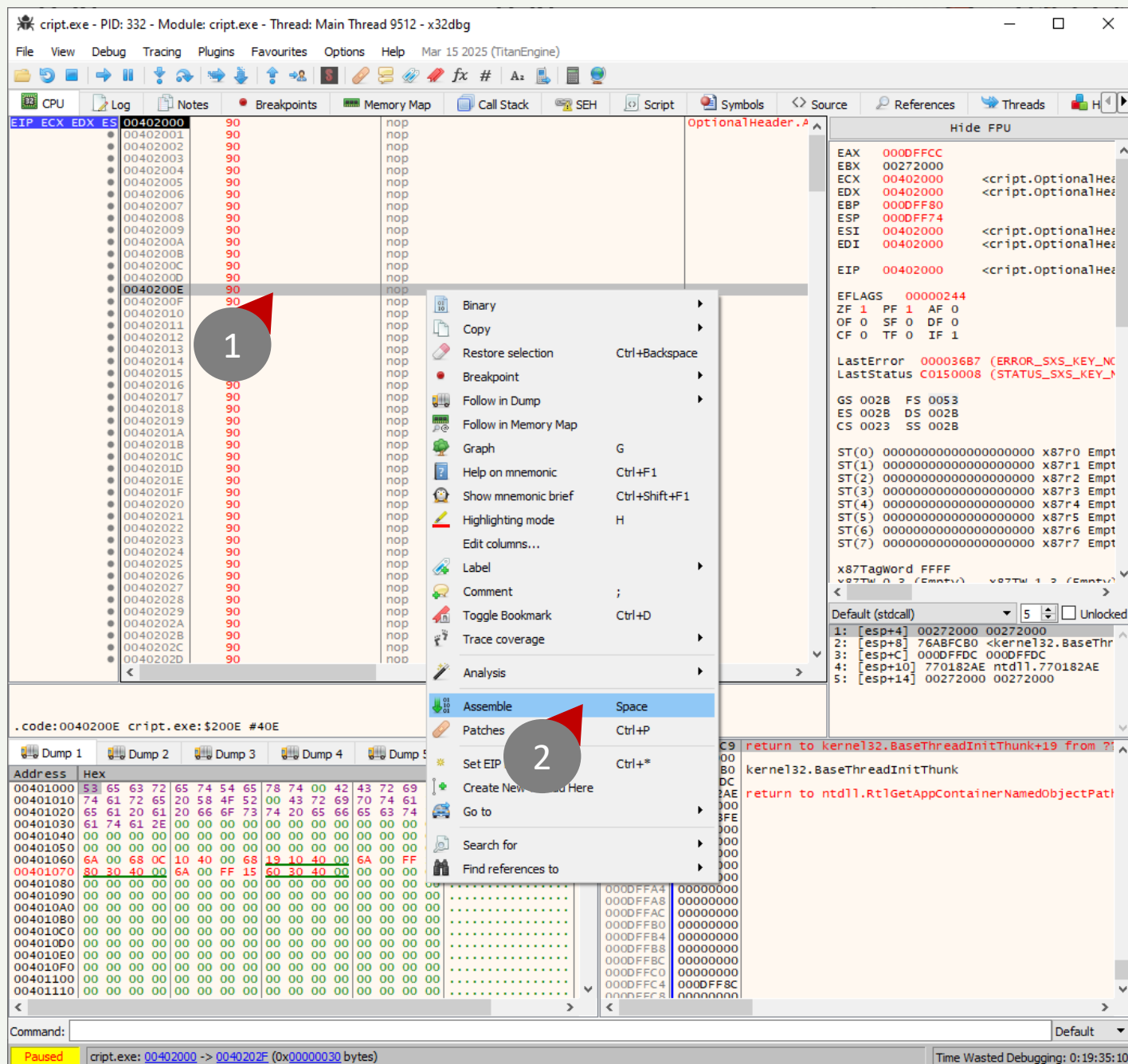
- În acest exemplu demonstrativ, relocăm o bucată de cod executabil din secțiunea .code (.text) într-o zonă liberă din secțiunea .data, care în mod normal nu este executabilă. Duplicăm codul, schimbăm permisiunile de memorie pentru a permite execuția și redirecționăm fluxul printr-un jmp. Această tehnică evidențiază posibilitatea rulării de cod din zone non-standard, un mecanism utilizat frecvent în malware sau tehnici avansate de ofuscare.



1. Observăm că în această zonă există deja cod — anterior injectat în secțiunea .data.
2. Confirmăm că ne aflăm în secțiunea .data — lucru vizibil în Dump (adresă + nume secțiune).
3. Comutăm în tab-ul CPU pentru a vizualiza codul din .data dezasamblat (disassembler view).
4. Selectăm blocul de cod din .code care trebuie eliminat.

5. Click dreapta → alegem Binary.
6. Selectăm Fill with NOPs pentru a neutraliza codul — aici vom introduce un jmp spre .data, deci acest cod nu mai este necesar.





- Pași pentru redirectionarea execuției (salt către .data):
1. Se selectează adresa din zona NOP (neutralizată anterior) unde vrem să inserăm saltul.
 2. Click dreapta → Assemble pentru a edita instrucțiunea.
 3. La prima adresă (0040200E), inserăm un nop — opțional, pentru aliniere.
 4. La adresa dorită (00402013), introducem comanda: jmp 0x00401060 (Aceasta este adresa codului injectat din .data, copiată anterior).
 5. Apăsăm OK → instrucțiunea jmp este codificată cu succes. Fluxul de execuție va fi deviat spre zona .data.

Execuția este acum deturnată către codul din .data. În locul vechiului cod, avem un jmp care sare direct în regiunea injectată. Aceasta finalizează tehnica de relocare + redirectionare.

crpt.exe - PID: 332 - Module: crpt.exe - Thread: Main Thread 9512 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

EIP ECX EDX ES 00402000 90 nop OptionalHeader.A

00402001 90 nop

00402002 90 nop

00402003 90 nop

00402004 90 nop

00402005 90 nop

00402006 90 nop

00402007 90 nop

00402008 90 nop

00402009 90 nop

0040200A 90 nop

0040200B 90 nop

0040200C 90 nop

0040200D 90 nop

0040200E - E9 4DF0FFFF jmp crpt.401060

00402013 90 nop

00402014 90 nop

00402015 90 nop

00402016 90 nop

00402017 90 nop

00402018 90 nop

00402019 90 nop

0040201A 90 nop

0040201B 90 nop

0040201C 90 nop

0040201D 90 nop

0040201E 90 nop

0040201F 90 nop

00402020 90 nop

00402021 90 nop

00402022 90 nop

00401010 90 nop

00401020 65 90 nop

00401030 61 90 nop

00401040 00 00 90 nop

00401050 00 00 90 nop

00401060 6A 00 90 nop

00401070 80 30 90 nop

00401080 00 00 90 nop

00401090 00 00 90 nop

004010A0 00 00 90 nop

004010B0 00 00 90 nop

004010C0 00 00 90 nop

004010D0 00 00 90 nop

0000 0000 add byte ptr ds:[eax],al

0000 0000 add byte ptr ds:[eax],al

Hide FPU

EAX 000FFCC

EBX 00272000

ECX 00402000 <crpt.OptionalHeader.A

EDX 00402000 <crpt.OptionalHeader.A

EBP 000FF80

ESP 000FF74

ESI 00402000 <crpt.OptionalHeader.A

EDI 00402000 <crpt.OptionalHeader.A

EIP 00402000 <crpt.OptionalHeader.A

EFLAGS 0000244

ZF 1 PF 1 AF 0

OF 0 SF 0 DF 0

CF 0 TF 0 IF 1

LastError 00003687 (ERROR_SXS_KEY_NO_VALUE)

LastStatus C0150008 (STATUS_SXS_KEY_NO_VALUE)

GS 002B FS 0053

ES 002B DS 002B

CS 0023 SS 002B

ST(0) 0000000000000000 x87r0 Empt

ST(1) 0000000000000000 x87r1 Empt

ST(2) 0000000000000000 x87r2 Empt

ST(3) 0000000000000000 x87r3 Empt

ST(4) 0000000000000000 x87r4 Empt

ST(5) 0000000000000000 x87r5 Empt

ST(6) 0000000000000000 x87r6 Empt

ST(7) 0000000000000000 x87r7 Empt

x87TagWord FFFF

x87TW 0 2 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 00272000 00272000

2: [esp+8] 76A8FC80 <kernel32.BaseThreadInitThunk

3: [esp+C] 000FFDC 000FFDC

4: [esp+10] 770182AE ntdll.770182AE

5: [esp+14] 00272000 00272000

.code:0040200E crpt.exe:\$200E #40E

Dump 1 Hex Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x=]

Address Hex ASCII

00401000 53 65 63 72 65 74 54 65 78 74 00 42 43 72 69 70 SecretText.BCrip

00401010 74 61 72 65 20 58 4F 52 00 43 72 69 70 74 61 72 tare XOR.Criptar

00401020 65 61 20 61 20 66 6F 73 74 20 65 66 65 63 74 75 ea a fost efectu

00401030 61 74 61 2E 00 00 00 00 00 00 00 00 00 00 00 00 ata.....

00401040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00401050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00401060 6A 00 68 0C 10 40 00 68 19 10 40 00 6A 00 FF 15 j.h..@.h..@.j.y.

00401070 80 30 40 00 6A 00 FF 15 60 30 40 00 00 00 00 00 .@.j.y.'0@.....

00401080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00401090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

004010F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00401100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

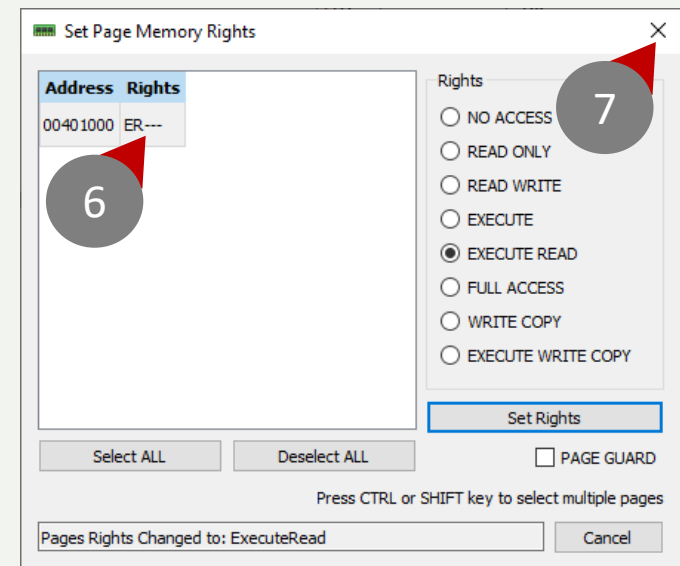
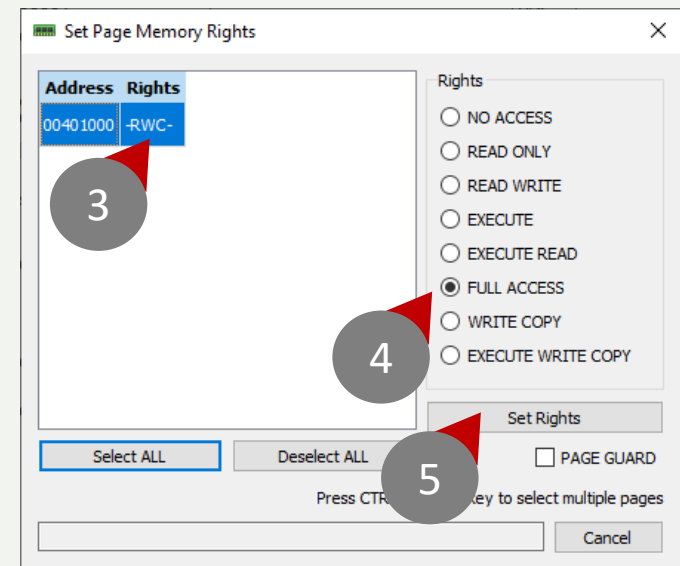
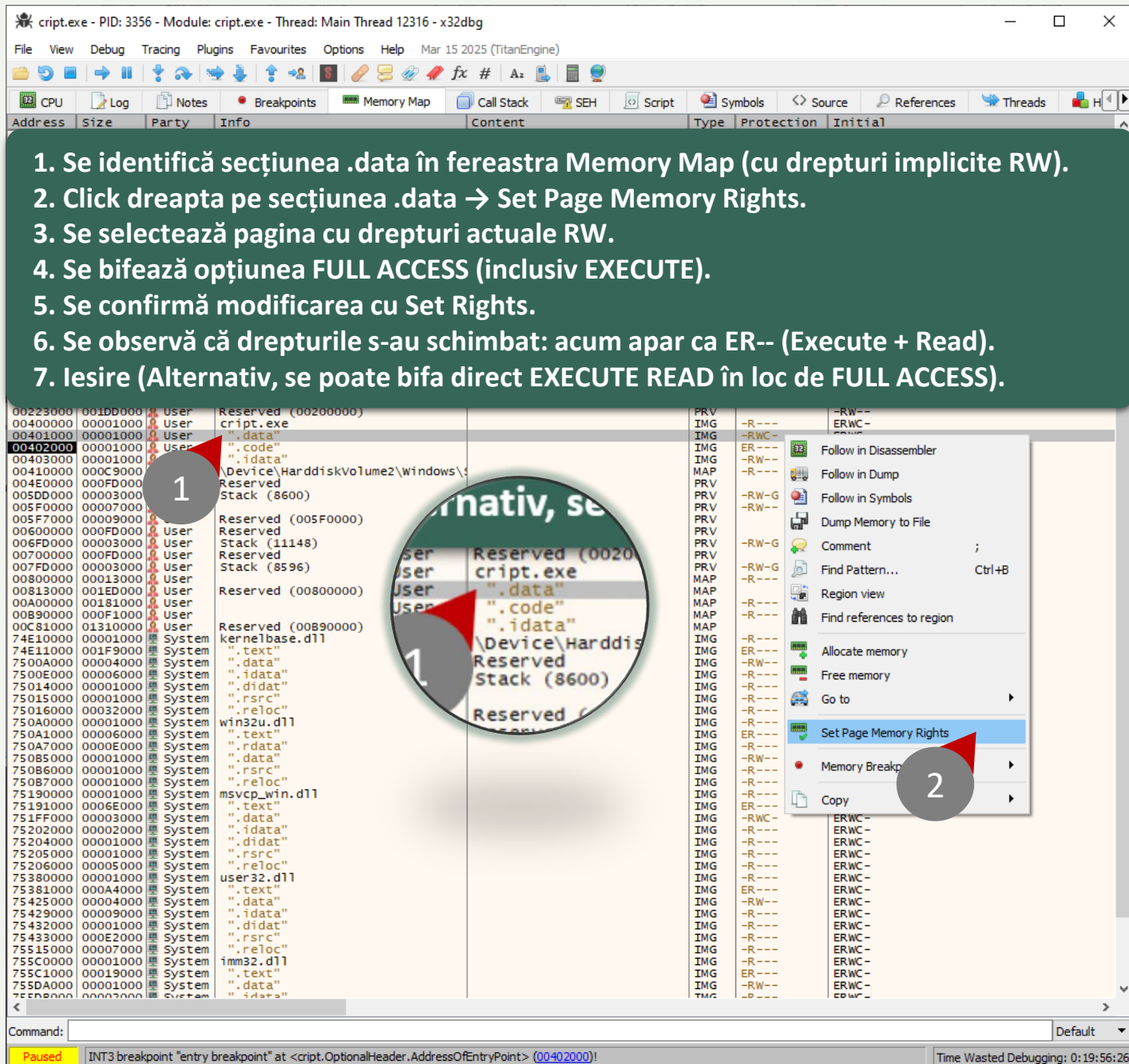
00401110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Command: crpt.exe: 0040200E -> 00402012 (0x00000005 bytes)

Paused Time Wasted Debugging: 0:19:38:49

1. Instrucțiunea jmp 0x00401060 a fost inserată corect în secțiunea .code. Aceasta redirecționează execuția către codul din .data. 2. În Dump, vedem clar că adresa 00401060 conține cod real, funcțional, nu doar date statice. Aceasta confirmă că saltul va ajunge într-o zonă validă de execuție.
2. Execuția este pregătită pentru a porni din zona .data. Instrucțiunea jmp a fost inserată cu succes, iar codul injectat în .data va fi executat ca parte a fluxului programului.

Execuția este pregătită pentru a porni din zona .data. Instrucțiunea jmp a fost inserată cu succes, iar codul injectat în .data va fi executat ca parte a fluxului programului.



Permisiunile secțiunii .data au fost extinse. Activarea EXECUTE permite rularea codului injectat, completând cu succes procesul de relocare și execuție.

1. Confirmăm că permisiunile din Memory Map pentru secțiunea .data sunt acum ER-- — permite EXECUTE, deci codul poate rula.
2. Execuția ajunge la instrucțiunea jmp către adresa injectată (codul relocat în .data).
3. MessageBox-ul apare cu textul „Criptarea a fost efectuată”, ceea ce înseamnă că fluxul de execuție a fost redirectionat și codul relocat rulează fără erori. Execuție confirmată în secțiunea .data. Datorită permisiunilor modificate și a saltului corect configurat, codul injectat se execută și generează mesajul final din aplicație.

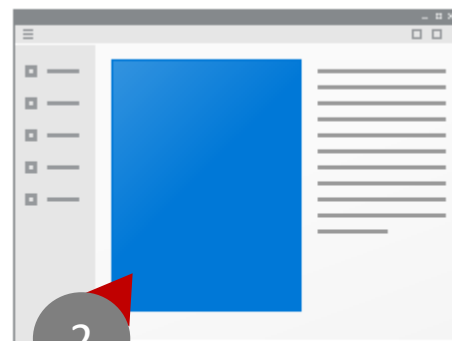
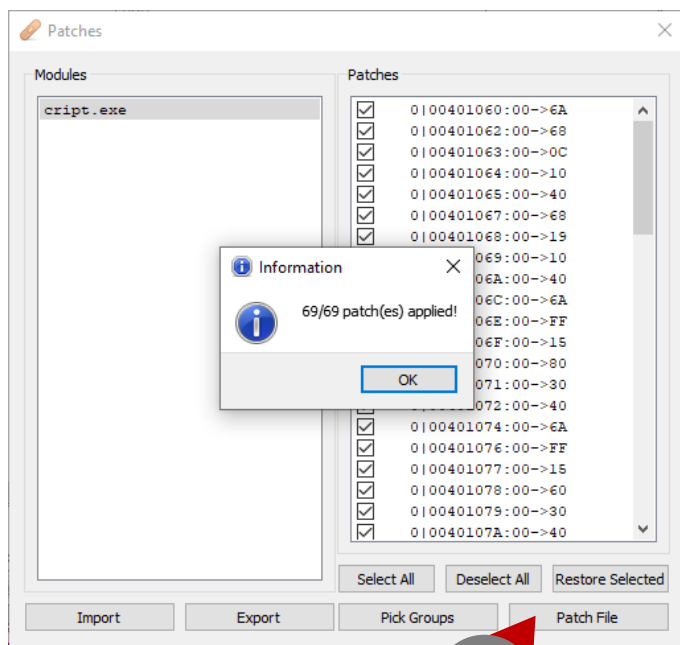
The screenshot shows the Memory Map window in x32dbg. The table lists memory sections with their addresses, sizes, parties, and permissions. The .data section is highlighted with a red circle and the number 1, showing permissions of ER-- (Executable and Readable).

Address	Size	Party	Info	Content	Type	Protection	Initial
00010000	00002000	User	Reserved		MAP	-RW--	-RW--
00020000	00002000	User	Reserved		MAP	-RW--	-RW--
00030000	00002000	User	Reserved		MAP	-RW--	-RW--
00040000	00001000	User	Reserved		MAP	-RW--	-RW--
00060000	00003500	User	Reserved		PRV	-RW--	-RW--
00095000	00008000	User	Reserved		PRV	-RW--	-RW--
000A0000	00003A00	User	Reserved		PRV	-RW--	-RW--
000DA000	00006000	User	Reserved		PRV	-RW--	-RW--
000E0000	00004000	User	Reserved		MAP	-RW--	-RW--
000F0000	00002000	User	Reserved		PRV	-RW--	-RW--
00100000	00001000	User	Reserved		MAP	-RW--	-RW--
00110000	00003500	User	Reserved		PRV	-RW--	-RW--
00145000	00008000	User	Reserved		PRV	-RW--	-RW--
00150000	00003500	User	Reserved		PRV	-RW--	-RW--
00185000	00008000	User	Reserved		PRV	-RW--	-RW--
00190000	00004000	User	Reserved		MAP	-RW--	-RW--
00194000	00004000	User	Reserved (00190000)		MAP	-RW--	-RW--
001C0000	00000000	User	Heap (ID 0)		PRV	-RW--	-RW--
001C0000	00003000	User	Reserved (001C0000)		PRV	-RW--	-RW--
001D0000	00007000	User	Reserved		PRV	-RW--	-RW--
001D7000	00009000	User	Reserved (001D0000)		PRV	-RW--	-RW--
00200000	00007100	User	Reserved		PRV	-RW--	-RW--
00271000	00011000	User	PEB, TEB (9512), Wow64 TEB (9512)		PRV	-RW--	-RW--
00282000	0017E000	User	Reserved (00200000)		PRV	-RW--	-RW--
00400000	00001000	User	crpt.exe		IMG	-RW--	-RW--
00401000	00001000	User	code		IMG	-RW--	-RW--
00403000	00001000	User	.idata		IMG	-RW--	-RW--
00410000	000C9000	User	\Device\HarddiskVolume2\windows\...		IMG	-RW--	-RW--
004E0000	000F0000	User	Reserved		IMG	-RW--	-RW--
005D0000	00003000	User	Stack (7240)		IMG	-RW--	-RW--
005E0000	000F0000	User	Reserved		IMG	-RW--	-RW--
006D0000	00003000	User	Stack (2944)		IMG	-RW--	-RW--
006E0000	00003500	User	Reserved		IMG	-RW--	-RW--
00715000	00008000	User	Reserved		IMG	-RW--	-RW--
00720000	000F0000	User	Reserved		IMG	-RW--	-RW--
0081D000	00003000	User	Stack (1576)		IMG	-RW--	-RW--
00820000	00013000	User	Reserved		IMG	-RW--	-RW--
00833000	001ED000	User	Reserved (00820000)		IMG	-RW--	-RW--
00A20000	00181000	User	Reserved		IMG	-RW--	-RW--
008B0000	000F1000	User	Reserved		IMG	-RW--	-RW--
00CA1000	01310000	User	Reserved (008B0000)		IMG	-RW--	-RW--
01FC0000	00035000	User	Reserved		IMG	-RW--	-RW--
01FF5000	00008000	User	Reserved		IMG	-RW--	-RW--
02000000	000F0000	User	Reserved		IMG	-RW--	-RW--
020FD000	00003000	User	Reserved		IMG	-RW--	-RW--
74E10000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
74E11000	001F9000	System	kernelbase.dll		IMG	-RW--	-RW--
750A0000	00004000	System	kernelbase.dll		IMG	-RW--	-RW--
750E0000	00006000	System	kernelbase.dll		IMG	-RW--	-RW--
75014000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75015000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75016000	00003200	System	kernelbase.dll		IMG	-RW--	-RW--
750A0000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
750A0000	00006000	System	kernelbase.dll		IMG	-RW--	-RW--
750A7000	0000E000	System	kernelbase.dll		IMG	-RW--	-RW--
75085000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75086000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75087000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75190000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75191000	00006000	System	kernelbase.dll		IMG	-RW--	-RW--
751FF000	00003000	System	kernelbase.dll		IMG	-RW--	-RW--
75202000	00002000	System	kernelbase.dll		IMG	-RW--	-RW--
75204000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75205000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75206000	00005000	System	kernelbase.dll		IMG	-RW--	-RW--
75380000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75381000	00004000	System	kernelbase.dll		IMG	-RW--	-RW--
75425000	00004000	System	kernelbase.dll		IMG	-RW--	-RW--
75429000	00009000	System	kernelbase.dll		IMG	-RW--	-RW--
75432000	00001000	System	kernelbase.dll		IMG	-RW--	-RW--
75433000	00002000	System	kernelbase.dll		IMG	-RW--	-RW--
75434000	00007000	System	kernelbase.dll		IMG	-RW--	-RW--

The screenshot shows the Disassembly window in x32dbg. The instruction at address 00402000 is a jmp instruction jumping to 40F0FFFF. A red circle with the number 2 points to this instruction. Below the instruction, a MessageBox dialog titled "Criptare XOR" is shown with the text "Criptarea a fost efectuată." and an OK button. A red circle with the number 3 points to the OK button.

Disassembly window showing the instruction at address 00402000: jmp 40F0FFFF. The instruction is highlighted with a red circle and the number 2.

MessageBox dialog titled "Criptare XOR" with the text "Criptarea a fost efectuată." and an OK button. The dialog is highlighted with a red circle and the number 3.



1. Fereastra Patches confirmă aplicarea cu succes a tuturor modificărilor (69/69 patch-uri aplicate). Aici am:injectat codul în .data,modificat permisiunile,redirecționat execuția cu jmp,și am validat funcționarea prin MessageBox.2. După apăsarea Patch file, se generează un executabil nou (indata.exe) care conține deja toate patch-urile. Acesta poate fi rulat direct din Windows, fără debugger, iar comportamentul va fi identic: cod relocat și executat din .data.
2. Modificările au fost salvate.Executabilul final (indata.exe) conține toate patch-urile aplicate în sesiunea de analiză și poate fi rulat independent.

9.3

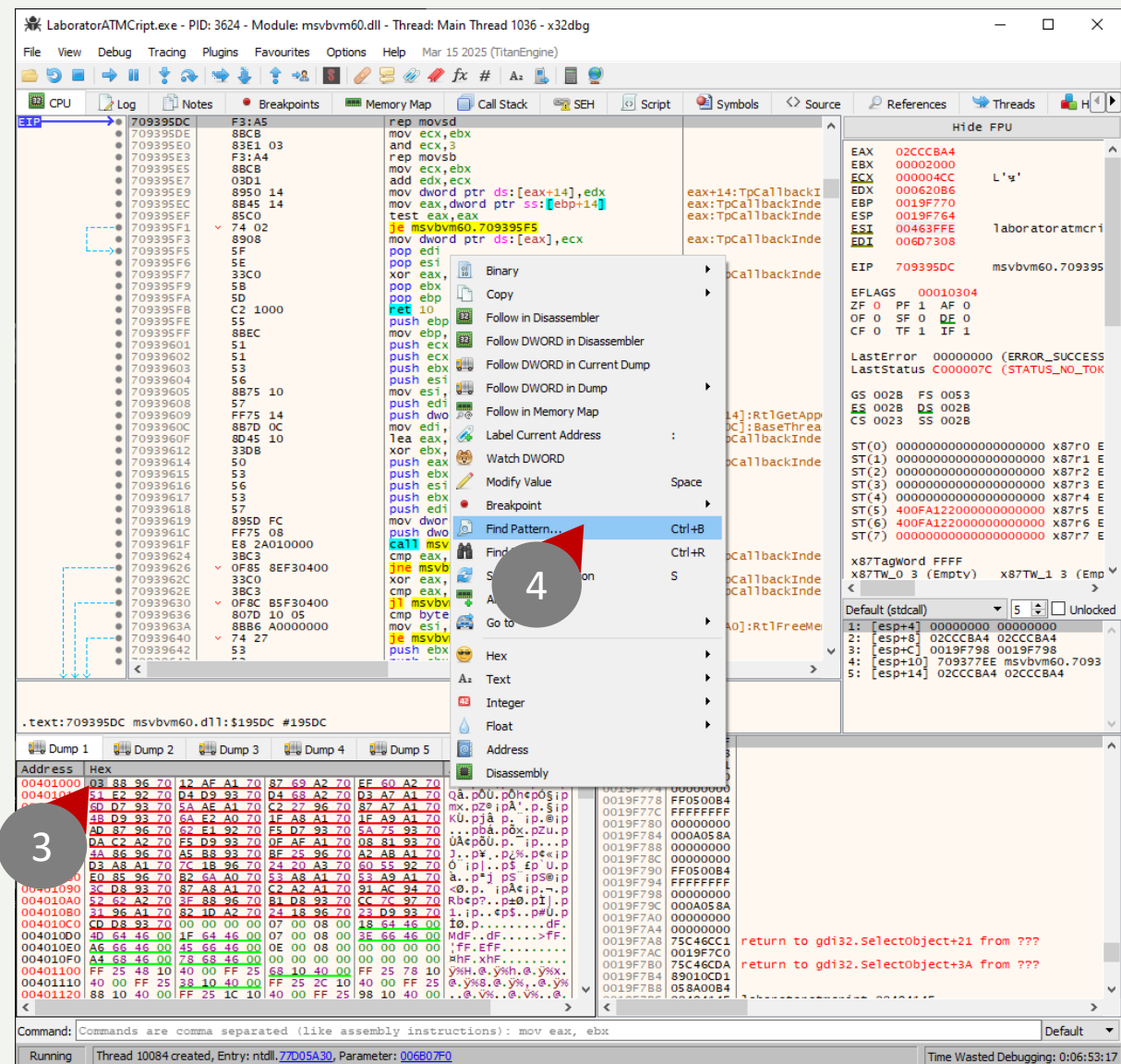
MODIFICAREA STRINGURILOR IN OBIECTE



CONTEXT INTRODUCTIV

- În acest experiment demonstrativ, explorăm cum pot fi localizate și modificate manual stringurile dintr-un executabil. Deși, în mod obișnuit, datele statice precum textele interfeței grafice sunt stocate în secțiunile `.data` sau `.rdata`, în unele cazuri acestea pot fi ascunse sau dispersate în alte regiuni ale memoriei.
- Scopul este să identificăm stringul unui buton din interfață (ex: „Verifică”), să-l localizăm în memorie, să-l modificăm (ex: în „Academia”), iar apoi să aplicăm patch-ul permanent în fișierul executabil.

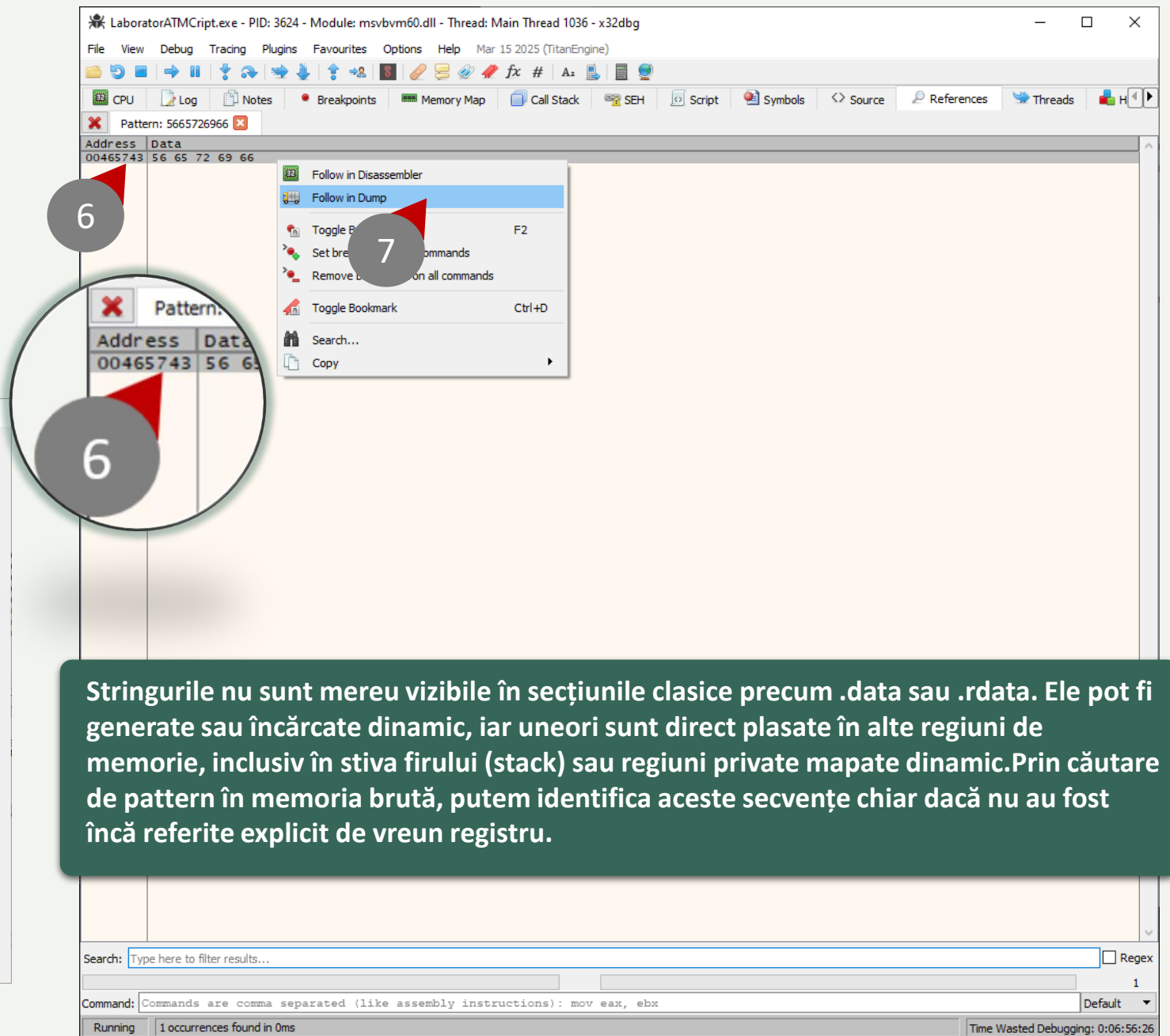
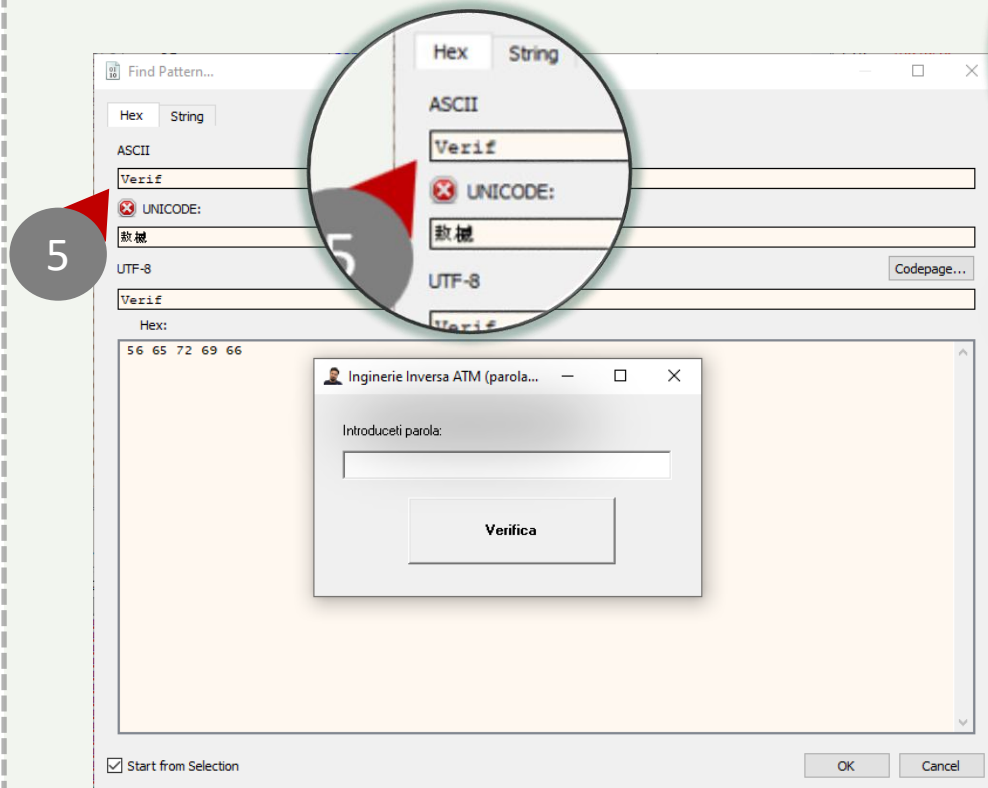
-



5. Se introduce textul „Verifică” în căsuța de căutare, în format UNICODE — căutăm stringul butonului.

6. Rezultatul căutării arată adresa unde este localizat stringul „Verifică” în memorie.

7. Click dreapta pe adresă → „Follow in Dump” pentru a vedea zona completă din memorie unde e stocat textul.



Stringurile nu sunt mereu vizibile în secțiunile clasice precum .data sau .rdata. Ele pot fi generate sau încărcate dinamic, iar uneori sunt direct plasate în alte regiuni de memorie, inclusiv în stiva firului (stack) sau regiuni private mapate dinamic. Prin căutare de pattern în memoria brută, putem identifica aceste secvențe chiar dacă nu au fost încă referite explicit de vreun registru.

1. Am găsit stringul „Verifică” și îl vedem în zona Dump, sub formă de UNICODE.
2. Confirmăm că stringul este mapat într-o zonă scrisă (de ex. RW-), deci poate fi modificat.
3. Click dreapta pe instrucțiunea care referă acest string → Binary.
4. Apoi alegem „Edit” pentru a modifica stringul (conținutul lui).

LaboratorATMCript.exe - PID: 3624 - Module: msbvm60.dll - Thread: Main Thread 1036 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

709395DC F3:A5 rep movsd
709395DE 8BC8 mov ecx,ebx
709395E0 83E1 03 and ecx,3
709395E3 F3:A4 rep movsb
709395E5 8BC8 mov ecx,ebx
709395E7 03D1 add edx,ecx
709395E9 8950 14 mov dword ptr ds:[eax+14],edx
709395EC 8845 14 mov eax,dword ptr ss:[ebp+14]
709395EF 85C0 test eax,eax
709395F1 74 02 je msbvm60.709395F5
709395F3 8908 mov dword ptr ds:[eax],ecx
709395F5 5F pop edi
709395F6 5E pop esi
709395F7 33C0 xor eax,eax
709395F9 5B pop ebx
709395FA 5D pop ebp
709395FB C2 1000 ret 10
709395FE 55 push ebp
709395FF 8BEC mov ebp,esp
70939601 51 push ecx
70939602 51 push ecx
70939603 53 push ebx
70939604 56 push esi
70939605 mov esi,dword ptr ss:[ebp+10]
70939608 push edi
70939609 push dword ptr ss:[ebp+14]
7093960B FF75 14 push dword ptr ss:[ebp+14]
7093960C 8B7D 0C mov edi,dword ptr ss:[ebp+14]
7093960F 8D45 10 lea eax,dword ptr ss:[ebp+10]
70939612 330B xor ebx,ebx
70939614 50 push eax
70939615 53 push ebx
70939616 56 push esi
70939617 53 push ebx
70939618 push edi
70939619 mov dword ptr ss:[ebp+14],edx
7093961C FF75 08 push dword ptr ss:[ebp+14]
7093961F 8B7D 0C mov edi,dword ptr ss:[ebp+14]
70939622 3BC3 cmp eax,ebx
70939626 33C0 xor eax,eax
7093962C 3BC3 cmp eax,ebx
70939630 50 push eax
70939636 8B7D 10 mov edi,dword ptr ss:[ebp+10]
7093963A 8B86 A0000000 mov esi,dword ptr ds:[eax+0]
70939640 74 27 je msbvm60.70939649
70939642 53 push ebx
70939643 5B pop ebx

Hide FPU

EAX 02CCCB44
EBX 00002000
ECX 000004CC L'y'
EDX 000620B6
EBP 0019F770
ESP 0019F764
ESI 00463FFE laboratoratmcri
EDI 006D7308

EIP 709395DC msbvm60.709395

EFLAGS 00010304
ZF 0 PF 1 AF 0
OF 0 SF 0 DE 0
CF 0 TF 1 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C000007C (STATUS_NO_TOK)

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 0000000000000000 x87r0 E
ST(1) 0000000000000000 x87r1 E
ST(2) 0000000000000000 x87r2 E
ST(3) 0000000000000000 x87r3 E
ST(4) 0000000000000000 x87r4 E
ST(5) 400F122000000000 x87r5 E
ST(6) 400F122000000000 x87r6 E
ST(7) 0000000000000000 x87r7 E

70939649 77TagWord FFFF x87TW_1_3 (Emp)
x87TW_0_3 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 00000000 00000000
2: [esp+8] 02CCCB44 02CCCB44
3: [esp+C] 0019F798 0019F798
4: [esp+10] 709377EE msbvm60.7093
5: [esp+14] 02CCCB44 02CCCB44

.text:709395DC msbvm60.dll:195DC #195DC

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x] [f]

0019F764 708E3200 coreessaging.DllCanUnloadNow+22CA0
0019F76C 00000034
0019F76E 00000000

Address Hex ASCII

00465743 56 65 72 69 66 69 63 61 00 04 B0 04 28 05 5F 0A Verifică...
00465747 57 03 11 02 00 1D 01 00 00 00 BC 02 44 42 01 00 W...
00465753 00 4D 53 20 53 61 6E 73 20 53 65 72 69 66 FE 03 .MS Sans Serif.
21 00 00 02 08 09 49 6E 70 75 74 50 61 72 6F InputParo
6C 61 00 02 04 68 01 00 02 77 10 77 01 12 09 00 la...h.D.W.w.
FF 03 33 00 00 03 06 00 4C 61 62 65 6C 31 00 y.3...Label1
01 01 13 00 49 6E 74 72 6F 64 75 63 65 74 69 20 Introducere
70 61 72 6F 6C 61 3A 00 05 68 01 68 01 37 05 77 parola:h.h.7.w
01 12 00 00 FE 02 04 00 50 00 00 7C 52 68 .R.
72 22 CF D9 4A 8C F7 99 8F C1 7A D8 BF 00 00 00 IUJ...AzO
004657E3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004657F3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465803 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465813 00 78 12 40 00 00 00 00 00 00 00 00 00 00
00465823 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465833 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465843 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465853 00 FF FF FF 08 00 00 00 01 00 00 00 00 00
00465863 00 E9 00 00 CC 57 46 00 CC 57 46 00 34 12 40 e...INF.4.0

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Running laboratoratmcri.exe: 00465743 -> 00465743 (0x00000001 bytes)

Time Wasted Debugging: 0:06:57:34

LaboratorATMCript.exe - PID: 3624 - Module: msbvm60.dll - Thread: Main Thread 1036 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

709395DC F3:A5 rep movsd
709395DE 8BC8 mov ecx,ebx
709395E0 83E1 03 and ecx,3
709395E3 F3:A4 rep movsb
709395E5 8BC8 mov ecx,ebx
709395E7 03D1 add edx,ecx
709395E9 8950 14 mov dword ptr ds:[eax+14],edx
709395EC 8845 14 mov eax,dword ptr ss:[ebp+14]
709395EF 85C0 test eax,eax
709395F1 74 02 je msbvm60.709395F5
709395F3 8908 mov dword ptr ds:[eax],ecx
709395F5 5F pop edi
709395F6 5E pop esi
709395F7 33C0 xor eax,eax
709395F9 5B pop ebx
709395FA 5D pop ebp
709395FB C2 1000 ret 10
709395FE 55 push ebp
709395FF 8BEC mov ebp,esp
70939601 51 push ecx
70939602 51 push ecx
70939603 53 push ebx
70939604 56 push esi
70939605 mov esi,dword ptr ss:[ebp+10]
70939608 push edi
70939609 push dword ptr ss:[ebp+14]
7093960B FF75 14 push dword ptr ss:[ebp+14]
7093960C 8B7D 0C mov edi,dword ptr ss:[ebp+14]
7093960F 8D45 10 lea eax,dword ptr ss:[ebp+10]
70939612 330B xor ebx,ebx
70939614 50 push eax
70939615 53 push ebx
70939616 56 push esi
70939617 53 push ebx
70939618 push edi
70939619 mov dword ptr ss:[ebp+14],edx
7093961C FF75 08 push dword ptr ss:[ebp+14]
7093961F 8B7D 0C mov edi,dword ptr ss:[ebp+14]
70939622 3BC3 cmp eax,ebx
70939626 33C0 xor eax,eax
7093962C 3BC3 cmp eax,ebx
70939630 50 push eax
70939636 8B7D 10 mov edi,dword ptr ss:[ebp+10]
7093963A 8B86 A0000000 mov esi,dword ptr ds:[eax+0]
70939640 74 27 je msbvm60.70939649
70939642 53 push ebx
70939643 5B pop ebx

Hide FPU

EAX 02CCCB44
EBX 00002000
ECX 000004CC L'y'
EDX 000620B6
EBP 0019F770
ESP 0019F764
ESI 00463FFE laboratoratmcri
EDI 006D7308

EIP 709395DC msbvm60.709395

EFLAGS 00010304
ZF 0 PF 1 AF 0
OF 0 SF 0 DE 0
CF 0 TF 1 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C000007C (STATUS_NO_TOK)

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 0000000000000000 x87r0 E
ST(1) 0000000000000000 x87r1 E
ST(2) 0000000000000000 x87r2 E
ST(3) 0000000000000000 x87r3 E
ST(4) 0000000000000000 x87r4 E
ST(5) 400F122000000000 x87r5 E
ST(6) 400F122000000000 x87r6 E
ST(7) 0000000000000000 x87r7 E

70939649 77TagWord FFFF x87TW_1_3 (Emp)
x87TW_0_3 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 00000000 00000000
2: [esp+8] 02CCCB44 02CCCB44
3: [esp+C] 0019F798 0019F798
4: [esp+10] 709377EE msbvm60.7093
5: [esp+14] 02CCCB44 02CCCB44

.text:709395DC msbvm60.dll:195DC #195DC

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x] [f]

0019F764 708E3200 coreessaging.DllCanUnloadNow+22CA0
0019F76C 00000034
0019F76E 00000000

Address Hex ASCII

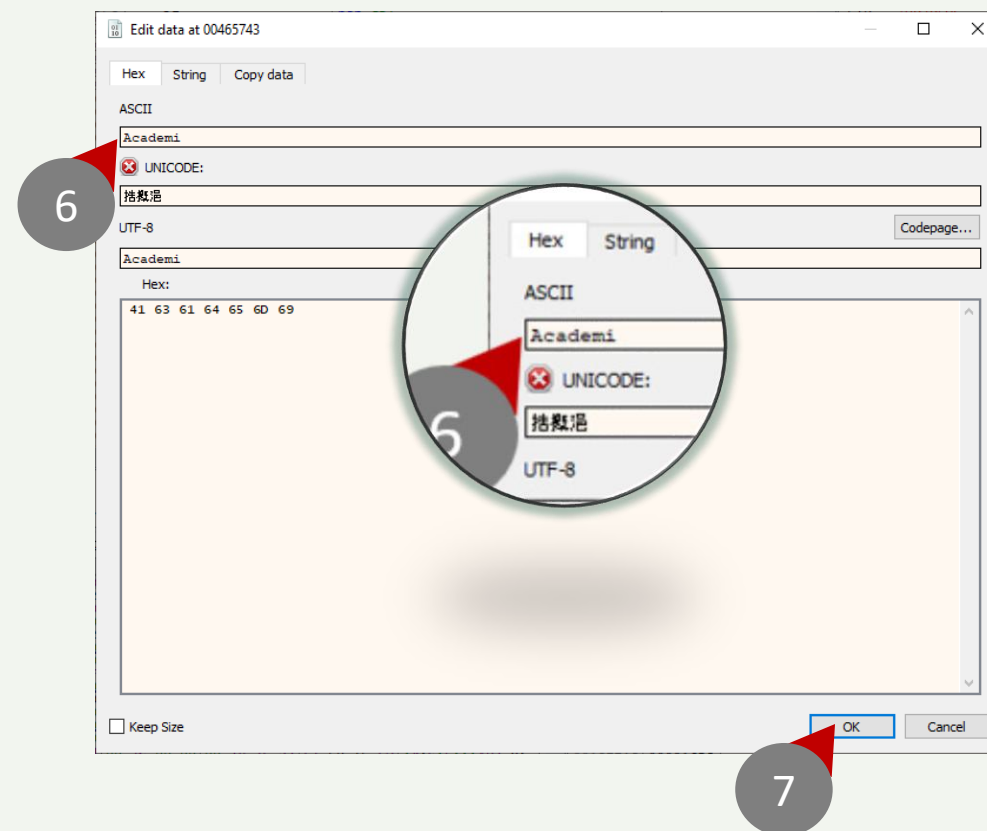
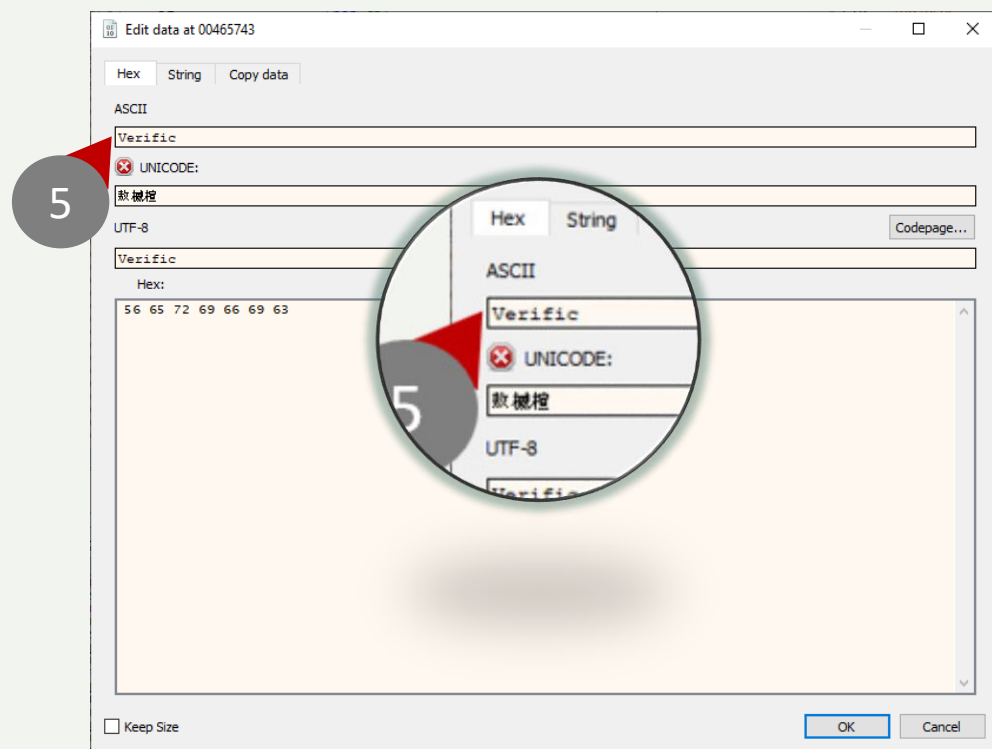
00465743 56 65 72 69 66 69 63 61 00 04 B0 04 28 05 5F 0A Verifică...
00465747 57 03 11 02 00 1D 01 00 00 00 BC 02 44 42 01 00 W...
00465753 00 4D 53 20 53 61 6E 73 20 53 65 72 69 66 FE 03 .MS Sans Serif.
21 00 00 02 08 09 49 6E 70 75 74 50 61 72 6F InputParo
6C 61 00 02 04 68 01 00 02 77 10 77 01 12 09 00 la...h.D.W.w.
FF 03 33 00 00 03 06 00 4C 61 62 65 6C 31 00 y.3...Label1
01 01 13 00 49 6E 74 72 6F 64 75 63 65 74 69 20 Introducere
70 61 72 6F 6C 61 3A 00 05 68 01 68 01 37 05 77 parola:h.h.7.w
01 12 00 00 FE 02 04 00 50 00 00 7C 52 68 .R.
72 22 CF D9 4A 8C F7 99 8F C1 7A D8 BF 00 00 00 IUJ...AzO
004657E3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004657F3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465803 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465813 00 78 12 40 00 00 00 00 00 00 00 00 00 00
00465823 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465833 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465843 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465853 00 FF FF FF 08 00 00 00 01 00 00 00 00 00
00465863 00 E9 00 00 CC 57 46 00 CC 57 46 00 34 12 40 e...INF.4.0

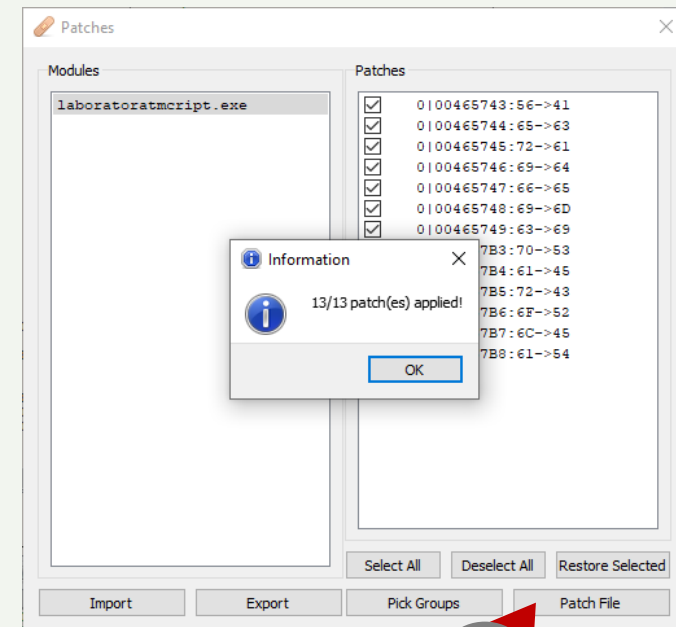
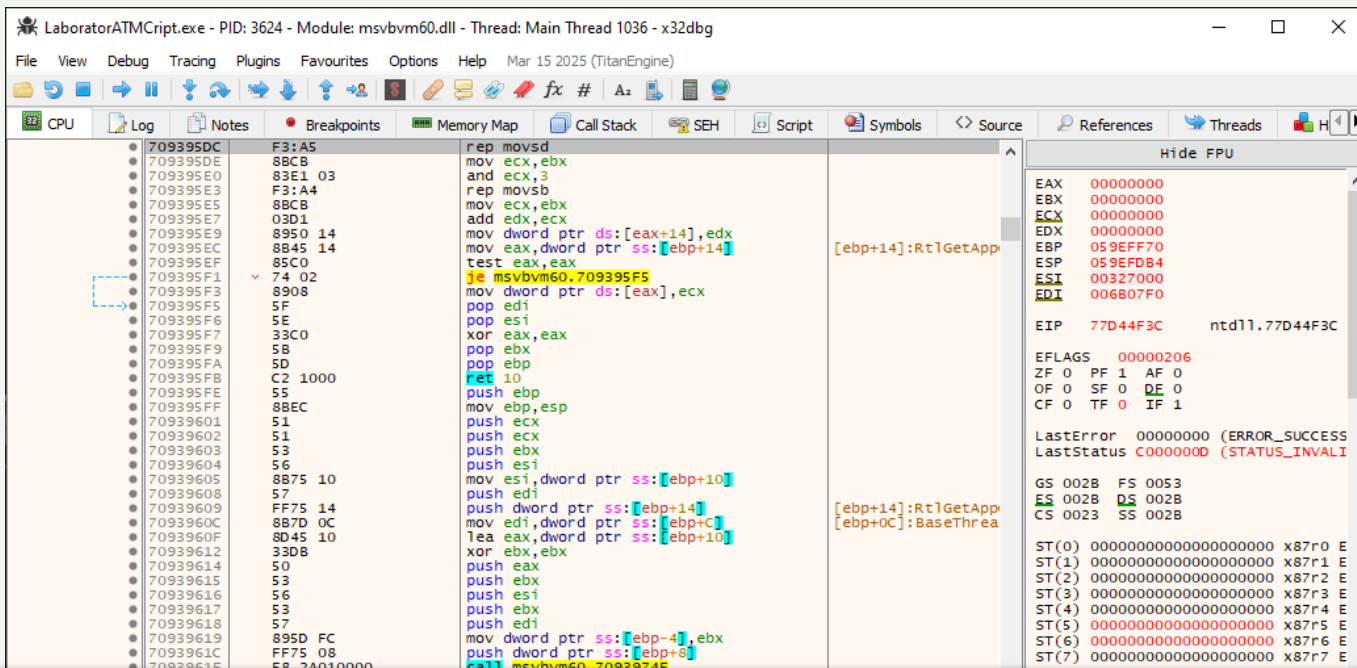
Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Running laboratoratmcri.exe: 00465743 -> 00465743 (0x00000001 bytes)

Time Wasted Debugging: 0:06:59:11

1. Selectăm stringul original „Verific” (format UNICODE), pe care vrem să-l înlocuim.
2. Introducem noul text în câmpul ASCII — aici „Academi”.
4. E important să nu depășim spațiul alocat în memorie. Apăsăm OK pentru a salva modificarea și a actualiza memoria procesului.

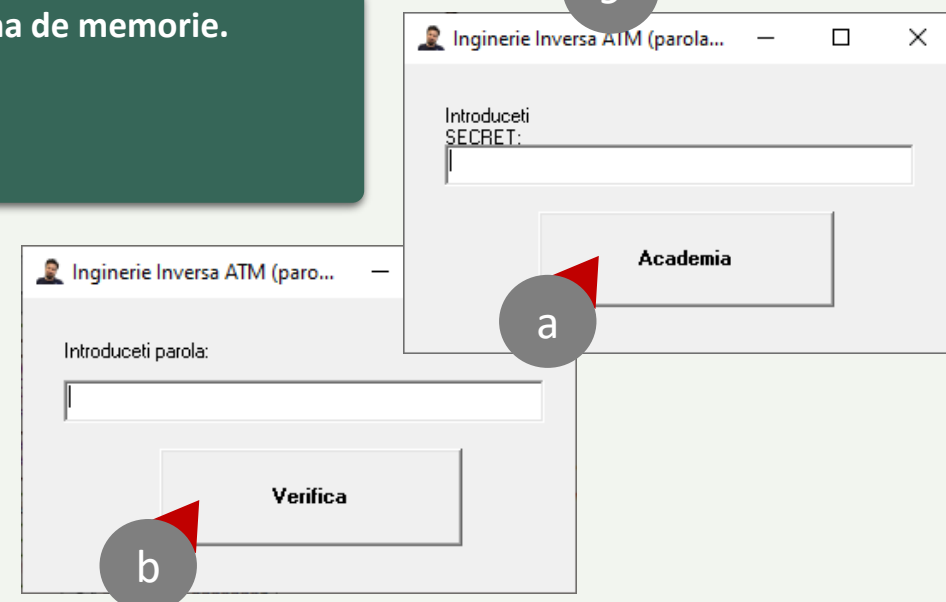
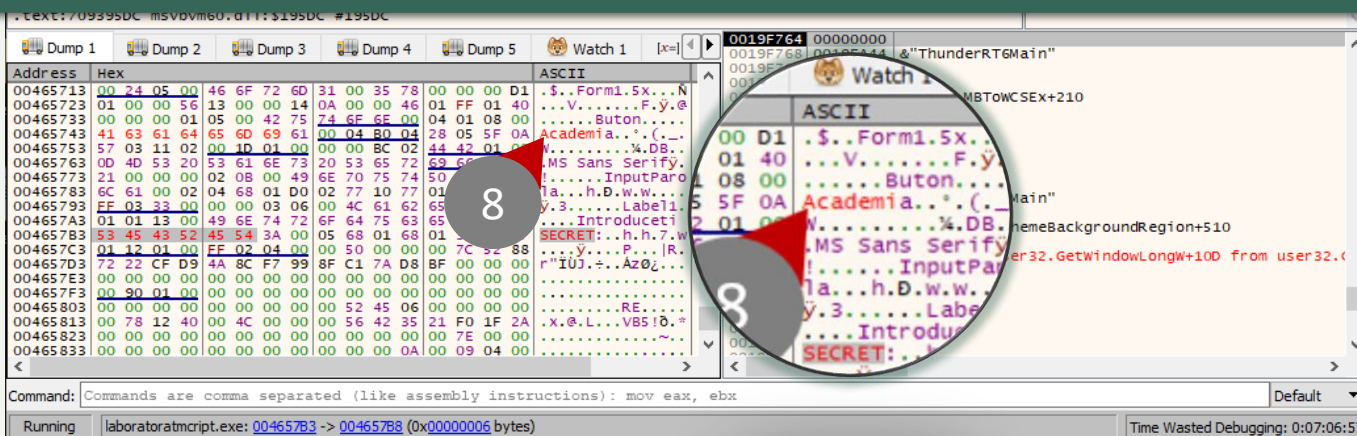




8. Confirmăm în Dump că stringul ASCII a fost schimbat corect — „Academia” apare în zona de memorie.

9. Aplicăm patch-ul final asupra fișierului executabil cu butonul Patch File.

- Interfața modificată afișează noul text pe buton: „Academia”.
- Comparație: versiunea veche conținea textul original „Verifica”.



9.4

MODIFICAREA STRINGURILOR IN MESAJE





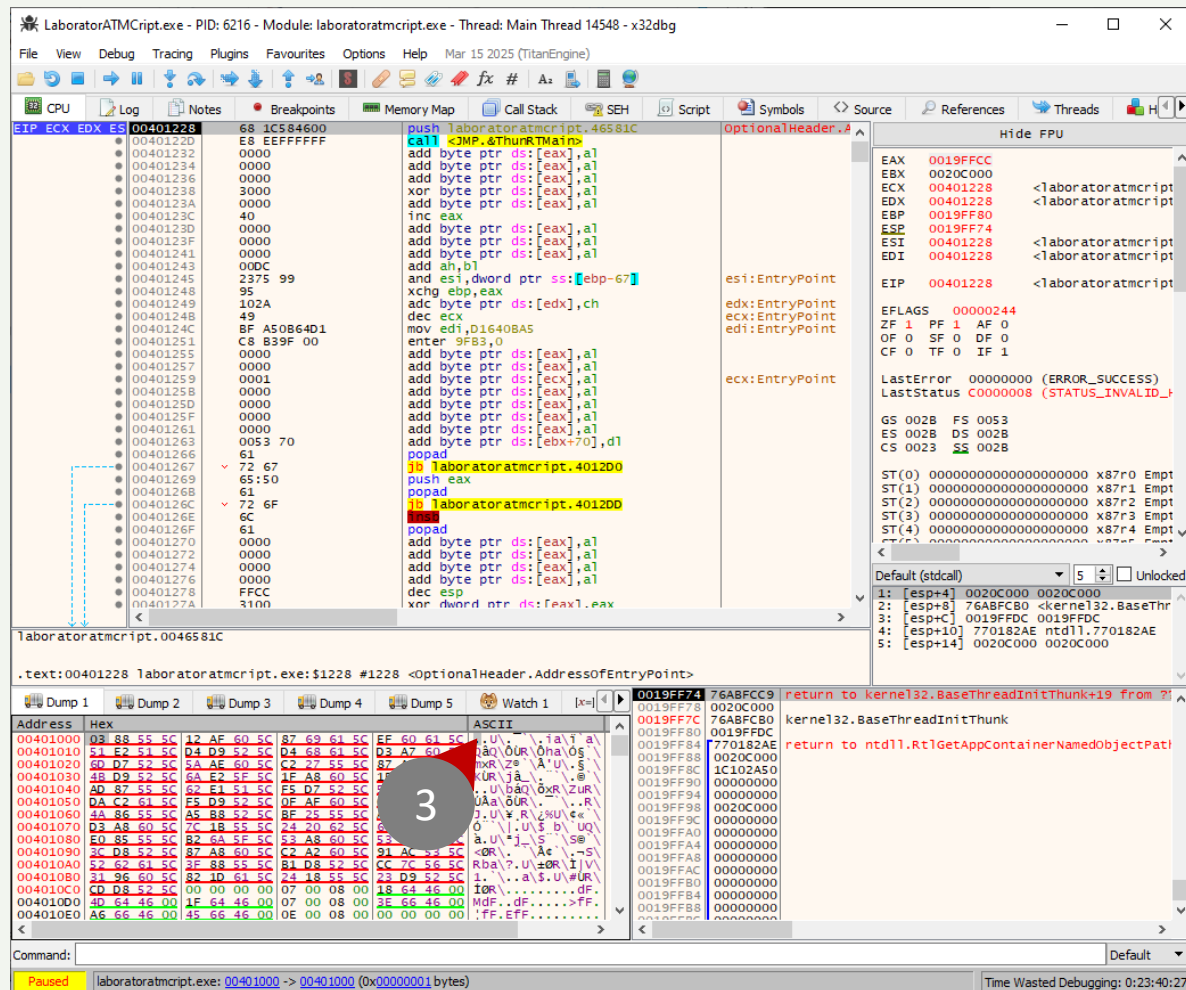
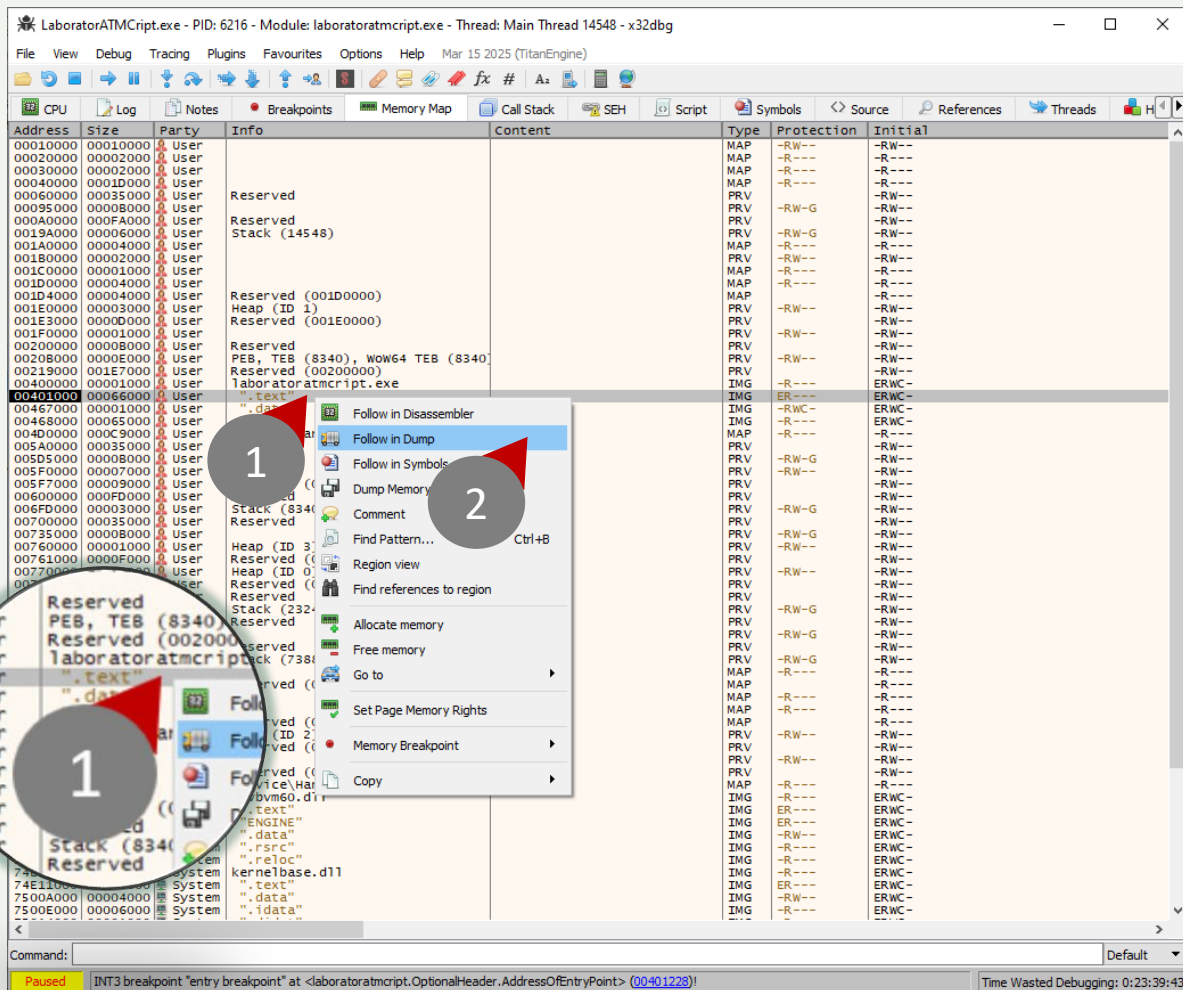
VERSIUNEA (I) CONTINUAM EXEMPLUL ANTERIOR ...

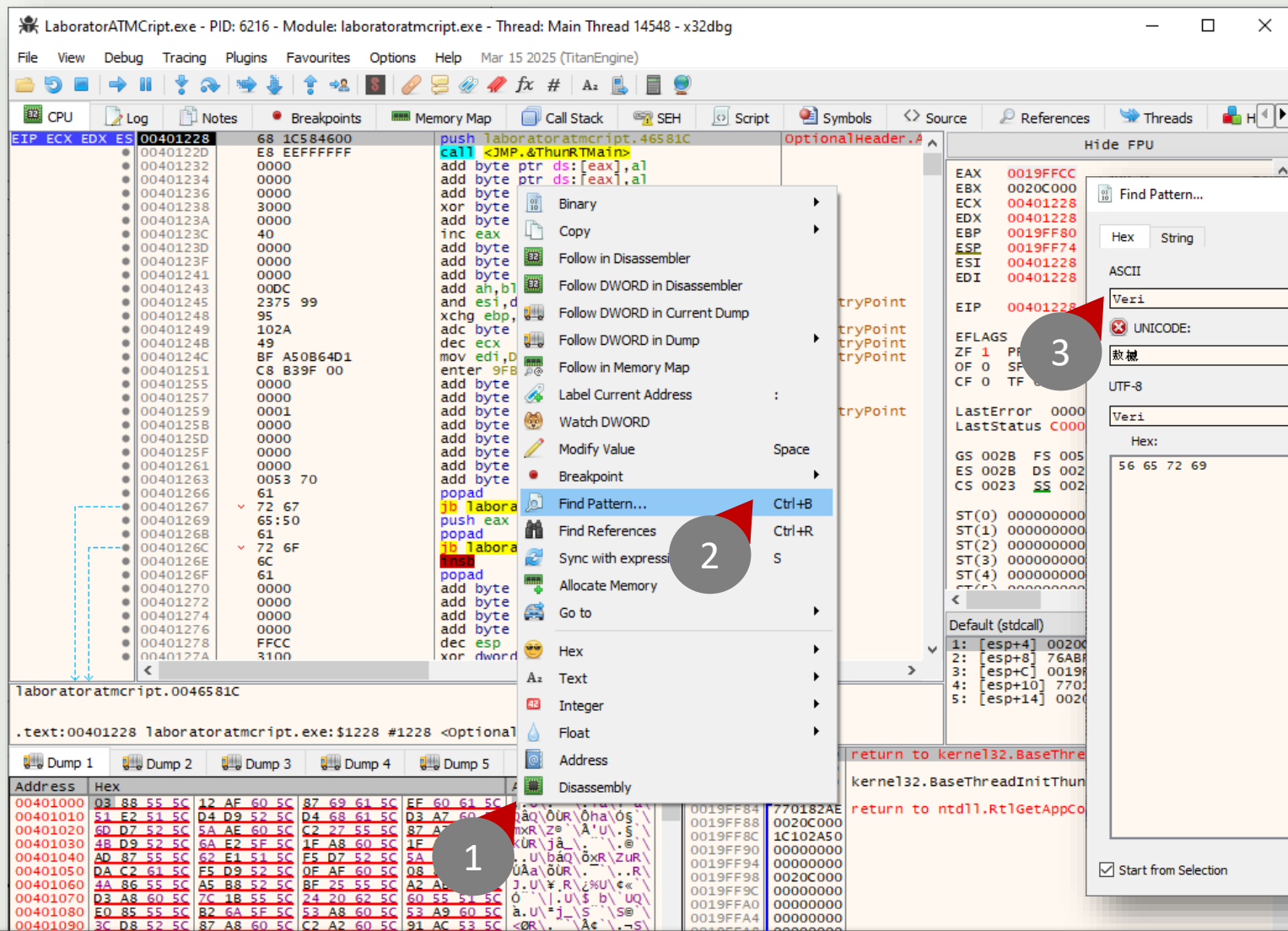
Metoda exploratorului !

În situațiile în care structura internă a unui program este necunoscută, iar indiciile sunt puține, intervine Metoda exploratorului. Această abordare nu urmează pași prestabiliți, ci presupune o navigare activă și atentă prin memorie, disassembler și dump, căutând vizual orice secvență recognoscibilă: un text, o semnătură, un fragment de cod. Scopul nu este identificarea imediată a țintei, ci declanșarea unui „declic” mental — acel moment în care o bucată de informație aparent banală face legătura cu o logică ascunsă. Această metodă dezvoltă instinctul analitic și oferă libertatea de a explora, exact ca un cercetător care descoperă sensuri noi într-un peisaj necunoscut.

Nota: Stringurile pot fi stocate și în alte regiuni de memorie, nu doar în cele clasice.

1. Se selectează secțiunea .text din Memory Map.
2. Click dreapta → Follow in Dump.
3. Dump-ul se deschide exact la începutul secțiunii .text, loc în care putem începe să căutăm pattern-uri.





1. Dăm click dreapta pe primul octet din zona Dump, corespunzător începutului secțiunii .text.
2. Se alege „Find Pattern...” pentru a căuta un string cunoscut.
3. Se scrie „Veri” în tabul String, iar hex-ul este generat automat.
5. Se apasă OK pentru a începe căutarea în .text.

Căutăm stringul "Veri" de pe buton în format ASCII, în speranța că în memoria executabilului — mai precis în zona secțiunii .text — acesta este stocat împreună cu alte stringuri hardcodate, precum textul afișat de MessageBoxA.

Localizarea exactă a offsetului unde este hardcodat stringul verificat, chiar dacă nu se află în .data/.rdata.

1. Adresa stringului „Veri” a fost localizată în memorie (pattern găsit).
2. Se face Follow in Dump pentru a analiza manual zona.

The screenshot shows the TitanEngine debugger window for 'LaboratorATMCript.exe'. The 'Pattern: 56657269' is entered in the search bar. The 'Follow in Dump' option is highlighted in the context menu. A red circle with the number '1' points to the search bar, and another red circle with the number '2' points to the 'Follow in Dump' option.

Address	Hex	ASCII
00465743	56 65 72 69 66 69 63 61 00 04 80 04 28 05 5F 0A	Verifica..%(. W.....%.DB..
00465753	57 03 11 02 00 1D 01 00 00 00 BC 02 44 42 01 00	MS Sans Serif. !.....InputParo
00465763	0D 4D 53 20 53 61 6E 73 20 53 65 72 69 66 FF 03	la..h.D.w.w... y.3.....Label1.
00465773	21 00 00 00 02 08 00 49 6E 70 75 74 50 61 72 6F	Introduceti parola:..h.h.7.w
00465783	6C 61 00 02 04 68 01 D0 02 77 10 77 01 12 00 00	P.....R. r"iUJ.÷.Äzö...
00465793	FF 03 33 00 00 00 03 06 00 4C 61 62 65 6C 31 00	RE..... x.@.L...VBS!D.*
004657A3	01 01 13 00 49 6E 74 72 6F 64 75 63 65 74 69 20	RE..... x.@.L...VBS!D.*
004657B3	70 61 72 6F 6C 61 3A 00 05 68 01 68 01 37 05 77	RE..... x.@.L...VBS!D.*
004657C3	01 12 01 00 FF 02 04 00 00 50 00 00 00 7C 52 88	RE..... x.@.L...VBS!D.*
004657D3	72 22 CF D9 4A 8C F7 99 8F C1 7A D8 BF 00 00 00	RE..... x.@.L...VBS!D.*
004657E3	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	RE..... x.@.L...VBS!D.*
004657F3	00 90 01 00 00 00 00 00 00 00 00 00 00 00 00	RE..... x.@.L...VBS!D.*
00465803	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	RE..... x.@.L...VBS!D.*
00465813	00 78 12 40 00 00 00 00 00 00 00 00 00 00 00	RE..... x.@.L...VBS!D.*
00465823	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	RE..... x.@.L...VBS!D.*

The screenshot shows the TitanEngine debugger window for 'LaboratorATMCript.exe'. The 'Dump' window is open, displaying the memory dump at address 00465743. The dump shows the string 'Verifica..%(.W.....%.DB..MS Sans Serif.!.InputParola..h.D.w.w...Label1.Introduceti parola:..h.h.7.w.....P.....R.r"iUJ.÷.Äzö...'. A red circle with the number '3' points to the 'Dump' window. The 'Command' field at the bottom shows 'laboratoratmcript.exe: 00465743 -> 00465743 (0x00000001 bytes)'. The 'Time Wasted Debugging' is 0:23:47:18.

3. Dump-ul este deschis în paralel cu zona de cod care folosește acel string — e o ancoră bună pentru identificare contextuală.

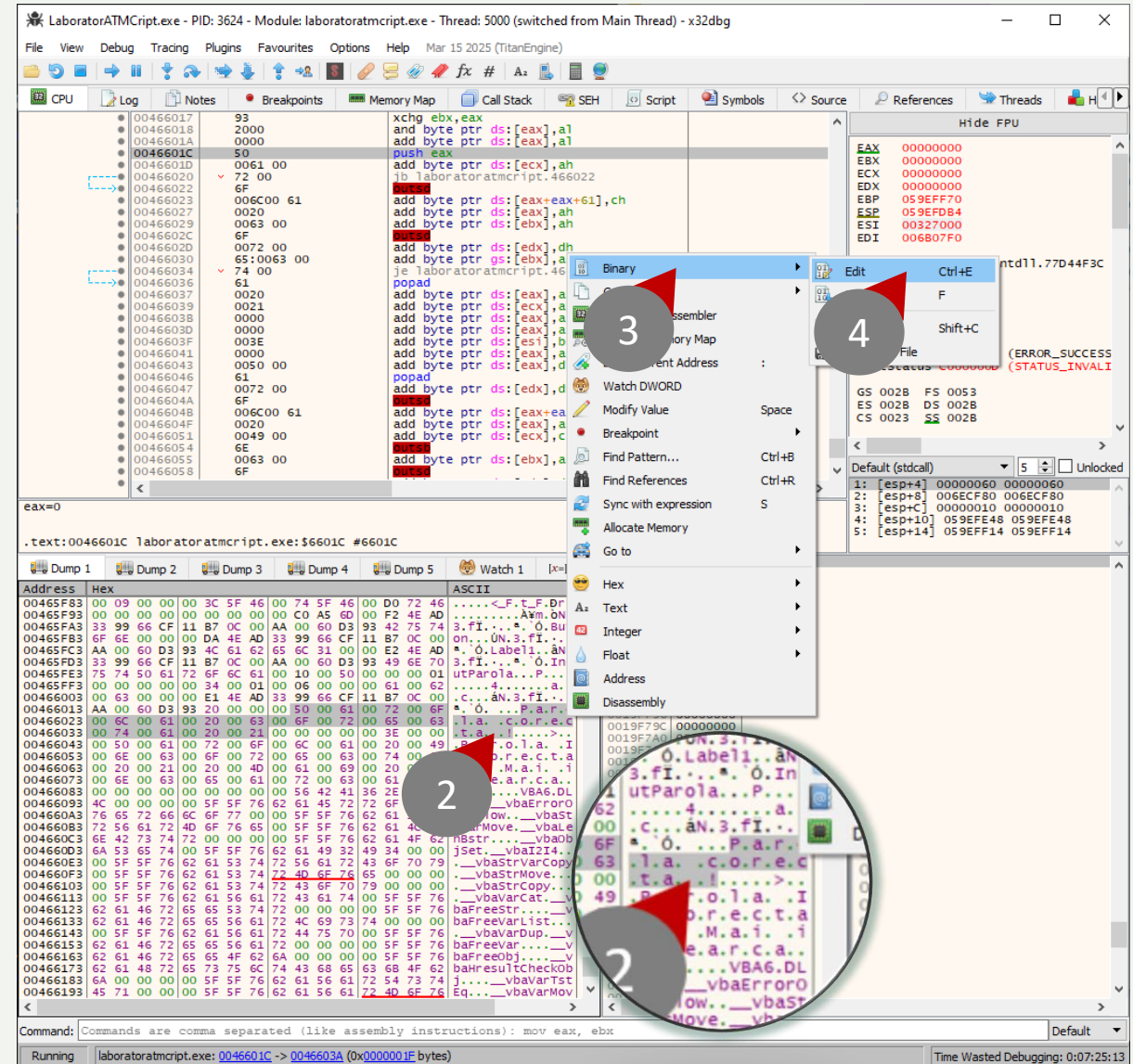
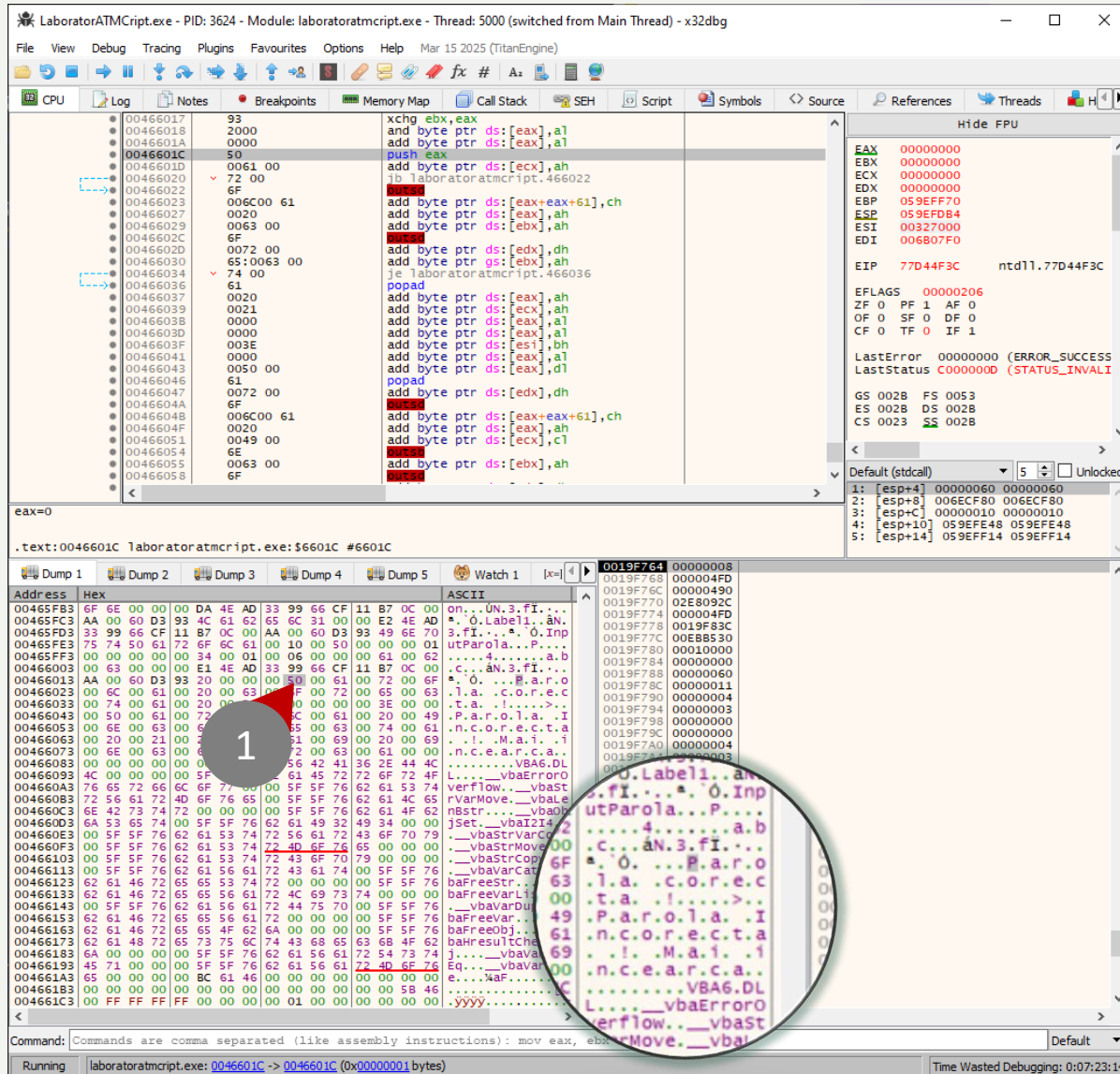
4. În fereastra de dump (video), se parcurge manual vecinătatea memoriei, căutând stringul din fereastra MessageBoxA ("Parola incorectă") — pentru a fi modificat ulterior.

1. Se identifică în dump stringul complet „Parola incorectă: Mai încerca”, confirmând prezența lui în memorie.

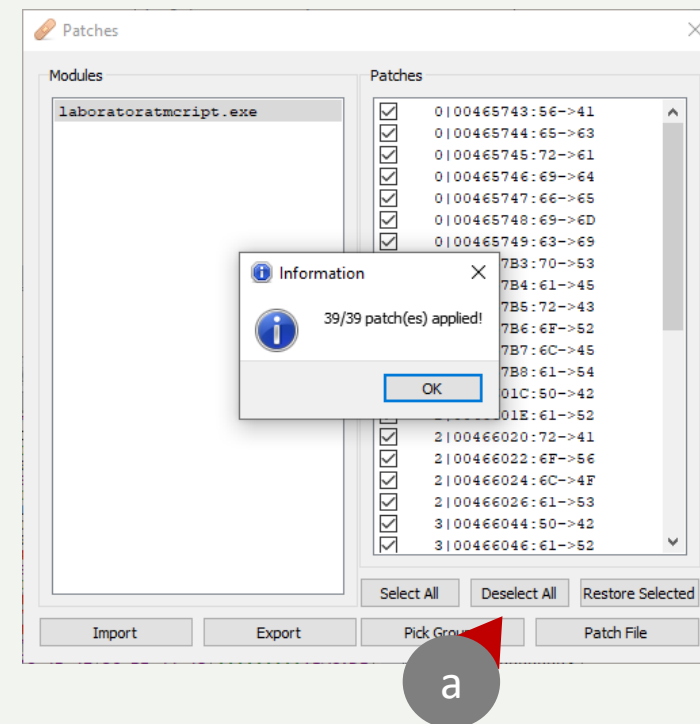
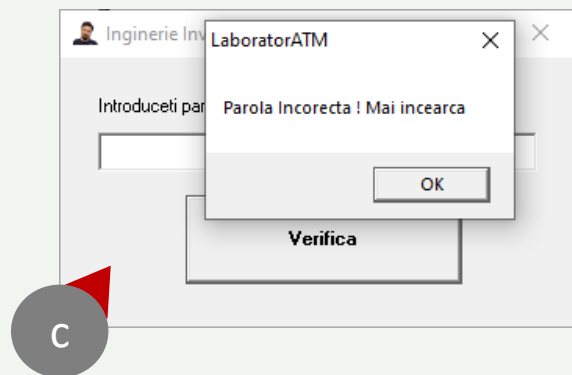
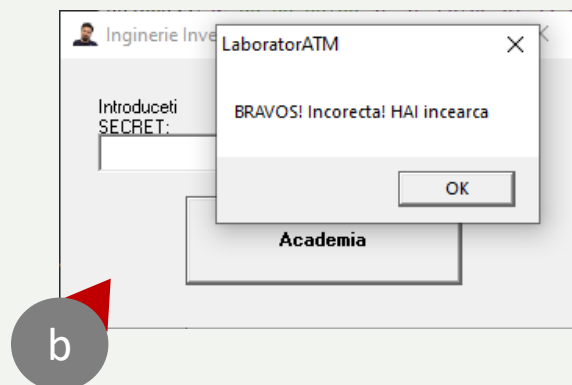
2. Se selectează în dump zona relevantă pentru editare.

3. Se face click dreapta pe selecție → Binary.

4. Se alege opțiunea Edit pentru a modifica mesajul afișat de MessageBoxA.



- ◆ În concluzie: prin editarea directă a stringurilor în dump și aplicarea patch-urilor din x32dbg, am reușit să personalizăm complet mesajele din interfața grafică a programului — fără a recompila codul sursă.



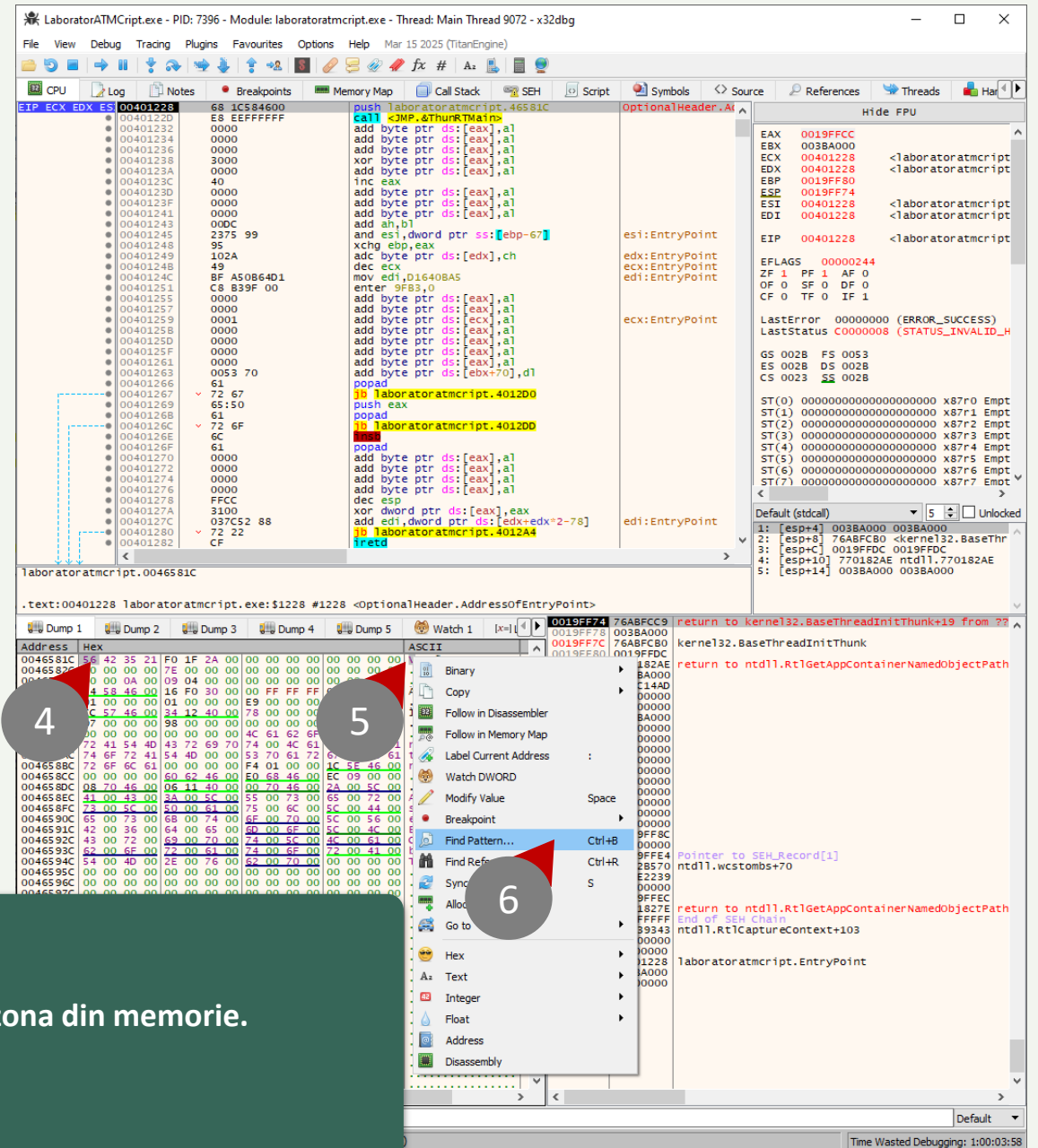
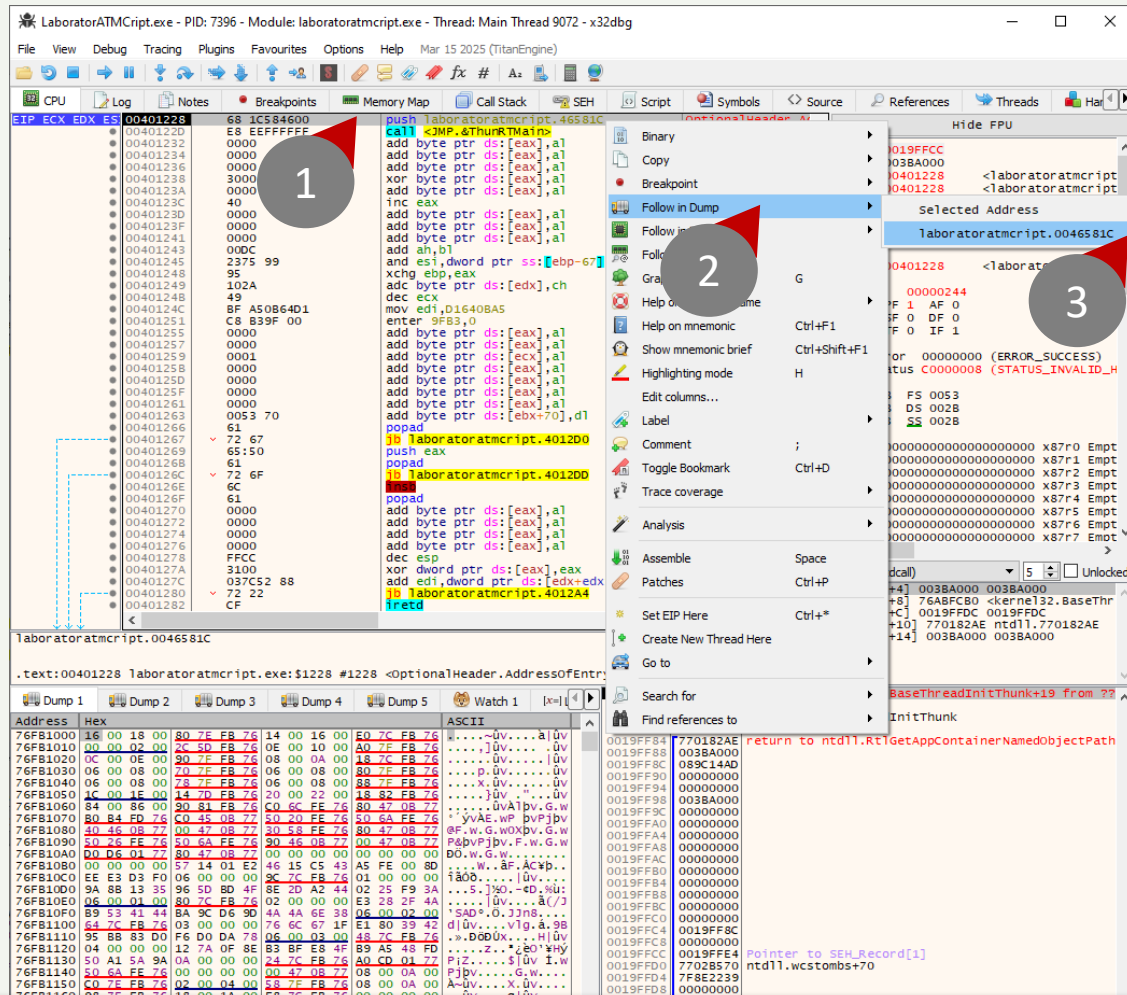
- Fereastra de patching confirmă aplicarea celor 39 de modificări în fișierul executabil laboratoratmcript.exe.
- Aplicația rulează cu stringul personalizat injectat: mesajul de eroare a fost înlocuit cu „BRAVOS! Incorectă! HAI încearcă”, afișat în MessageBox.
- Alternativ, într-o versiune diferită a binarului, apare „Parola Incorecta ! Mai incearca”, semn că patch-ul corespunde unei alte ediții/ramuri de test.

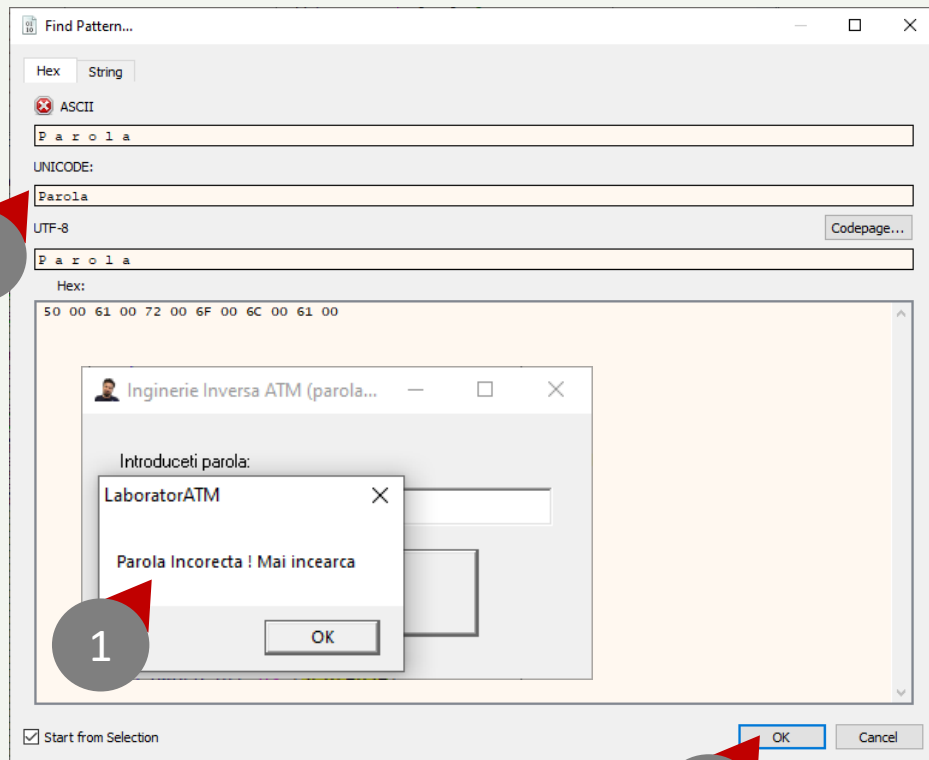


VERSIUNEA (II) CAUTAM SPECIFIC CE NE INTERESEAZA ...

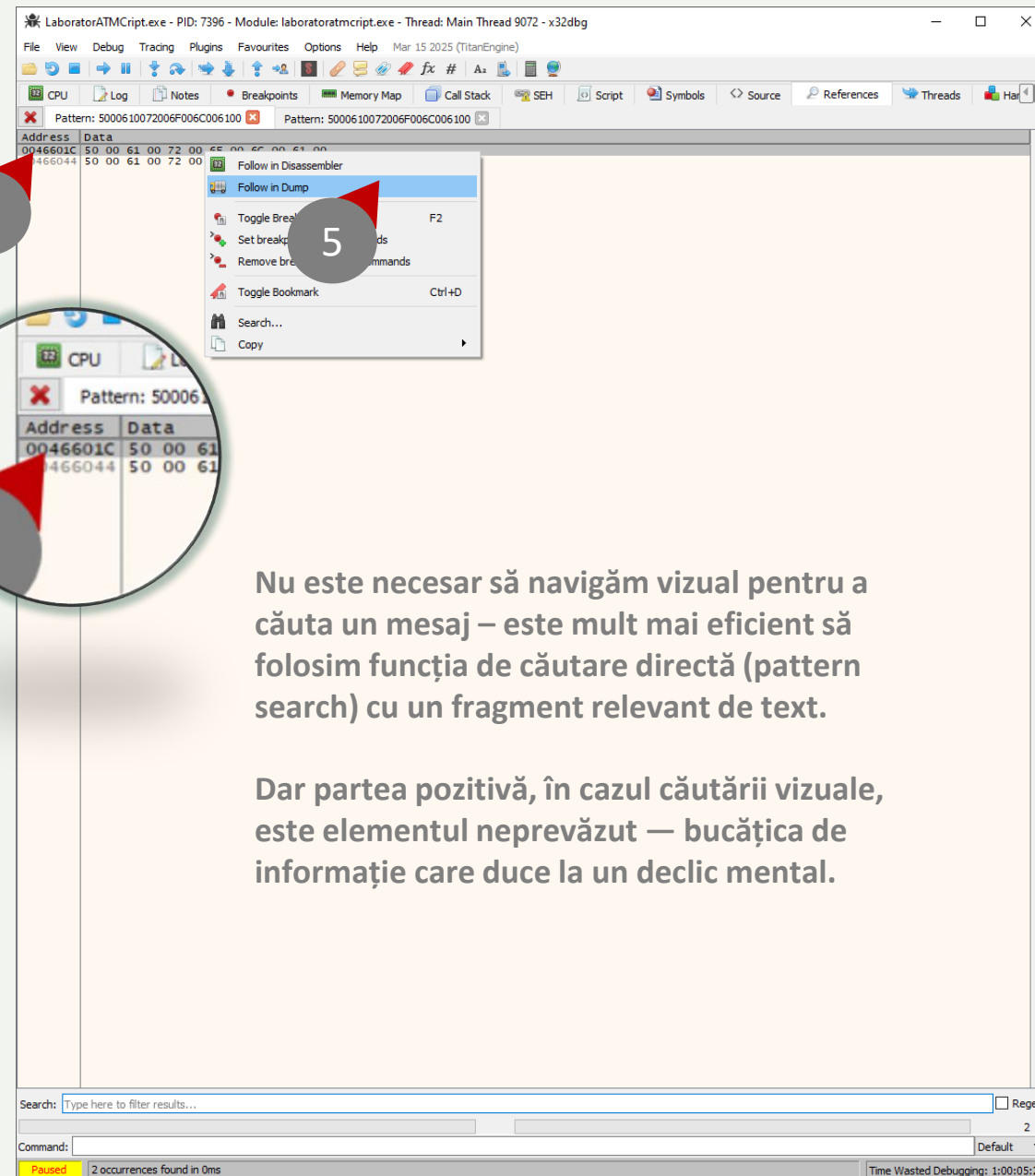
Metoda ob^e_uzului !

Atunci când fișierele sunt de dimensiuni mari, iar timpul de analiză este limitat, abordările exploratorii devin ineficiente (nu mai avem loc de explorări romantice prin dump). În aceste cazuri, se impune o strategie punctuală, directă, axată pe obiectivul imediat: identificarea rapidă a unui element cunoscut (de exemplu, un șir de caractere sau o instrucțiune specifică). Această abordare este denumită informal „metoda obezului” – o metaforă pentru situațiile în care nu este fezabil să se parcurgă întregul spațiu de memorie, ci este necesar să se ajungă imediat la esență. Se renunță astfel la parcurgerea vizuală, în favoarea unei căutări precise și automatizate. Este o metodă eficientă în contexte urgente, unde analiza trebuie să fie concentrată și rezultatele rapide.



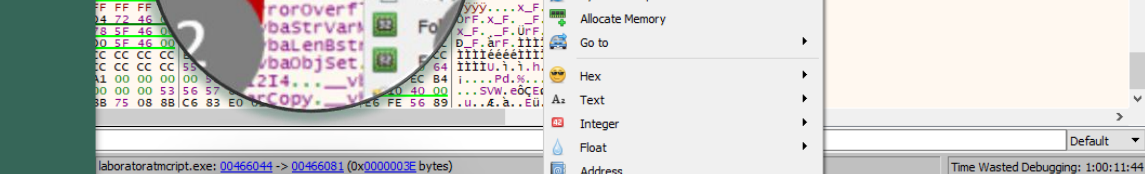
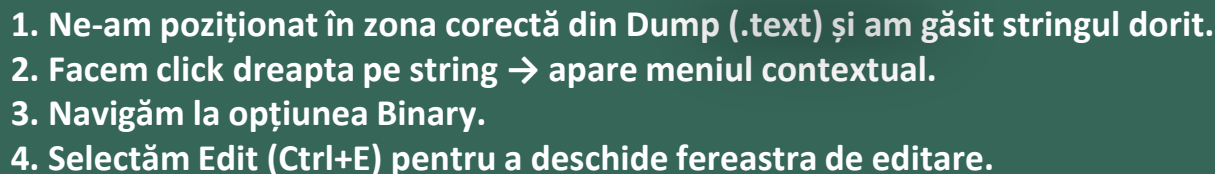


1. Observăm în aplicație mesajul afișat într-un MessageBox: „Parola Incorecta ! Mai incearca” – ne interesează să găsim acest text în fișierul executabil.
2. În fereastra „Find Pattern”, tastăm un cuvânt reprezentativ, de exemplu „Parola”, și bifăm opțiunea Start from selection (dacă e cazul).
3. Apăsăm OK pentru a începe căutarea în memorie.x32dbg afișează rezultatul căutării: adresa unde apare stringul este evidențiată.
4. Dăm click dreapta pe adresa respectivă și selectăm Follow in Dump → Follow in Dump pentru a inspecta și edita mai ușor zona cu stringul.

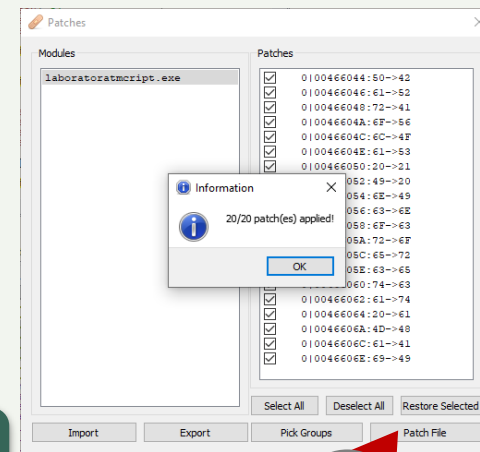
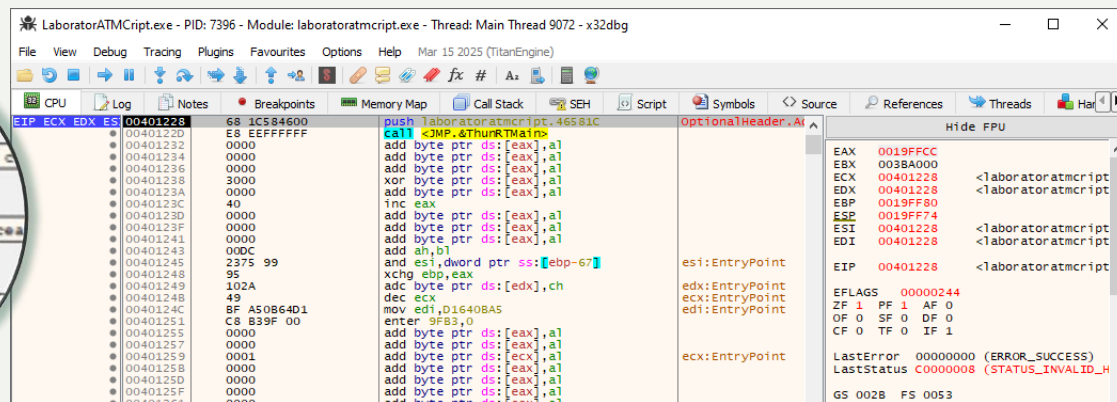
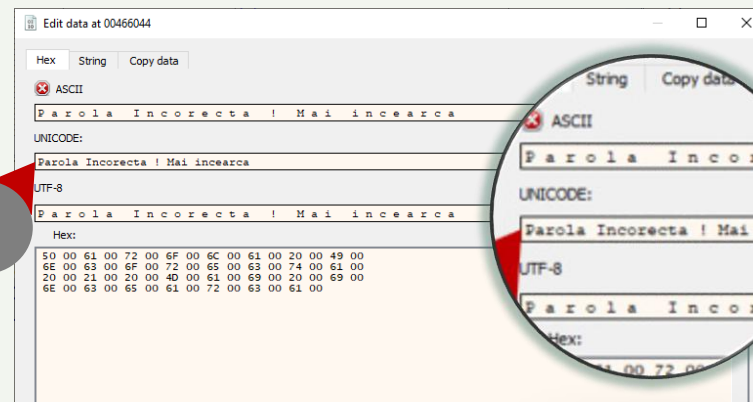


Nu este necesar să navigăm vizual pentru a căuta un mesaj – este mult mai eficient să folosim funcția de căutare directă (pattern search) cu un fragment relevant de text.

Dar partea pozitivă, în cazul căutării vizuale, este elementul neprevăzut — bucățica de informație care duce la un declic mental.



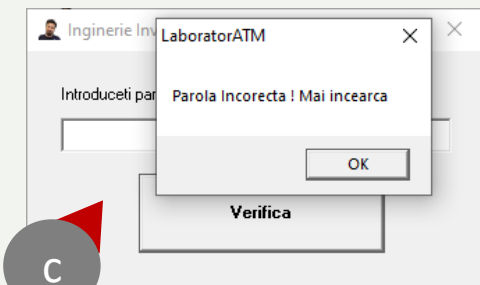
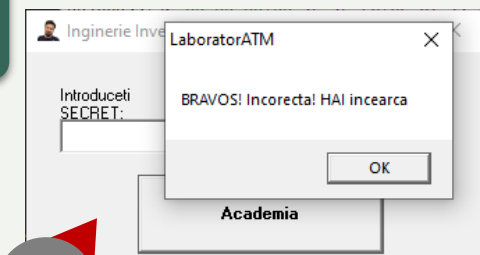
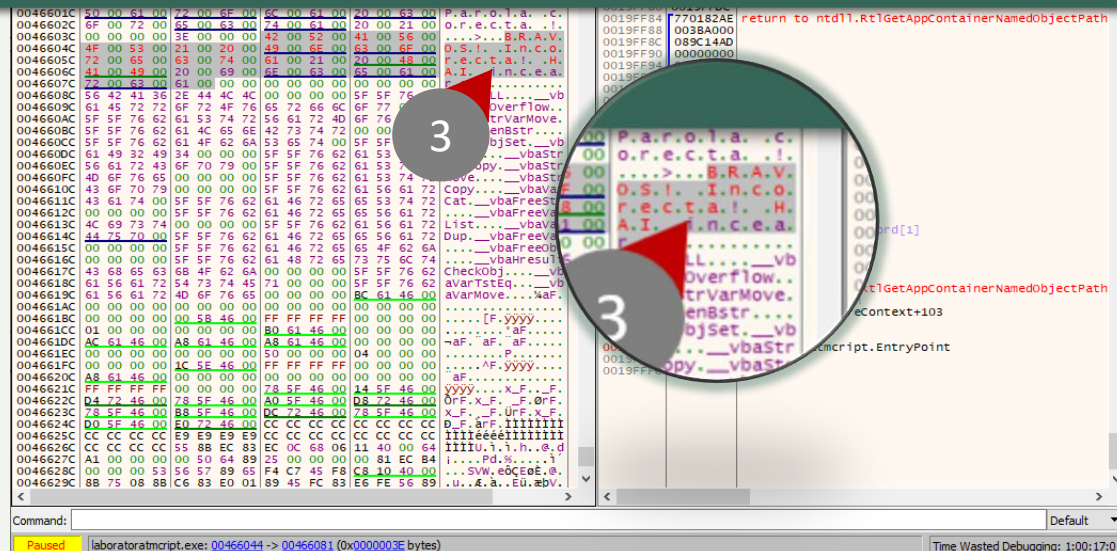
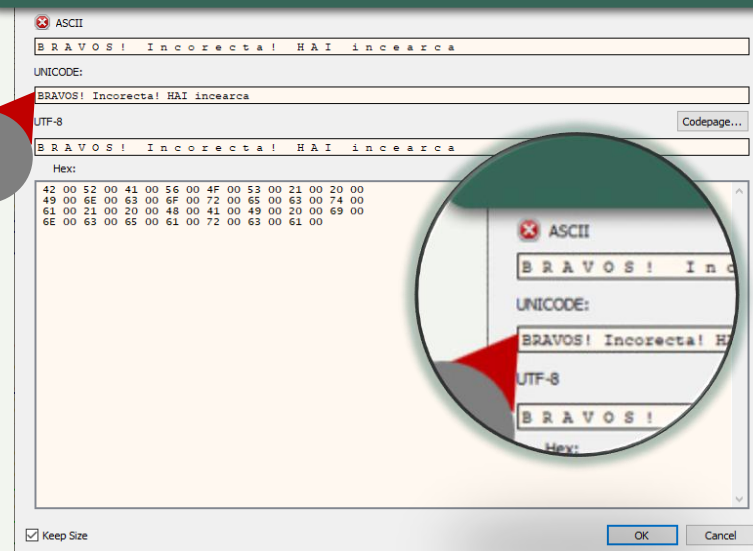
1



a

1. Se deschide fereastra de editare a stringului (1), unde avem mesajul original „Parola Incorecta ! Mai incearca”.
2. Se editează textul, spre exemplu în „BRAVOS! Incorecta! HAI incearca” fără a depăși lungimea (2).
3. Modificarea este imediat vizibilă în dump (3).
4. Se generează patch-ul cu noile date și se aplică toate modificările (a).
5. La rulare, aplicația arată noul mesaj modificat în MessageBox (b), înlocuind textul original (c).

2



c

9.5

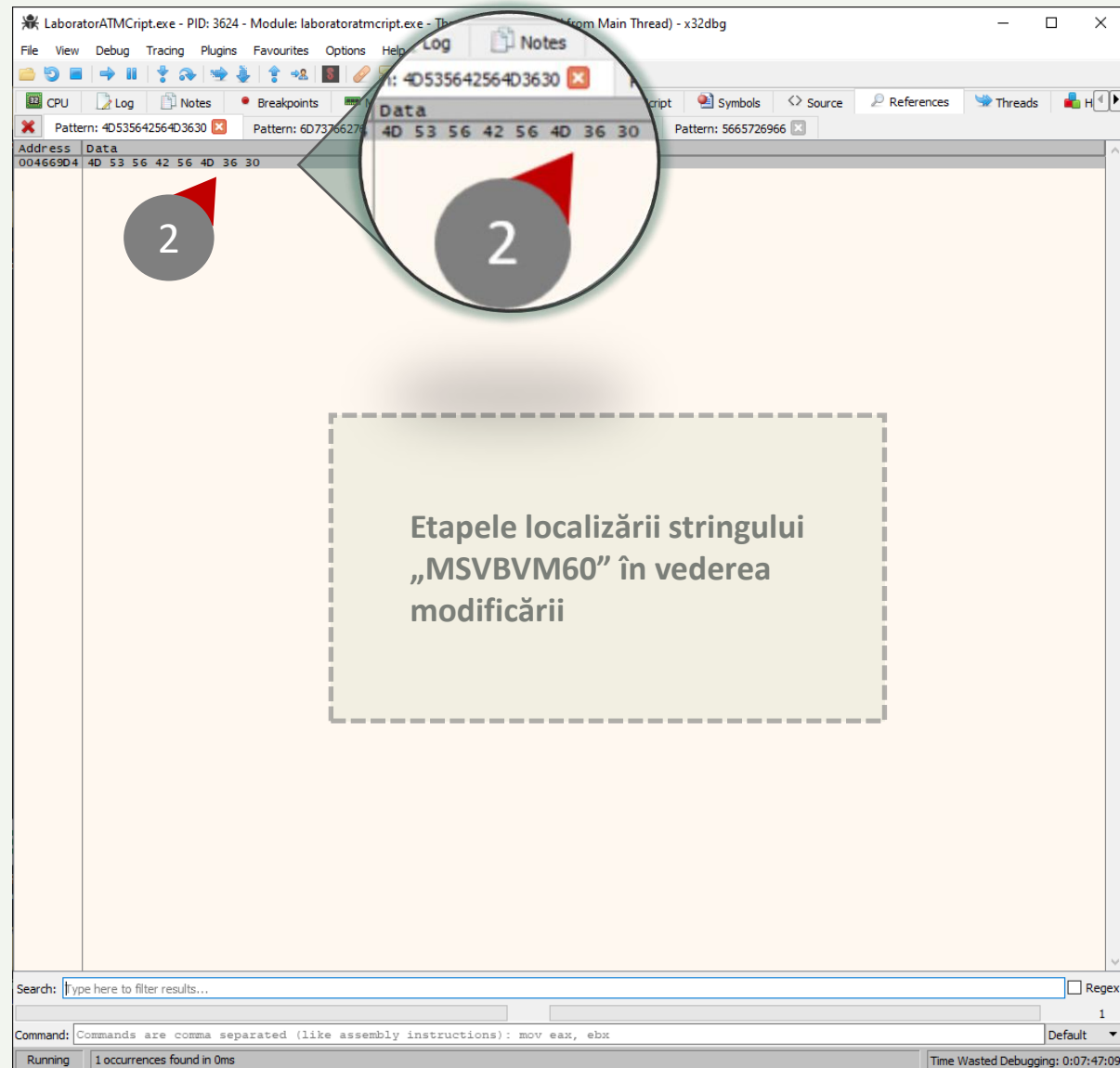
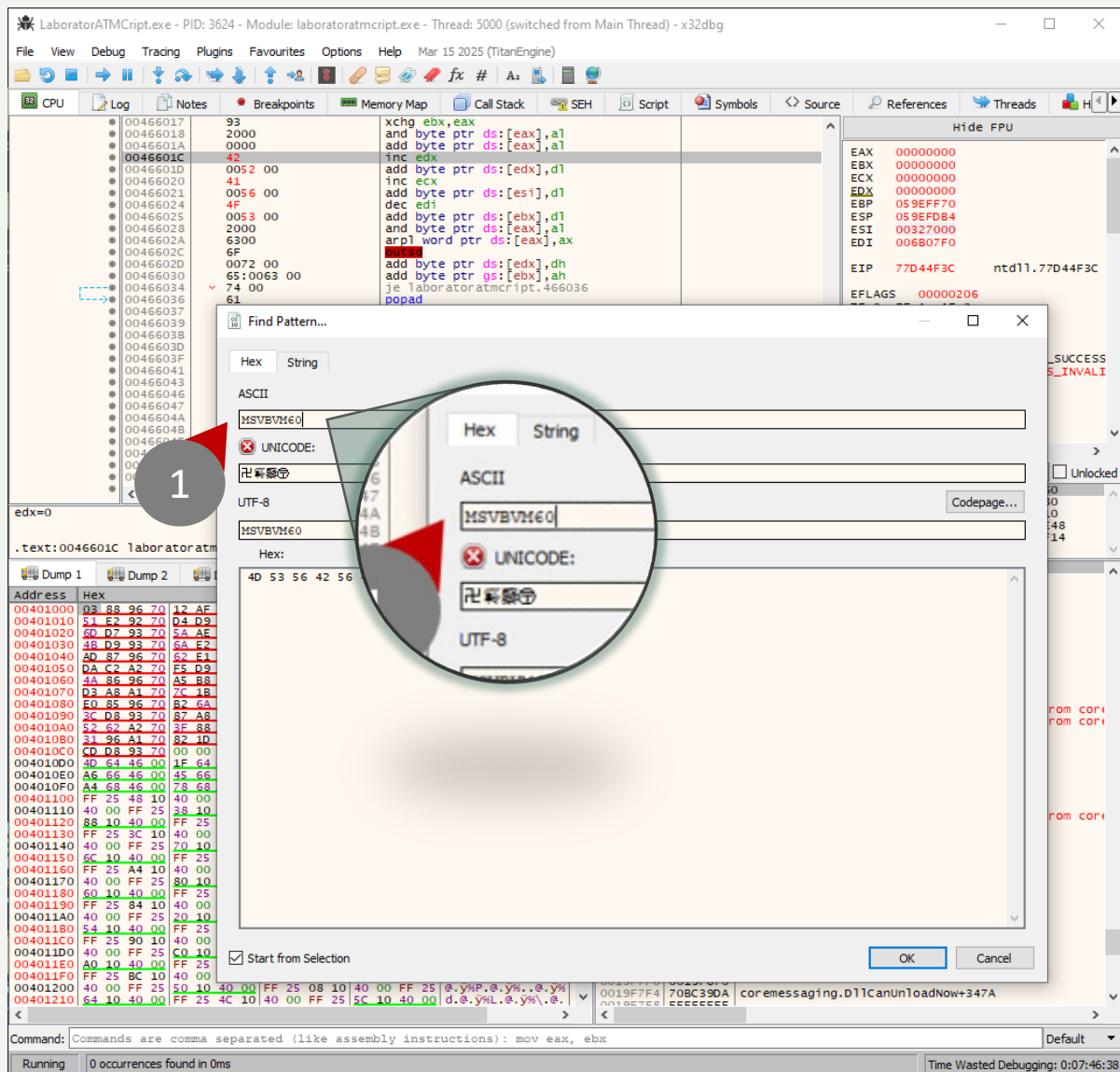
SCHIMBAREA MASINII VIRTUALE



CONTEXT INTRODUCTIV

- În anumite scenarii de analiză sau manipulare a executabilelor, poate deveni necesară modificarea numelui unui DLL dinamic legat static în binar, cum este cazul bine-cunoscutului `msvbvm60.dll`. Această tehnică poate avea diverse scopuri, printre care: Redirecționarea încărcării către o versiune personalizată a DLL-ului; Evitarea detectării de către un antivirus; Crearea unui mediu de test controlat; Pregătirea pentru injectarea unei biblioteci proprii (ex: `fakevbm60.dll`). Modificarea nu afectează direct comportamentul codului executabil, ci modul în care acesta își caută și încarcă bibliotecile externe.

1. Se deschide fereastra Find Pattern din meniul contextual, iar în câmpul ASCII se introduce șirul MSVBVM60 (vezi 1).
2. Acesta este numele bibliotecii dinamice (DLL) standard folosit de aplicațiile Visual Basic 6. După confirmarea căutării, șirul este găsit în memoria executabilului, afișat în zona de dump (2). Reprezentarea hexazecimală este: 4D 53 56 42 56 4D 36 30.



LaboratorATMCript.exe - PID: 3624 - Module: laboratoratmcript.exe - Thread: 5000 (switched from Main Thread) - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References

00466017 93 xchg ebx, eax
00466018 2000 and byte ptr ds:[eax], al
0046601A 0000 add byte ptr ds:[eax], al
0046601C 42 inc edx
0046601D 0052 00 add byte ptr ds:[edx], dl
00466020 41 inc ecx
00466021 0056 00 add byte ptr ds:[esi], dl
00466024 4F dec edi
00466025 0053 00 add byte ptr ds:[ebx], dl
00466028 2000 and byte ptr ds:[eax], al
0046602A 6300 arpl word ptr ds:[eax], ax
0046602C 6F outsd
0046602D 0072 00 add byte ptr ds:[edx], dh
00466030 65:0063 00 add byte ptr gs:[ebx], ah
00466034 74 00 je laboratoratmcript.466036
00466036 61 popad
00466037 0020 add byte ptr ds:[edx], dl
00466039 0021 add byte ptr ds:[edx], dl
0046603B 0000 add byte ptr ds:[edx], dl
0046603D 0000 add byte ptr ds:[edx], dl
0046603F 003E add byte ptr ds:[edx], dl
00466041 0000 add byte ptr ds:[edx], dl
00466043 0042 00 add byte ptr ds:[edx], dl
00466046 52 push edx
00466047 0041 00 add byte ptr ds:[edx], dl
0046604A 56 push esi

edx=0

.text:0046601C laboratoratmcript.exe:\$6601C #6601C

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch

Address	Hex	ASCII
004668A4	8B 35 C0 10 40 00 8D 4D	k. .pk.
004668B4	8D 4D D0 FF D6 C3 88 50	Ok. ak.
004668C4	89 02 5E 33 C0 64 89 00	.l. d. .
004668D4	C2 10 00 FF 15 84 10 00	>l. .Nl.
004668E4	0C 69 06 00 FF FF F6 00	t1. .~l.
004668F4	00 10 00 00 00 00 00 00	
00466904	00 00 00 00 00 00 00 00	
00466914	FA 69 06 00 0A 6A 5F 66	MSVBVM60.DLL
00466924	3C 6A 06 00 5F 66 66 66	...Cicos. .
00466934	6E 6A 06 00 86 6A 06 00	...tan. x. .vbav
00466944	A4 6A 06 00 86 6A 06 00	...Move. . .vbaf
00466954	D2 6A 06 00 E0 6A 06 00	...Var. .e. .vba
00466964	10 68 06 00 1E 68 06 00	...Bstr. .H. .vba
00466974	5A 68 06 00 70 68 06 00	...VarMove. . .vba
00466984	90 68 06 00 A4 68 06 00	...reeVarList. .
00466994	D2 68 06 00 E4 68 06 00	...dj_fdiv_m64. .
004669A4	18 6C 06 00 64 06 00 80	...dj_fprem1. .
004669B4	3E 6C 06 00 4E 6C 06 00	
004669C4	74 6C 06 00 7E 6C 06 00	
004669D4	4D 53 56 42 56 4D 36 30	MSVBVM60.DLL
004669E4	5F 43 49 63 6F 73 00 00	...Cicos. .
004669F4	70 74 61 6E 00 00 78 01	...tan. x. .vbav
00466A04	4D 6F 76 65 00 00 B1 0F	...Move. . .vbaf
00466A14	65 56 61 72 00 00 F9 00	...Var. .e. .vba
00466A24	42 73 74 72 00 00 48 01	...Bstr. .H. .vba
00466A34	56 61 72 4D 6F 76 65 00	...VarMove. . .vba
00466A44	72 65 56 61 72 4C 69 73	...reeVarList. .
00466A54	64 6A 5F 66 64 69 76 5F	...dj_fdiv_m64. .
00466A64	64 6A 5F 66 70 72 65 6D	...dj_fprem1. .

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Running laboratoratmcript.exe: 004669D4 -> 004669DB (0x00000008 bytes)

4

2

3

4

5

5

1

1. Se localizează stringul MSVBVM60 în dump.
2. Click dreapta pe string → Binary.
3. Se alege opțiunea Edit.
4. Se deschide editorul cu textul original.
5. Se înlocuiește cu un string de aceeași lungime, de exemplu: DE_CE_NU.

LaboratorATMCript.exe - PID: 3624 - Module: laboratoratmcript.exe - Thread: 5000 (switched from Main Thread) - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

00466017 93 xchg ebx, eax
 00466018 2000 and byte ptr ds:[eax], al
 0046601A 0000 add byte ptr ds:[eax], al
 0046601C 42 inc edx
 0046601D 0052 00 add byte ptr ds:[edx], dl
 00466020 41 inc ecx
 00466021 0056 00 add byte ptr ds:[esi], dl
 00466024 4F dec edi
 00466025 0053 00 add byte ptr ds:[ebx], dl

Hide FPU

EAX 00000000
 EBX 00000000
 ECX 00000000
 EDI 00000000
 EBP 059EFF70
 ESP 059EFDB4

LastError 00000000 (ERROR_SUCCESS)
 LastStatus C000000D (STATUS_INVALID_DEVICE_REQUEST)

Default (stdcall) 5 Unlocked

1: [esp+4] 00000060 00000060
 2: [esp+8] 006ECF80 006ECF80
 3: [esp+C] 00000010 00000010
 4: [esp+10] 059EFE48 059EFE48
 5: [esp+14] 059EFF14 059EFF14

edx=0

.text:0046601C laboratoratmcript.exe:\$6601C #6601C

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x=] 0019F764 0019F728

Address Hex ASCII

004668A4 8B 35 C0 10 40 00 8D 4D E0 FF D6 8D 4D DC FF D6 .SA.@.Mayo.Muyo
 004668B4 8D 4D D0 FF D6 C3 8B 55 14 88 45 E4 88 4D EC 5F .MdyOA.U..Ea. 00
 004668C4 89 02 5E 33 C0 64 89 0D 00 00 00 00 58 88 E5 5D .A3Ad. 00
 004668D4 C2 10 00 FF 15 84 10 40 00 90 90 90 9E 9E 9E 9E .A.y..@. 06 00
 004668E4 0C 69 06 00 FF FF FF FF FF FF FF FF D4 69 06 00 .i..yyyyyy 06 00
 004668F4 00 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00466904 00 00 00 00 00 00 00 00 E2 69 06 00 EC 69 06 00a
 00466914 FA 69 06 00 0A 6A 06 00 1A 6A 06 00 2A 6A 06 00 .i..j..
 00466924 3C 6A 06 00 50 6A 06 00 04 02 00 80 60 6A 06 00 <.Pj..
 00466934 6E 6A 06 00 86 6A 06 00 96 6A 06 00 53 02 00 80 n].j..
 00466944 A4 6A 06 00 B6 6A 06 00 C8 6A 06 00 77 02 00 80 h].j..
 00466954 D2 6A 06 00 E0 6A 06 00 F4 6A 06 00 4A 68 06 00 Oj..aj..
 00466964 10 68 06 00 1E 68 06 00 34 68 06 00 3E 68 06 00 .k..k..k..
 00466974 5A 68 06 00 70 68 06 00 7E 68 06 00 60 02 00 80 Zk..pk..~
 00466984 90 68 06 00 A4 68 06 00 7E 68 06 00 BC 68 06 00 .k..k..k..
 00466994 D2 68 06 00 E4 68 06 00 7E 68 06 00 06 6C 06 00 .k..k..k..
 004669A4 18 6C 06 00 64 06 00 80 26 6C 06 00 34 6C 06 00 .l..d..&1..4
 004669B4 3E 6C 06 00 4E 6C 06 00 60 6C 06 00 6A 6C 06 00 >.l..~1..1..j
 004669C4 74 6C 06 00 7E 6C 06 00 8E 6C 06 00 00 6C 06 00 t1..~1..1..1..
 004669D4 44 45 5F 43 45 5F 4E 55 2E 44 4C 4C 00 00 53 00 DE_CE_NU.DLL..S
 004669E4 5F 43 49 63 6F 73 00 00 B3 01 5F 61 64 6A 5F 6A .Cicos..adi f
 004669F4 70 74 61 6E 00 00 78 01 5F 5F 76 62 61 56 6A 5F .ptan..x..vbaVar
 00466A04 4D 6F 76 65 00 00 B1 00 5F 5F 76 62 61 56 6A 5F .Move..±..vbaFre
 00466A14 65 56 61 72 00 00 E9 00 5F 5F 76 62 61 56 6A 5F .eVar..ē..vbaLen
 00466A24 42 73 74 72 00 00 48 01 5F 5F 76 62 61 56 6A 5F .Bstr..H..vbaStr
 00466A34 56 61 72 4D 6F 76 65 00 B2 00 5F 5F 76 62 61 56 .VarMove..±..vbaF
 00466A44 72 65 65 56 61 72 4C 69 73 74 00 00 7A 01 5F 61 .reeVarList..a
 00466A54 64 6A 5F 66 64 69 76 5F 6D 36 34 00 B2 01 5F 61 dj_fdiv_m64..a
 00466A64 64 6A 5F 66 70 72 65 6D 31 00 C0 00 5F 5F 76 62 d1_fpremi1.A..vbi

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Running laboratoratmcript.exe: 004669D4 -> 004669D8 (0x00000008 bytes)

Time Wasted Debugging: 0:07:50:27

1. Se localizează stringul în .text („MSVBVM60” în dump) → Find Pattern + vizualizare directă.
2. Se modifică în alt nume de aceeași lungime (ex: DE_CE_NU) → se aplică patch-ul.

Patches

Modules

laboratoratmcript.exe

Patches

0100465743:56->41
 0100465744:65->63
 0100465745:72->61
 0100465746:69->64
 0100465747:66->65
 0100465748:69->6D
 0100465749:63->69
 7B3:70->53
 7B4:61->46
 7B5:72->43
 7B6:6F->52
 7B7:6C->45
 7B8:61->64
 01C:50->42
 01E:61->52
 2100466020:72->41
 2100466022:6F->56
 2100466024:6C->4F
 2100466026:61->53
 3100466044:50->42
 3100466046:61->52

Information

47/47 patch(es) applied!

OK

Select All Deselect All Restore Selected

Import Export Pick Groups Patch File

2



de_ce_nu.dll



test3.exe

Fișierul executabil modificat (test3.exe) va căuta acum biblioteca „DE_CE_NU.dll” în loc de „msvbvm60.dll”.

1. În primul executabil (test2.exe), vedem că s-a încărcat vechiul msvbvm60.dll.
2. În al doilea executabil (test3.exe), vedem că s-a încărcat noul de_ce_nu.dll.

test2.exe - PID: 3824 - Module: test2.exe - Thread: Main Thread 7308 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

Address	Size	Party	Info	Content	Type	Protection	Initial
001D0000	00001000	User			MAP	-R---	-R---
001E0000	00004000	User			MAP	-R---	-R---
001E4000	00004000	User			MAP	-R---	-R---
001F0000	00001000	User			PRV	-RW--	-RW--
00200000	0000E000	User	Reserved (00200000)		PRV	-RW--	-RW--
00280000	0000E000	User	PEB, TEB (7308), WOW64 TEB (7308)		PRV	-RW--	-RW--
00290000	00167000	User	Reserved (00290000)		PRV	-RW--	-RW--
00400000	00001000	User	test2.exe		PRV	-RW--	-RW--
00401000	00006000	User	".text"		IMG	-R---	ERWC
00407000	00001000	User	".data"		IMG	ER---	ERWC
00408000	00005000	User	".rsrc"		IMG	-RW-	ERWC
00409000	0000C9000	User	Device\HarddiskVolume2\Windows\...		PRV	-RW-	ERWC
005A0000	00035000	User	Reserved		PRV	-RW-	-RW-
005D5000	00008000	User	Reserved		PRV	-RW-G	-RW-
005E0000	00035000	User	Reserved		PRV	-RW-	-RW-
00615000	00008000	User	Reserved		PRV	-RW-G	-RW-
00620000	00020000	User	Heap (ID 0)		PRV	-RW-	-RW-
00640000	000E0000	User	Reserved (00620000)		PRV	-RW-	-RW-
00720000	000F0000	User	Reserved		PRV	-RW-	-RW-
0081D000	00003000	User	Stack (13740)		PRV	-RW-G	-RW-
00820000	000F0000	User	Reserved		PRV	-RW-	-RW-
0091D000	00003000	User	Stack (7032)		PRV	-RW-G	-RW-
00920000	00035000	User	Reserved		PRV	-RW-	-RW-
00955000	00008000	User	Reserved		PRV	-RW-G	-RW-
00965000	000F0000	User	Reserved		PRV	-RW-	-RW-
00A50000	00003000	User	Stack (3908)		PRV	-RW-	-RW-
00A60000	00013000	User	Reserved		MAP	-R---	-R---
00A73000	001ED000	User	Reserved (00A60000)		MAP	-R---	-R---
00C60000	00181000	User	Reserved		MAP	-R---	-R---
00DF0000	000F1000	User	Reserved (00DF0000)		MAP	-R---	-R---
00EE1000	01310000	User	Reserved		MAP	-R---	-R---
02260000	00001000	User	Heap (ID 3)		PRV	-RW-	-RW-
02261000	0000F000	User	Reserved (02260000)		PRV	-RW-	-RW-
02280000	00002000	User	Heap (ID 2)		PRV	-RW-	-RW-
02282000	0000E000	User	Reserved (02280000)		PRV	-RW-	-RW-
02360000	00003000	User	Heap (ID 1)		PRV	-RW-	-RW-
02363000	0000D000	User	Reserved (02360000)		PRV	-RW-	-RW-
02370000	00010000	User	Reserved		PRV	-RW-	-RW-
02380000	003F0000	User	Reserved (02370000)		PRV	-RW-	-RW-
6FBE0000	00001000	System	msvbvm60.dll		IMG	-R---	ERWC
6FBE1000	00105000	System	".text"		IMG	ER---	ERWC
6FCE6000	0000E000	System	".ENGINE"		IMG	-RW-	ERWC
6FCF4000	00008000	System	".data"		IMG	-RW-	ERWC
6FCFC000	00031000	System	".rsrc"		IMG	-RW-	ERWC
6FD20000	00010000	System	".reloc"		IMG	-RW-	ERWC
74E10000	00001000	System	kernelbase.dll		IMG	-R---	ERWC
74E11000	001F9000	System	".text"		IMG	ER---	ERWC
750A0000	00004000	System	".data"		IMG	-RW-	ERWC
750A1000	00006000	System	".idata"		IMG	-RW-	ERWC
75014000	00001000	System	".didat"		IMG	-RW-	ERWC
75015000	00001000	System	".rsrc"		IMG	-RW-	ERWC
75016000	00032000	System	".reloc"		IMG	-RW-	ERWC
750A0000	00001000	System	win32u.dll		IMG	-R---	ERWC
750A1000	00006000	System	".text"		IMG	ER---	ERWC
750A7000	0000E000	System	".data"		IMG	-RW-	ERWC
750B5000	00001000	System	".data"		IMG	-RW-	ERWC
750B6000	00001000	System	".rsrc"		IMG	-RW-	ERWC
750B7000	00001000	System	".reloc"		IMG	-RW-	ERWC
750C0000	00001000	System	rpcrt4.dll		IMG	-R---	ERWC
750C1000	000A0000	System	".text"		IMG	-RW-	ERWC
75168000	00001000	System	".data"		IMG	-RW-	ERWC
7516C000	00003000	System	".idata"		IMG	-RW-	ERWC
7516F000	00001000	System	".didat"		IMG	-RW-	ERWC
75170000	0000E000	System	".rsrc"		IMG	-RW-	ERWC
75175000	00007000	System	".reloc"		IMG	-RW-	ERWC
75190000	00001000	System	msvc_p_w.in.dll		IMG	-R---	ERWC
75191000	0006E000	System	".text"		IMG	ER---	ERWC
751FF000	00003000	System	".data"		IMG	-RW-	ERWC
75202000	00002000	System	".idata"		IMG	-RW-	ERWC
75204000	00001000	System	".didat"		IMG	-RW-	ERWC
75205000	00001000	System	".rsrc"		IMG	-RW-	ERWC
75206000	00001000	System	".reloc"		IMG	-RW-	ERWC

Command: [Default]

Paused INT3 breakpoint "entry breakpoint" at <test2.OptionalHeader.AddressOfEntryPoint> (00401228)!

Time Wasted Debugging: 0:17:15

test3.exe - PID: 11196 - Module: test3.exe - Thread: Main Thread 10628 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads H

Address	Size	Party	Info	Content	Type	Protection	Initial
001D7000	00009000	User	Reserved (001D0000)		PRV	-RW-	-RW-
001E0000	00004000	User	Reserved (001E0000)		MAP	-R---	-R---
001E4000	00004000	User	Reserved (001E0000)		MAP	-R---	-R---
001F0000	00001000	User	Reserved		PRV	-RW-	-RW-
00200000	00180000	User	Reserved		PRV	-RW-	-RW-
00380000	00014000	User	Reserved		PRV	-RW-	-RW-
00394000	0000C000	User	Reserved (00200000)		PRV	-RW-	-RW-
00401000	00001000	User	test3.exe		PRV	-RW-	-RW-
00407000	00006000	User	".text"		IMG	-R---	ERWC
00408000	00001000	User	".data"		IMG	ER---	ERWC
00409000	00005000	User	".rsrc"		IMG	-RW-	ERWC
0040A000	0000C9000	User	Device\HarddiskVolume2\Windows\...		PRV	-RW-	ERWC
005A0000	00035000	User	Reserved		PRV	-RW-	-RW-
005D5000	00008000	User	Reserved		PRV	-RW-G	-RW-
005E0000	00035000	User	Reserved		PRV	-RW-	-RW-
00615000	00008000	User	Reserved		PRV	-RW-G	-RW-
00620000	00020000	User	Reserved		PRV	-RW-	-RW-
0066A000	00006000	User	Reserved (00660000)		PRV	-RW-	-RW-
00670000	0003C000	User	Heap (ID 0)		PRV	-RW-	-RW-
006AC000	000C4000	User	Reserved (00670000)		PRV	-RW-	-RW-
00770000	000FC000	User	Reserved		PRV	-RW-	-RW-
0086C000	00004000	User	Stack (5792)		PRV	-RW-G	-RW-
00870000	000FC000	User	Reserved		PRV	-RW-	-RW-
0096C000	00004000	User	Stack (10736)		PRV	-RW-G	-RW-
00970000	000FC000	User	Reserved		PRV	-RW-	-RW-
00A6C000	00004000	User	Stack (6248)		PRV	-RW-G	-RW-
00A70000	00013000	User	Reserved		MAP	-R---	-R---
00A83000	001ED000	User	Reserved (00A70000)		MAP	-R---	-R---
00C70000	00181000	User	Reserved		MAP	-R---	-R---
00E00000	00005000	User	Reserved (00E00000)		MAP	-R---	-R---
00ED5000	0132C000	User	Reserved		MAP	-R---	-R---
02210000	00001000	User	Reserved		PRV	-RW-	-RW-
02220000	00068000	User	Device\HarddiskVolume2\Users\Pa...		MAP	-RW-	-RW-
02288000	00018000	User	Reserved (02220000)		MAP	-RW-	-RW-
022A0000	00004000	User	Reserved		MAP	-RW-	-RW-
022B0000	00005000	User	Device\HarddiskVolume2\Windows\...		MAP	-RW-	-RW-
022C0000	00002000	User	Heap (ID 2)		PRV	-RW-	-RW-
022C2000	0000E000	User	Reserved (022C0000)		PRV	-RW-	-RW-
022E0000	0000B000	User	Heap (ID 1)		PRV	-RW-	-RW-
022E8000	00005000	User	Reserved (022E0000)		PRV	-RW-	-RW-
022F0000	00010000	User	Reserved (022F0000)		PRV	-RW-	-RW-
02300000	003F0000	User	Device\HarddiskVolume2\Windows\...		PRV	-RW-	-RW-
026F0000	00338000	User	Reserved		MAP	-R---	-R---
02A30000	00001000	User	Reserved (02A30000)		PRV	-RW-	-RW-
02A31000	0007F000	User	Reserved		PRV	-RW-	-RW-
02A80000	00035000	User	Reserved		PRV	-RW-	-RW-
02AE5000	00008000	User	Reserved		PRV	-RW-G	-RW-
02B20000	00003000	User	Heap (ID 4)		PRV	-RW-	-RW-
02B30000	0000B000	User	Reserved (02B20000)		PRV	-RW-	-RW-
02B30000	00035000	User	Reserved		PRV	-RW-	-RW-
02B65000	00008000	User	Reserved		PRV	-RW-G	-RW-
02B80000	0000B000	User	Heap (ID 3)		PRV	-RW-	-RW-
02B80000	00003000	User	Reserved (02B80000)		PRV	-RW-	-RW-
02B80000	00004000	User	Reserved		PRV	-RW-	-RW-
02B80000	000FC000	User	Reserved (02B80000)		PRV	-RW-	-RW-
02D50000	00001000	User	Heap (ID 5)		PRV	-RW-	-RW-
02D51000	0000F000	User	Reserved (02D50000)		PRV	-RW-	-RW-
02D60000	00006000	User	Reserved		PRV	-RW-	-RW-
02D66000	003FA000	User	Reserved (02D60000)		PRV	-RW-	-RW-
03160000	000E2000	User	Reserved		PRV	-RW-	-RW-
03242000	0091E000	User	Reserved (03160000)		PRV	-RW-	-RW-
03B60000	00001000	User	Reserved (03B60000)		PRV	-RW-	-RW-
03B61000	000FF000	User	Reserved		PRV	-RW-	-RW-
03C60000	01260000	User	Device\HarddiskVolume2\...		PRV	-RW-	-RW-
05CD0000	00001000	User	Reserved (05CD0000)		PRV	-RW-	-RW-
05CD1000	007FF000	User	Reserved		PRV	-RW-	-RW-
06400000	000FD000	User	Reserved		PRV	-RW-	-RW-
06500000	00003000	User	Stack (6820)		PRV	-RW-G	-RW-
066CD000	000FD000	User	Reserved		PRV	-RW-	-RW-
066CD000	00003000	User	Stack (10048)		PRV	-RW-G	-RW-
70180000	00001000	System	sxs.dll		IMG	-R---	ERWC
70181000	0007A000	System	".text"		IMG	ER---	ERWC
701F8000	00001000	System	".data"		IMG	-RW-	ERWC
701FC000	00002000	System	".idata"		IMG	-RW-	ERWC
701FE000	00001000	System	".didat"		IMG	-RW-	ERWC
701FF000	00001000	System	".rsrc"		IMG	-RW-	ERWC
70200000	00007000	System	".reloc"		IMG	-RW-	ERWC
70880000	00001000	System	textshaping.dll		IMG	-R---	ERWC
70881000	00090000	System	".text"		IMG	ER---	ERWC
70911000	00001000	System	".data"		IMG	-RW-	ERWC
70912000	00001000	System	".idata"		IMG	-RW-	ERWC
70913000	00002000	System	".reloc"		IMG	-RW-	ERWC
70920000	00001000	User	de_ce_nu.dll		IMG	-R---	ERWC
70921000	00105000	User	".text"		IMG	ER---	ERWC
70A36000	0000E000	User	".ENGINE"		IMG	-RW-	ERWC
70A34000	00008000	User	".data"		IMG	-RW-	ERWC
70A3C000	00031000	User	".rsrc"		IMG	-RW-	ERWC
70A3D000	00003000	User	".reloc"		IMG	-RW-	ERWC
70A80000	00001000	User	wintypes.dll		IMG	-R---	ERWC
70A81000	0008F000	User	".text"		IMG	ER---	ERWC
70A84000	00002000	User	".data"		IMG	-RW-	ERWC
70A82000	00002000	User	".idata"		IMG	-RW-	ERWC
70A44000	00001000	User	".didat"		IMG	-RW-	ERWC
70845000	00001000	System	".rsrc"		IMG	-RW-	ERWC
70846000	00017000	System	".reloc"		IMG	-RW-	ERWC
70860000	00001000	System	coremessaging.dll		IMG	-R---	ERWC

Command: [Default]

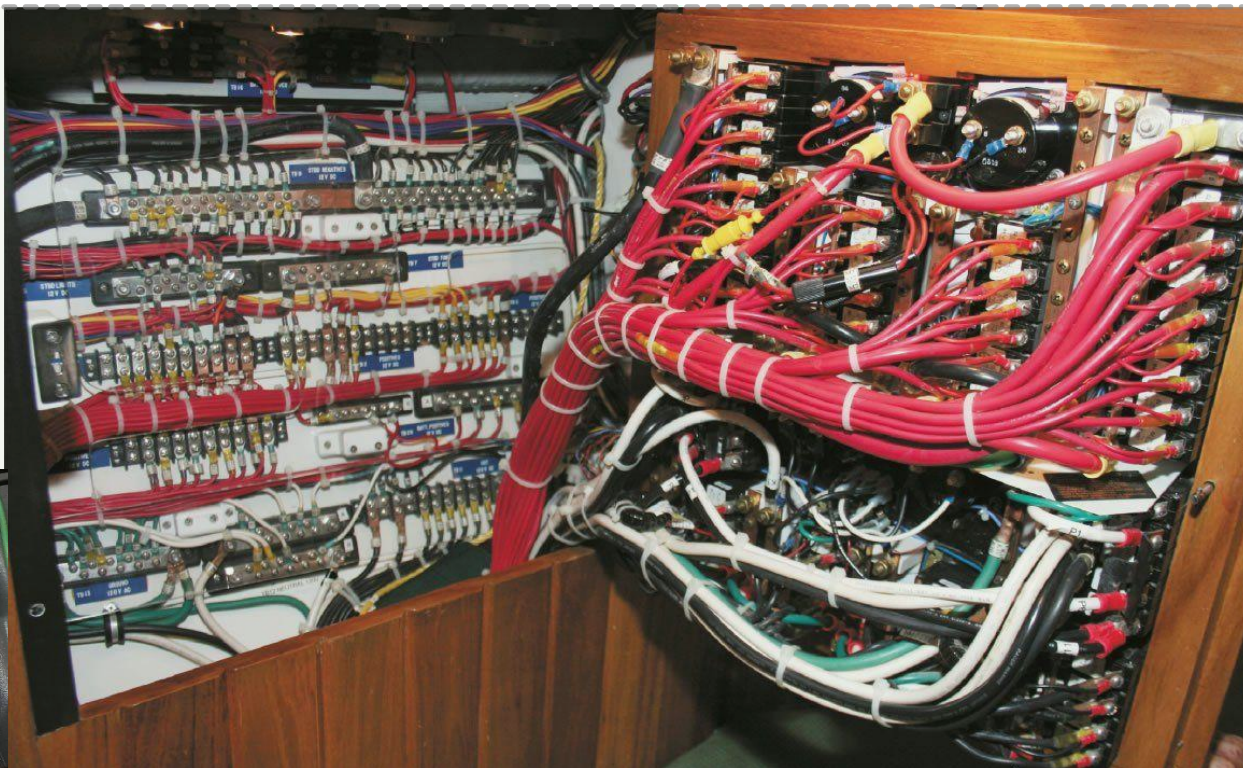
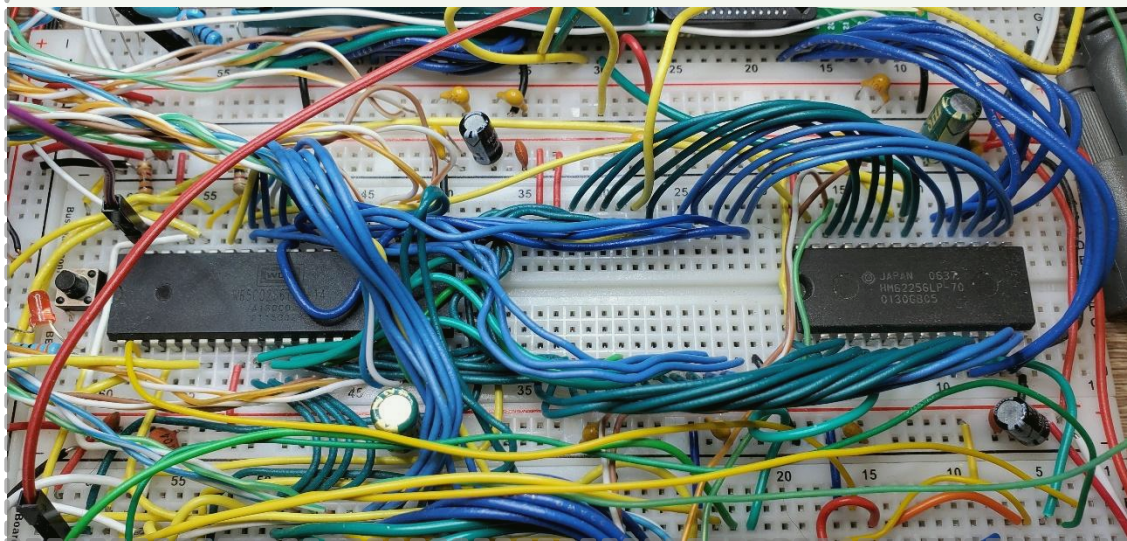
Running Thread 10048 created, Entry: ntldr.72005A30, Parameter: 005892B0

Time Wasted Debugging: 0:09:11:45

PROGRAMARE SOFTWARE DIRECT IN DEBUGGER? SIGUR CA DA !



- În acest exemplu demonstrativ, relocăm o bucată de cod executabil din secțiunea .code (.text) într-o zonă liberă din secțiunea .data, care în mod normal nu este executabilă. Duplicăm codul, schimbăm permisiunile de memorie pentru a permite execuția și redirectionăm fluxul printr-un jmp. Această tehnică evidențiază posibilitatea rulării de cod din zone non-standard, un mecanism utilizat frecvent în malware sau tehnici avansate de ofuscare.

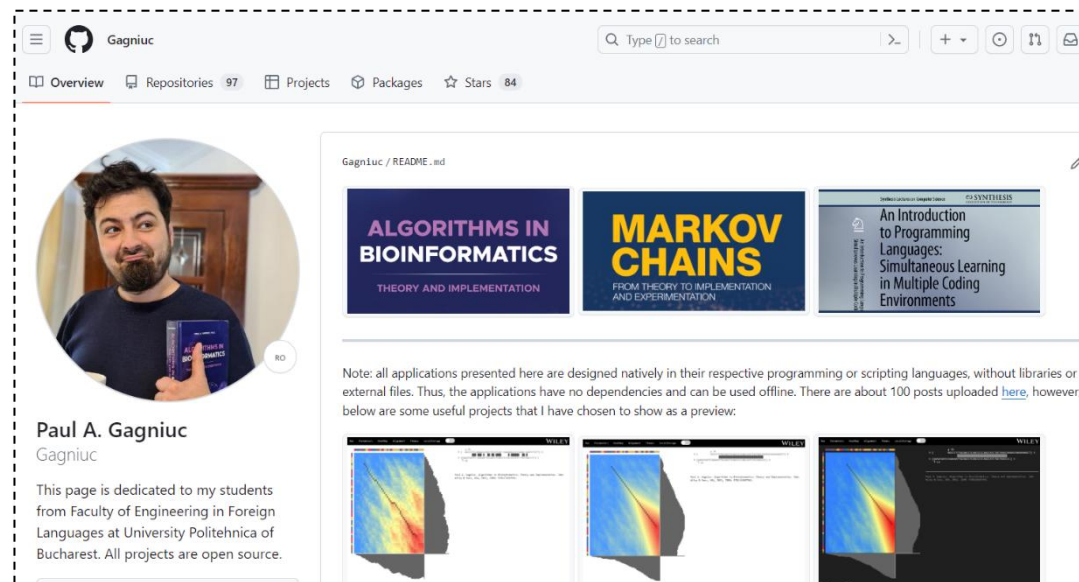


- În acest exemplu demonstrativ, relocăm o bucată de cod executabil din secțiunea .code (.text) într-o zonă liberă din secțiunea .data, care în mod normal nu este executabilă. Duplicăm codul, schimbăm permisiunile de memorie pentru

BIBLIOGRAFIE / RESURSE

- Paul A. Gagniuc. *Antivirus Engines: From Methods to Innovations, Design, and Applications*. Cambridge, MA: Elsevier Syngress, 2024. pp. 1-656.
- Paul A. Gagniuc. *An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments. Synthesis Lectures on Computer Science*. Springer International Publishing, 2023, pp. 1-280.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in MATLAB*, Springer, 2024, pp. 1-255.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Python*, Springer, 2024, pp. 1-245.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Javascript*, Springer, 2024, pp. 1-240.
- Paul A. Gagniuc. *Markov chains: from theory to implementation and experimentation*. Hoboken, NJ, John Wiley & Sons, USA, 2017, ISBN: 978-1-119-38755-8.

<https://github.com/gagniuc>



Gagniuc

Overview Repositories 97 Projects Packages Stars 84

Gagniuc / README.md

ALGORITHMS IN BIOINFORMATICS
THEORY AND IMPLEMENTATION

MARKOV CHAINS
FROM THEORY TO IMPLEMENTATION AND EXPERIMENTATION

An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments

Note: all applications presented here are designed natively in their respective programming or scripting languages, without libraries or external files. Thus, the applications have no dependencies and can be used offline. There are about 100 posts uploaded [here](#), however, below are some useful projects that I have chosen to show as a preview.

Paul A. Gagniuc
Gagniuc

This page is dedicated to my students from Faculty of Engineering in Foreign Languages at University Politehnica of Bucharest. All projects are open source.